

MFJA Third-Floor ROS 2 and Gazebo Project

Architecture, Data Flow, Operations, and Maintenance Handover Manual

A source-traceable reference for architecture, interfaces, runtime data flow, operation, testing, and long-term maintenance.

Manual revision	1.0
Document date	28 August 2026
Repository branch	INTERNSHIP-ALI-2026
Documented commit	aadf967b1188
Commit timestamp	2026-08-28T02:16:47+02:00
Supported platform	Ubuntu 24.04 LTS; ROS 2 Jazzy; Gazebo Harmonic (gz-sim 8)

Prepared as the authoritative engineering handover for future developers, operators, and maintainers.

Document Information

How this manual is controlled

This manual is versioned against repository commit `aadf967b1188`. It records the architecture and maintenance rules but does not replace the code. Exact message fields, launch defaults, and configuration values remain authoritative in the repository paths named throughout the manual. Refresh the Repository Source Index and revise the affected chapter after a public change.

Table 1: Document control and maintenance rules

Item	Control rule
Document owner	The maintainers declared in the four ROS package manifests.
Primary language	English. Historical Arabic/French explanatory artifacts are supplemental and are not runtime sources.
Scope	The four ROS 2 packages, their public interfaces, launch/configuration surface, simulation assets, control and research software, tests, and operational lifecycle.
Excluded as source	Colcon output, caches, temporary files, locally recorded datasets, external checkpoints, presentation binaries, and generated evidence. They are classified in this manual but must not be edited as product source.
Update trigger	Any changed package dependency, executable, launch argument, message/service, topic, parameter, configuration schema, safety gate, supported command, or required operational step.
Required release checks	Refresh the Repository Source Index, compile the PDF, check references/overflow, inspect the diff, and run the change-appropriate project tests.

Maintainer Contacts in Package Metadata

All four package manifests name Olivier Stasse, Diane Bury, and Ali Ibrahim as maintainers, with contact addresses stored in each `package.xml`. The exact authoritative paths are `mfja_rail_interfaces/package.xml`, `mfja_3rd_floor_description/package.xml`, `mfja_robot_control_config/package.xml`, and `mfja_3rd_floor_bringup/package.xml`. Update these manifests first when ownership changes; do not maintain a divergent contact list only in this PDF.

Revision History

Table 2: Manual revision history

Revision	Date	Change
1.0	2026-08-28	Initial comprehensive architecture, operation, maintenance, and continuation handover at commit <code>aadf967b1188</code> .

Executive Summary

The repository implements a ROS 2 Jazzy and Gazebo Harmonic digital twin of the MFJA third floor. Its most detailed cell is Room 315: two directed rail loops, up to eight identity-bearing shuttles, four industrial robots, optional TIAGo variants, paired RGB-D cameras, typed rail interfaces, deterministic safety supervision, and an optional V4 visual-state plus PlanSys2 language-to-motion research path. The four ROS packages form a deliberately layered system. Their concise ownership map is given in Table 3; detailed dependency and installation rules are kept in Chapters 2 and 4.

Table 3: Executive package and source-path map

Package path	Responsibility and authoritative package files
mfja_rail_interfaces	Typed rail messages and the shuttle lifecycle service. Authority: mfja_rail_interfaces/package.xml; mfja_rail_interfaces/CMakeLists.txt.
mfja_3rd_floor_description	Worlds, Gazebo models, simulation plugins, URDF descriptions, and structural tests. Authority: mfja_3rd_floor_description/package.xml; mfja_3rd_floor_description/CMakeLists.txt.
mfja_robot_control_config	Robot, rail, perception, dataset, planning, and execution logic. Authority: mfja_robot_control_config/package.xml; mfja_robot_control_config/CMakeLists.txt.
mfja_3rd_floor_bringup	Stable user-facing launch entry points and shared composition. Authority: mfja_3rd_floor_bringup/package.xml; mfja_3rd_floor_bringup/CMakeLists.txt.

The base simulation needs no dataset or learned checkpoint. Learned V4 inference and closed-loop execution require separate, immutable, checksum-verified artifacts and remain disabled by default. The relevant defaults and guards are defined in mfja_robot_control_config/config/room_315_vla/visual_state_runtime.yaml and mfja_robot_control_config/config/room_315_vla/task_execution_runtime.yaml.

Non-negotiable scope boundary

This is simulation and research software. It is not a safety-certified controller for physical equipment. Rail motion is kinematic, collision protection is software-only, robot/gripper motion is primarily visual, and the current V4 authorization is Gazebo-only. A physical deployment requires a new hardware interface, trusted presence/device sources, independent risk analysis, and an approved safety architecture.

Source files: mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py, mfja_3rd_floor_description/src/SmoothJointTrajectoryController.cc, mfja_3rd_floor_description/src/SymmetricGripperController.cc, and mfja_robot_control_config/config/room_315_vla/task_execution_runtime_v4.yaml.

The appendices proceed from shared terminology to exact ROS interfaces, launch arguments, and the package-oriented Repository Source Index.

How to Use This Manual

Use the shortest reading path that matches the current role or maintenance phase. The sequence column is intentional: it moves from context and authority to action and then verification.

Table 4: Recommended reading paths by role and maintenance phase

Role or phase	Start with	Continue in this order	Finish with
New maintainer	Chapters 1–3	Chapters 4–6, then the relevant sub-system in Chapters 7–9	Chapters 10–13
Host setup or re-build	Chapter 4	Chapters 5–6, then the operator pre-flight in Chapter 10	Chapter 11
Simulation operator	Chapter 6	Follow the selected profile in Chapter 10	Chapters 11–12 for evidence or faults
Robot maintainer	Chapter 3	Chapters 5–7 as relevant	Chapters 11–12
Rail maintainer	Chapter 3	Chapters 5–6, then Chapter 8	Chapters 11–12
Visual/planning maintainer	Chapter 3	Chapters 5–6, then Chapter 9	Chapters 10–13
Release or hand-over	Chapter 11	Chapter 12 and the exact interface, launch, and source appendices	Chapter 13

Contents

Document Information	i
Executive Summary	iii
I System Orientation and Architecture	1
1 Project Overview and Boundaries	2
1.1 Chapter Source Map	2
1.2 System Purpose and Capabilities	3
1.3 Supported and Optional Surfaces	3
1.4 Boundaries and Non-goals	3
1.5 Source-of-Truth Order	4
1.6 First Twenty Minutes for a New Maintainer	5
2 Repository and Package Ownership	6
2.1 Chapter Source Map	6
2.2 Repository Boundary and Directory Classification	6
2.3 Package Responsibility and Dependency Matrix	7
2.4 Package: mfja_rail_interfaces	8
2.5 Package: mfja_3rd_floor_description	8
2.6 Package: mfja_robot_control_config	9
2.7 Package: mfja_3rd_floor_bringup	10
2.8 Dependency Rules and Known Exception	11
3 System Architecture, Interfaces, and Data Flow	12
3.1 High-Level Runtime Architecture	12
3.2 Authority Matrix	13
3.3 Gazebo-ROS Boundary	14
3.4 Canonical Rail API	15
3.5 Switch Visual Synchronization	17
3.6 Optional VLA and Task Topics	17
3.7 Adding or Changing an Interface	18
II Environment and Runtime Foundation	19
4 Installation and Build	20
4.1 Chapter Source Map	20
4.2 Supported Environment	20
4.3 Dependency Layers	20
4.4 Fresh Workspace Setup	21
4.5 Build and Verification	23
4.6 Optional and External Environment Boundaries	24
5 Configuration and Deployment Inputs	25
5.1 Configuration Ownership Map	25
5.2 Active, Offline, and Frozen Families	27

5.3	Precedence Rules	27
5.4	Robot Registry Contract	28
5.5	Rail Network and Device Contract	28
5.6	Visual and Task Parameter Precedence	29
5.7	Host-Specific Artifact Paths	29
5.8	Install Coverage and Portability Gaps	30
5.9	Runtime and Tooling Dependencies Beyond Package Manifests	30
5.10	Configuration Review Checklist	31
6	Launch and Runtime Composition	32
6.1	Chapter Source Map	32
6.2	Stable Entry Points	32
6.3	Launch Profile Defaults	33
6.4	Base Gazebo Composition	33
6.5	Runtime Process Inventory	34
6.6	Floor Launch Sequence	35
6.7	Singleton Locks and Namespace Isolation	36
6.8	Temporary and Persistent Runtime Files	36
6.9	Shutdown Semantics	37
III	Subsystem Design and Closed-Loop Execution	38
7	Simulation and Robot Control	39
7.1	Chapter Source Map	39
7.2	Asset and Model Resolution	39
7.3	Robot Instances and Runtime Boundary	40
7.4	Industrial Arm Control	41
7.5	Gripper Configuration and Control	42
7.6	TIAGo Frame Isolation	42
7.7	Robot Model Limitations	43
8	Room 315 Rail Transport	44
8.1	Chapter Source Map	44
8.2	System Model and Authorities	44
8.3	Static Rail Definition	45
8.4	Devices and Occupancy	47
8.5	Shuttle Runtime	47
8.6	Simulation-only Markers and Payloads	49
9	Visual State, Planning, and Closed-Loop Execution	50
9.1	Layering and Safety Intent	50
9.2	V4 Visual-State Runtime	51
9.3	Visual Observation Contract	52
9.4	English TaskGoal Understanding	52
9.5	PDDL Authorities and Translation	53
9.6	Closed-Loop Task Gateway	53
9.7	Primitive Supervisor	54
9.8	Dataset Lifecycle and Leakage Control	54
9.9	Promotion and Authorization	56
9.10	Changing the Learned/Planning Stack	57

IV	Operations and Project Lifecycle	58
10	Gazebo Operator Runbook	59
10.1	Scope and Safety Preconditions	59
10.2	Runbook Source Map	59
10.3	Terminal Setup and Preflight	61
10.4	Base Headless Smoke	61
10.5	Supported Simulation Profiles	61
10.6	Robot Joint and Gripper Operation	62
10.7	Direct Rail Operation	63
10.8	Camera Bridge and Deterministic Supervisor	65
10.9	V4 Visual Runtime	65
10.10	Planning and Closed-Loop Task Execution	67
10.11	Controlled Shutdown	68
11	Verification Strategy and Evidence	69
11.1	What Each Test Layer Proves	69
11.2	Change-to-Test Matrix	70
11.3	Dependency and Static Checks	70
11.4	Focused Source Tests	71
11.5	Registered Package Tests	71
11.6	Runtime Smoke Protocol	71
11.7	Current Test Inventory and Registration Gaps	72
11.8	Evidence Recorded for This Manual	72
11.9	CI Gap and Recommended Baseline	73
11.10	Test Result Record	73
12	Maintenance, Troubleshooting, and Extension	75
12.1	Standard Change Cycle	75
12.2	Change Impact by Package	75
12.3	Extension Recipes	76
12.4	Troubleshooting by Symptom	78
12.5	Build, Temporary, and User Data	81
12.6	Documentation Maintenance	82
12.7	Release and Rollback	82
13	Safety Boundary, Risk Register, and Handover	83
13.1	Authoritative Source Registry	83
13.2	Non-Negotiable Deployment Boundary	84
13.3	Known-Gap Register	84
13.4	High-Risk Maintenance Hot Spots	85
13.5	Third-Party and Redistribution Boundary	86
13.6	What Must Accompany a Handover	86
13.7	Release/Handover Acceptance Checklist	87
13.8	Recommended First Maintenance Milestone	87
13.9	Handover Completion Record	87
V	Reference Appendices	89
A	Glossary and Naming Conventions	90
A.1	Source Domains	90
A.2	Case, Side, and Identity Conventions	92

B	ROS Messages and Services Reference	93
B.1	Interface Ownership and Source Paths	93
B.2	Shared Named State	93
B.3	Shuttle Command and State	94
B.4	Switch and Stopper Arrays	95
B.5	Sensor Feedback	95
B.6	Visual Shuttle State	95
B.7	Visual Observation	96
B.8	Add Shuttle Service	96
B.9	Compatibility Rules	97
C	Launch Argument Reference	98
C.1	Launch Surfaces and Source Paths	98
C.2	Stable Floor Wrappers (49 Arguments)	98
C.3	Stable Isolated-Robot Wrapper (9 Arguments)	100
C.4	VLA Supervisor/Recorder Launch (20 Arguments)	101
C.5	V4 Visual Runtime Launch (11 Arguments)	101
C.6	Task Execution Launch (6 Arguments)	102
C.7	Observation-Only Runtime Acceptance (11 Arguments)	103
C.8	Lower-Level-Only Arguments	104
C.9	Composition Consistency Rules	104
D	Repository Source Index	105
D.1	Repository overview	105
D.2	Python executables and support modules	105
D.3	Internal Python dependency hot spots	113
D.4	Additional package source modules	115
D.5	Launch files	115
D.6	Configuration and data contracts	116
D.7	Gazebo models	118
D.8	Automated tests	120
D.9	Non-package automation and frozen code	124

List of Tables

1	Document control and maintenance rules	i
2	Manual revision history	ii
3	Executive package and source-path map	iii
4	Recommended reading paths by role and maintenance phase	iv
1.1	Authoritative sources for the project overview and boundaries	2
1.2	Supported runtime surfaces and prerequisites beyond a clean clone	3
1.3	Maintained project boundaries and explicitly excluded assurances	4
2.1	Authoritative sources for repository and package ownership	6
2.2	Repository areas and their maintenance classifications	7
2.3	Repository package dependencies, responsibilities, and build sources	7
2.4	Installed description-package directories and maintained contents	8
2.5	Gazebo system-plugin responsibilities and implementation sources	9
2.6	Control-package directories and their maintenance boundaries	10
3.1	End-to-end runtime flows and their implementation sources	12
3.2	Runtime authority and explicitly non-authoritative representations	13
3.3	Namespaced robot interfaces crossing the Gazebo–ROS boundary	15
3.4	Canonical public Room 315 rail endpoints	15
3.5	Optional VLA, visual-state, diagnostics, and task endpoints	17
4.1	Authoritative sources for installation and build procedures	20
4.2	Dependency layers and their supported resolution mechanisms	21
4.3	Minimum rebuild and installed-surface check by source-change category	23
5.1	Configuration files, primary consumers, and owned contracts	25
5.2	Operational classification of active, offline, frozen, and historical files	27
5.3	Full robot-registry instances and room-only enablement	28
5.4	Rail configuration change types and mandatory validation	29
6.1	Authoritative sources for launch and runtime composition	32
6.2	Stable operator entry points and their default simulation profiles	32
6.3	Default values selected by the two stable floor profiles	33
6.4	Processes participating in base and optional runtime compositions	34
6.5	Ordered floor-startup stages and their implementation sources	35
6.6	Generated runtime files, user data, and their maintenance rules	36
7.1	Authoritative sources for simulation and robot control	39
7.2	Per-instance robot command and feedback boundaries	40
7.3	Canonical namespaced ROS topics exposed for robot instances	41
8.1	Authoritative sources for the Room 315 rail transport system	44
8.2	Right-rail directed routing stages and matching convergence guards	46
8.3	Mapping between raw/internal and public left-rail names	46
8.4	Configured slot and identity-bearing DZI sensor locations	47
8.5	Command-driven shuttle transitions and required verification	48
8.6	Autonomous shuttle transitions caused by guards and targets	48
9.1	Closed-loop runtime stages and authority boundaries	50

9.2	Dataset-to-candidate lifecycle and owning tools	55
9.3	Frozen V4 dataset roles, counts, and permitted use	55
10.1	Operator-runbook groups and their authoritative source files	59
11.1	Testing layers and their assurance boundaries	69
11.2	Minimum validation by changed source area	70
12.1	Package-level change impact and compatibility risk	75
12.2	Extension recipe source-path registry	76
12.3	Troubleshooting source-group registry	79
12.4	Troubleshooting symptoms, checks, actions, and source groups	79
12.5	Data classes, locations, and maintenance policy	81
13.1	Authoritative source registry for safety boundaries and known gaps	83
13.2	Prioritized known-gap register and current containment	85
13.3	High-risk maintenance hot spots	85
13.4	Handover completion record	88
A.1	Glossary source domains and owning files	90
A.2	Project glossary and source domains	91
B.1	ROS interface ownership and implementation paths	93
B.2	Shuttle command field contract	94
C.1	Launch surfaces and owning source files	98
C.2	Stable floor core and base arguments	99
C.3	Stable floor rail and shuttle arguments	99
C.4	Stable floor optional VLA arguments	100
C.5	Stable isolated-robot arguments	100
C.6	VLA supervisor and recorder arguments	101
C.7	V4 visual runtime arguments	102
C.8	Task-execution arguments	102
C.9	Observation-only runtime acceptance arguments	103
C.10	Lower-level-only launch arguments	104
D.1	Repository top-level areas and maintenance classification	105
D.2	Python modules: Robot and demonstration control	106
D.3	Python modules: Rail runtime and fleet foundations	106
D.4	Python modules: Visual datasets, models, training, and evaluation	107
D.5	Python modules: Language and TaskGoal understanding	110
D.6	Python modules: Contracts, planning, and execution	111
D.7	Python modules: VLA supervision, recording, and operations	112
D.8	Python modules: Qualification and immutable runtime packaging	113
D.9	Internal Python dependency hot spots	113
D.10	Compiled plugins and isolated Experiment-A source modules	115
D.11	Launch files and responsibilities	115
D.12	Configuration files and data contracts	116
D.13	Gazebo model directories	118
D.14	Test modules: mfja_3rd_floor_description	120
D.15	Test modules: mfja_robot_control_config	121
D.16	Source-like automation outside the ROS package runtime	124

Part I

System Orientation and Architecture

Chapter 1

Project Overview and Boundaries

Chapter guide

Purpose. Establish the project's purpose, supported surfaces, explicit boundaries, and evidence hierarchy before any implementation detail is introduced.

Recommended use. Onboarding to the repository, deciding whether a requested use is supported, or locating the authoritative source for a high-level claim.

1.1 Chapter Source Map

The table maps each topic in this chapter to its primary repository authority. Detailed inventories remain in the subsystem chapters and appendices.

Table 1.1: Authoritative sources for the project overview and boundaries

Topic	Authoritative repository path(s)
Simulation profiles and composition	mfja_3rd_floor_bringup/launch/room_315_only.launch.py; mfja_3rd_floor_bringup/launch/full_floor.launch.py; mfja_3rd_floor_bringup/launch/single_industrial_robot.launch.py; mfja_3rd_floor_bringup/launch/room_315_floor_common.py; mfja_robot_control_config/launch/multi_robot_sim.launch.py
Rail and robot simulation boundaries	mfja_robot_control_config/scripts/room_315_kinematic_shuttle.py; mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py; mfja_3rd_floor_description/src/SmoothJointTrajectoryController.cc; mfja_3rd_floor_description/src/SymmetricGripperController.cc
Visual, language, and execution boundaries	mfja_robot_control_config/scripts/room_315_visual_state_inference_node.py; mfja_robot_control_config/scripts/room_315_visual_runtime_v4.py; mfja_robot_control_config/scripts/room_315_task_goal_builder.py; mfja_robot_control_config/scripts/room_315_task_execution.py; mfja_robot_control_config/scripts/room_315_vla_supervisor.py
Supported and optional dependency surfaces	mfja_rail_interfaces/package.xml; mfja_3rd_floor_description/package.xml; mfja_robot_control_config/package.xml; mfja_3rd_floor_bringup/package.xml; mfja_robot_control_config/config/room_315_vla/visual_state_runtime.yaml; mfja_robot_control_config/config/room_315_vla/task_execution_runtime.yaml; mfja_robot_control_config/config/room_315_vla/task_goal_understanding.yaml; mfja_robot_control_config/scripts/room_315_vla_to_lerobot.py
Onboarding procedure	docs/INSTALLATION.md; docs/QUICK_START_AND_FEATURE_GUIDE.md; docs/TROUBLESHOOTING.md; mfja_3rd_floor_bringup/launch/room_315_only.launch.py

Topic	Authoritative repository path(s)
Repository structure baseline	mfja_rail_interfaces/CMakeLists.txt;mfja_3rd_floor_description/CMakeLists.txt;mfja_robot_control_config/CMakeLists.txt;mfja_3rd_floor_bringup/CMakeLists.txt

1.2 System Purpose and Capabilities

The maintained project provides a focused Room 315 simulation, the complete MFJA third-floor simulation, and an isolated industrial-robot simulation for fast arm and gripper work. Chapter 6 identifies the stable launch entry point and defaults for each profile.

Within Room 315, the software can spawn configured robots, command robot joints and mobile bases through namespaced bridges, move multiple shuttles on calibrated directed rail geometry, command switches and stoppers, publish binary occupancy sensors, attach simulation-only payload visuals, and expose the maintained control surface through typed ROS 2 interfaces.

An optional vision-language-action (VLA) research stack adds paired-camera visual-state estimation, constrained English task understanding, PDDL/PlanSys2 planning, one-step closed-loop execution, deterministic supervision, effect verification, dataset generation, model training, artifact promotion, and fault/acceptance campaigns. These are separate capabilities: enabling the VLA camera/supervisor branch does not implicitly start learned inference or task execution.

1.3 Supported and Optional Surfaces

Table 1.2: Supported runtime surfaces and prerequisites beyond a clean clone

Surface	Clean clone/base installation	Extra prerequisite
Worlds and robots	Available	Working OpenGL display only for GUI
Kinematic rails and typed devices	Available	None beyond ROS/Gazebo dependencies
Camera and primitive supervisor topics	Available, disabled by default	Enable the VLA launch branch
Dataset generation tools	Code/config available	Disk space; some workflows need long Gazebo runs
Learned visual inference	Code available, model absent	Isolated Torch environment and complete promoted V4 bundle
English semantic model	Deterministic parser remains available	llama-cpp-python and separately downloaded GGUF checkpoint
PlanSys2 execution	Code/config available, actuation disabled	PlanSys2 packages plus exact V4 promotion and task authorization artifacts
LeRobot export	Export adapter available	Separately installed LeRobot stack

1.4 Boundaries and Non-goals

Table 1.3: Maintained project boundaries and explicitly excluded assurances

Boundary	Inside this project	Explicitly outside
Simulation	Gazebo worlds, models, kinematic joints, pose services, cameras, and ROS bridges	Validated equivalence to every physical sensor, drive, PLC, and collision envelope
Rail propulsion	Arc-length motion over calibrated directed CSV geometry	Wheel/rail contact, motor dynamics, traction, and certified block control
Robot motion	Deterministic visual joint trajectories and symmetric gripper articulation	Force-controlled grasping, payload attachment, and validated collision avoidance
Perception	Visual shuttle segment/position, bounding box, and loaded-state facts after strict acceptance	Direct learned planning, direct learned actuation, or controller pose used to improve visual localization
Language	Constrained <code>TaskGoal</code> drafts, deterministic validation, clarification, and confirmation	Arbitrary English converted directly into robot programs
Execution	Gazebo-only, checksum-authorized, one-step planning through a deterministic supervisor	Physical-machine authorization or bypass of supervisor/effect checks

The complete deployment boundary and known-gap register are maintained in Chapter 13. Subsystem chapters state only the additional local limitations needed to interpret their behavior safely.

1.5 Source-of-Truth Order

When two statements disagree, use this order:

1. `mfja_rail_interfaces/msg/*.msg` and `mfja_rail_interfaces/srv/*.srv` for wire fields;
2. current launch code and `ros2launch...--show-args` for launch inputs and defaults;
3. versioned YAML, JSON, PDDL, CSV, SDF, and URDF for declarative contracts;
4. node or plugin implementation for runtime behavior;
5. maintained operator and architecture documentation for supported procedures; and
6. historical reports, audits, presentations, and evidence for provenance only.

Never copy a value from a generated `install/` tree back into source. A symlink install can make source and install appear identical, but interfaces, CMake metadata, compiled plugins, and many installed files still require a rebuild.

1.6 First Twenty Minutes for a New Maintainer

Onboarding checklist

1. Confirm the checked-out branch and commit with `gitstatus` and `gitlog-1--online`.
2. Read the non-negotiable deployment boundary in Chapter 13 and the package ownership map in Chapter 2.
3. Create or identify the colcon workspace; keep this repository as one directory under `src/`.
4. Follow the dependency and build procedure in Chapter 4.
5. Launch the headless Room 315 smoke profile from Chapter 10.
6. Verify `/clock`, the three world services, and one rail state and sensor topic.
7. Run the smallest test that covers the intended first change.
8. Before editing, identify the source owner using Chapter 5 and the Repository Source Index in Appendix D.

Chapter 2

Repository and Package Ownership

Chapter guide

Purpose. Explain the repository boundary, package ownership, installed surfaces, and permitted dependency direction.

Recommended use. Locating the owner of a file or behavior, assessing package-level change impact, or adding a new repository component without introducing a dependency cycle.

2.1 Chapter Source Map

Table 2.1: Authoritative sources for repository and package ownership

Topic	Authoritative repository path(s)
Repository boundary and build discovery	.gitignore; README.md; mfja_rail_interfaces/package.xml; mfja_3rd_floor_description/package.xml; mfja_robot_control_config/package.xml; mfja_3rd_floor_bringup/package.xml
Rail-interface package	mfja_rail_interfaces/package.xml; mfja_rail_interfaces/CMakeLists.txt; mfja_rail_interfaces/msg/*.msg; mfja_rail_interfaces/srv/AddShuttle.srv
Description package	mfja_3rd_floor_description/package.xml; mfja_3rd_floor_description/CMakeLists.txt; mfja_3rd_floor_description/worlds/room_315_only.world; mfja_3rd_floor_description/worlds/mfja_3rd_floor.world; mfja_3rd_floor_description/worlds/isolated_industrial_robot.world; mfja_3rd_floor_description/THIRD_PARTY.md
Control package	mfja_robot_control_config/package.xml; mfja_robot_control_config/CMakeLists.txt; mfja_robot_control_config/launch/; mfja_robot_control_config/config/; mfja_robot_control_config/scripts/; mfja_robot_control_config/test/; mfja_robot_control_config/experiment_a_v3r1/
Bringup package	mfja_3rd_floor_bringup/package.xml; mfja_3rd_floor_bringup/CMakeLists.txt; mfja_3rd_floor_bringup/launch/room_315_only.launch.py; mfja_3rd_floor_bringup/launch/full_floor.launch.py; mfja_3rd_floor_bringup/launch/single_industrial_robot.launch.py; mfja_3rd_floor_bringup/launch/room_315_floor_common.py
Known reverse dependency	mfja_robot_control_config/launch/room_315_runtime_acceptance.launch.py; mfja_robot_control_config/package.xml; mfja_3rd_floor_bringup/package.xml

2.2 Repository Boundary and Directory Classification

The repository root is a meta-repository, not a ROS package: there is no root package.xml. Colcon should discover the four explicit package roots, not reports, extracted datasets, presentation material, or generated output.

Table 2.2: Repository areas and their maintenance classifications

Path	Class	Maintenance rule
Four <code>mfja_*</code> package directories	Product source	Edit, review, build, and test
<code>docs/</code> and root <code>README.md</code>	Maintained operational documentation	Update with public behavior; English is authoritative
<code>docs/project_documentation/</code>	Handover source and PDF	Refresh the Repository Source Index and compile after public or structural changes
<code>report/</code> , <code>report_ar/</code>	Academic report sources/output and evidence	Treat checksum-bound evidence as immutable; not runtime source
<code>defense_presentation*/</code>	Presentation source and large derived media	Supplemental only; do not let it enter colcon discovery
<code>tutorial_videos/</code> and standalone HTML files	Operator training material	Keep indexed; not executable authority
<code>build/</code> , <code>install/</code> , <code>log/</code> , <code>caches</code>	Generated local output	Regenerate; never hand-edit or use as the only copy
External datasets/checkpoints	User/research artifacts outside Git	Preserve provenance and hashes; never place frozen duplicate ROS packages under workspace <code>src/</code>

2.3 Package Responsibility and Dependency Matrix

The safe dependency and build order is `mfja_rail_interfaces` and `mfja_3rd_floor_description` first, then `mfja_robot_control_config`, and finally `mfja_3rd_floor_bringup`. Colcon resolves this order automatically when all four paths are provided.

Table 2.3: Repository package dependencies, responsibilities, and build sources

Package	Direct repository dependencies	Responsibility and installed surface	Package authority
<code>mfja_rail_interfaces</code>	None	Stable typed contracts: eleven messages, <code>AddShuttle</code> , and generated bindings	<code>mfja_rail_interfaces/package.xml</code> ; <code>mfja_rail_interfaces/CMakeLists.txt</code>
<code>mfja_3rd_floor_description</code>	None	Three worlds, 47 model directories, six URDFs, two Gazebo libraries, and attribution	<code>mfja_3rd_floor_description/package.xml</code> ; <code>mfja_3rd_floor_description/CMakeLists.txt</code>

Package	Direct repository dependencies	Responsibility and installed surface	Package authority
mfja_robot_control_config	Interfaces; description	Runtime integration plus launch, selected config/PDDL/CSV, and 108 CMake-listed Python files	mfja_robot_control_config/package.xml; mfja_robot_control_config/CMakeLists.txt
mfja_3rd_floor_or_bringup	Description; control	Stable Room-only, full-floor, and isolated-robot launch composition	mfja_3rd_floor_or_bringup/package.xml; mfja_3rd_floor_or_bringup/CMakeLists.txt

2.4 Package: mfja_rail_interfaces

This package deliberately has the smallest dependency surface. Its `mfja_rail_interfaces/CMakeLists.txt` invokes `rosidl_generate_interfaces`; its manifest depends on `builtin_interfaces`, `std_msgs`, the ROSIDL generator, and the ROSIDL runtime. It contains no node or business logic.

- `mfja_rail_interfaces/msg/NamedState.msg` is the name/value unit used by switch and stopper command/state arrays.
- Shuttle command/state and add-service definitions own shuttle lifecycle and exact-target wire semantics.
- Sensor definitions own binary occupancy plus simulator debug metadata.
- Visual definitions own the accepted perception contract; they are not aliases for controller state.

Changing an interface is a compatibility change. Rebuild this package and all consumers; update serializers, dataset schemas, tests, and this manual. Never edit generated Python/C++ bindings in `build/` or `install/`.

2.5 Package: mfja_3rd_floor_description

2.5.1 Installed directories and worlds

Table 2.4: Installed description-package directories and maintained contents

Component	Contents
mfja_3rd_floor_description/worlds/	mfja_3rd_floor_description/worlds/room_315_only.world, mfja_3rd_floor_description/worlds/mfja_3rd_floor.world, and mfja_3rd_floor_description/worlds/isolated_industrial_robot.world
mfja_3rd_floor_description/models/	Reusable rooms, rail cell, furniture, switches, robots, eight identity shuttles, cameras, payloads, and obstacle assets

Component	Contents
mfja_3rd_floor_description/urdf/	KUKA, Stäubli, HC10, HC10DT, TIAGo, and TIAGo-base frame/joint trees for robot_state_publisher
mfja_3rd_floor_description/src/	Two C++17 Gazebo Sim system plugins
mfja_3rd_floor_description/test/	Seven registered static/model/plugin contract tests
mfja_3rd_floor_description/THIRD_PARTY.md	Imported asset provenance and licensing record

The world filename stem and the internal SDF `<worldname="...">` must match. The launch layer derives service names from the internal world name; mismatch causes rail pose/spawn/delete clients to target nonexistent services. The governing files are the three world files named in Table 2.1 and `mfja_robot_control_config/launch/room_315_launch_utils.py`.

2.5.2 Gazebo system plugins

Table 2.5: Gazebo system-plugin responsibilities and implementation sources

Library	Behavior and maintenance boundary	Source file
mfja-symmetric-gripper-controller-system	Subscribes to a model-specific Gazebo Double position command, rejects non-finite input, clamps to configured limits, rate-limits motion in simulation time, and writes equal scalar positions to opposite-axis jaws. It is motion-only and does not grasp physics payloads.	mfja_3rd_floor_description/src/SymmetricGripperController.cc
mfja-smooth-joint-trajectory-controller-system	Consumes Gazebo JointTrajectory; validates joints, positions, timestamps, and limits; uses minimum-jerk interpolation for one-point goals and retimed linear interpolation for multi-point trajectories; publishes progress and writes authoritative positions/velocities after physics.	mfja_3rd_floor_description/src/SmoothJointTrajectoryController.cc

Both libraries require Gazebo vendor packages for common, messages, plugin, simulation, and transport APIs. A plugin change requires a description-package rebuild and model-level tests; a source-only Python test cannot validate the compiled library installation. Chapter 7 owns the detailed runtime semantics.

2.6 Package: mfja_robot_control_config

This is the integration core and the primary maintenance risk because it owns several distinct systems in one ament package.

Table 2.6: Control-package directories and their maintenance boundaries

Directory	Owns	Key warning
<code>mfja_robot_control_config/launch/</code>	Gazebo/robot composition, dual rails, VLA supervisor, visual runtime, task execution, acceptance campaigns	Distinguish stable bringup wrappers from lower-level research launches
<code>mfja_robot_control_config/config/</code>	Robot registries, GUI profiles, rail topology/devices/CSV, visual and VLA contracts, PDDL	Many install rules are explicit; adding a source config does not guarantee installation
<code>mfja_robot_control_config/scripts/</code>	112 command tools, ROS nodes, pure cores, dataset/-model/evaluation workflows	108 are CMake-listed, 49 have a tracked executable bit, and four are source-only
<code>mfja_robot_control_config/test/</code>	98 source test modules	Direct pytest and CMake/ament registration cover different sets
<code>mfja_robot_control_config/experiment_3r1/</code>	Eight isolated experiment helpers	Not installed as ROS executables; invoke only from the documented experiment workflow

The package manifest declares the core ROS/CV/PlanSys2 surface, including Torch/TorchVision. Torch is optional for base simulation even though it is a runtime dependency for advanced tools. Internal Python imports rely on the installed program directory or a source `PYTHONPATH`; shared foundation modules such as `mfja_robot_control_config/scripts/room_315_visual_state_dataset.py`, `mfja_robot_control_config/scripts/room_315_multi_shuttle.py`, `mfja_robot_control_config/scripts/room_315_rail_defaults.py`, and `mfja_robot_control_config/scripts/room_315_contracts.py` have many consumers and require broad tests when changed.

The CMake `PROGRAMS` list is broader than the intended public CLI because it also installs importable support modules. Verify the public surface with `ros2pkgexecutables`; do not infer it from the CMake list alone. Four tracked scripts are intentionally absent from that list:

- `mfja_robot_control_config/scripts/room_315_experiment_a_smoke_v2_package.py`;
- `mfja_robot_control_config/scripts/room_315_visual_final_test_v4.py`;
- `mfja_robot_control_config/scripts/room_315_visual_v4_final_test_coverage_compat.py`; and
- `mfja_robot_control_config/scripts/room_315_visual_v4_final_test_coverage_extension.py`.

Do not advertise these files through `ros2run` unless an explicit install rule, dependency review, and installed-surface test are added. Chapter 4 explains the resulting install and stale-overlay traps.

2.7 Package: `mfja_3rd_floor_bringup`

The bringup package keeps operator commands stable while lower-level launch implementation evolves:

- `mfja_3rd_floor_bringup/launch/room_315_only.launch.py` selects the focused running profile;
- `mfja_3rd_floor_bringup/launch/full_floor.launch.py` selects the complete paused profile;
- `mfja_3rd_floor_bringup/launch/single_industrial_robot.launch.py` forwards a bounded selector to the isolated lower-level launch; and
- `mfja_3rd_floor_bringup/launch/room_315_floor_common.py` owns shared arguments, the singleton lock, obstacle-cache clearing, and delayed rail/VLA includes.

The package installs its launch directory as a whole. A new launch file is installed after rebuild, but every new argument still needs validation, wrapper forwarding, tests, `--show-args` verification, and documentation.

2.8 Dependency Rules and Known Exception

Lower layers must not import user-facing bringup policy. Interfaces should not depend on control logic; description assets should not depend on planning; and pure data/contract helpers should stay importable without ROS when designed that way. This direction keeps geometry and tests usable offline and prevents a research runtime from becoming mandatory for a base simulation.

One current packaging defect breaks that intended direction at runtime: `mfja_robot_control_config/launch/room_315_runtime_acceptance.launch.py` resolves and includes `mfja_3rd_floor_bringup/launch/room_315_only.launch.py`, while the control manifest does not declare bringup. Adding that dependency directly would create a cycle because bringup already depends on control. The acceptance launch therefore requires all four packages to be installed. Move the top-level acceptance composition into bringup, or extract shared composition into a lower package, rather than formalizing the cycle. Treat this exception as a defect, not a precedent.

Chapter 3

System Architecture, Interfaces, and Data Flow

Chapter guide

Purpose. Defines the runtime boundaries, authoritative representations, and public interfaces that connect configuration, Gazebo, rail control, visual state, and task execution.

Recommended use. Read the architecture and authority tables first, then use the boundary and API sections to trace a value from its producer to the feedback that proves its effect. Exact message fields remain in appendix B.

This chapter describes how information crosses package and process boundaries. Field-by-field message definitions are collected in appendix B; the purpose here is to state ownership, direction, and the checks that make a value safe to consume.

3.1 High-Level Runtime Architecture

Table 3.1: End-to-end runtime flows and their implementation sources

Flow	Runtime contract	Source file(s)
Configuration → launch	YAML/CSV/SDF/URDF and launch arguments select the world, robots, rails, and optional VLA branch.	mfja_3rd_floor_bringup/launch/room_315_floor_common.py;mfja_robot_control_config/launch/multi_robot_sim.launch.py
Launch → Gazebo/robots	Gazebo, world bridges, robot spawners, state publishers, and per-robot bridges are composed before delayed rail startup.	mfja_robot_control_config/launch/multi_robot_sim.launch.py
Rail → Gazebo	Logical rail state is transformed into pose/create/remove service requests; Gazebo is visual/physical presentation, not topology authority.	mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py;mfja_robot_control_config/scripts/room_315_kinematic_shuttle.py
Gazebo → cameras → vision	Paired RGB images are bridged and synchronized before V4 inference; presence heartbeats accompany image input.	mfja_robot_control_config/launch/room_315_vla_supervisor.launch.py;mfja_robot_control_config/launch/room_315_visual_state_runtime.launch.py;mfja_robot_control_config/scripts/room_315_visual_state_inference_node.py

Flow	Runtime contract	Source file(s)
Vision → gate- way	Only accepted typed observations enter task grounding; raw JSON remains diagnostic.	mfja_robot_control_config/scripts/room_315_visual_state_inference_node.py; mfja_robot_control_config/scripts/room_315_task_execution_node.py
Gateway ↔ supervisor	The gateway emits one JSON primitive and consumes structured safety/status feedback before replanning.	mfja_robot_control_config/scripts/room_315_task_execution.py;mfja_robot_control_config/scripts/room_315_vla_supervisor.py
Supervisor ↔ rails	The supervisor publishes typed shuttle/switch/stopper commands and consumes newer typed actual state and sensors.	mfja_robot_control_config/scripts/room_315_vla_supervisor.py;mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py;mfja_rail_interfaces/CMakeLists.txt

3.2 Authority Matrix

Table 3.2: Runtime authority and explicitly non-authoritative representations

Fact/action	Authority	Non-authoritative representation	Source file(s)
Message fields/types	Tracked message/service definitions	Wiki prose, JSON examples, or stale installed bindings	mfja_rail_interfaces/msg/;mfja_rail_interfaces/srv/;mfja_rail_interfaces/CMakeLists.txt
World/entity existence	Gazebo plus create/delete result	Logical registry or successful asynchronous request alone	mfja_robot_control_config/launch/multi_robot_sim.launch.py; mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py
Rail route/position	Rail node, network YAML, CSV geometry	Visual switch model, marker pose, retained speed	mfja_robot_control_config/scripts/room_315_kinematic_shuttle.py;mfja_robot_control_config/config/room_315_kinematics/rail_network_right.yaml

Fact/action	Authority	Non-authoritative representation	Source file(s)
Actual switch/stopper state	Delayed typed state registry	Requested command or animation	mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py
Exact slot occupancy	Fresh identity-bearing DZI state	Learned position or privileged coordinates	mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py;mfja_robot_control_config/scripts/room_315_task_execution.py
Learned position/load facts	Accepted authorized V4 observation	Raw JSON, controller geometry, simulator payload state	mfja_robot_control_config/scripts/room_315_visual_state_inference_node.py;mfja_robot_control_config/scripts/room_315_visual_runtime_validation.py
Primitive safety/recovery	Deterministic supervisor	Learned action vector or planner assumption	mfja_robot_control_config/scripts/room_315_vla_supervisor.py
Symbolic plan	Completely validated PlanSys2 response	One unvalidated PDDL action name	mfja_robot_control_config/scripts/room_315_pddl_plan_translator.py;mfja_robot_control_config/scripts/room_315_task_execution.py
Task completion	Fresh effect plus explicit controller stop	Publication, elapsed time, or planner success alone	mfja_robot_control_config/scripts/room_315_task_execution.py
Deployment identity	Immutable manifest/authorization hashes	Directory name, latest symlink, recollection	mfja_robot_control_config/scripts/room_315_visual_runtime_v4.py;mfja_robot_control_config/scripts/room_315_task_execution_config.py

3.3 Gazebo–ROS Boundary

The base bridge exposes ROS simulation time at `/clock` and three world-scoped services under `/world/<world>/: set_pose, create, and remove`. The rail nodes call these services to move, spawn, and delete visuals. Their logical state is internal Python state; Gazebo does not compute rail topology.

Source files for this boundary are `mfja_robot_control_config/launch/multi_robot_sim.launch.py`, `mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py`, `mfja_3rd_floor_description/src/SmoothJointTrajectoryController.cc`, and `mfja_3rd_floor_description/src/SymmetricGripperController.cc`.

Every robot instance has a namespaced bridge:

Table 3.3: Namespaced robot interfaces crossing the Gazebo–ROS boundary

ROS endpoint	Direction	Gazebo endpoint/meaning
<code>/<robot>/joint_trajectory</code>	ROS → GZ	<code>/model/<robot>/joint_trajectory</code> ; plugin command
<code>/<robot>/joint_trajectory_progress</code>	GZ → ROS	Normalized plugin trajectory progress
<code>/<robot>/joint_states</code>	GZ → ROS	Joint feedback for state publisher/consumers
<code>/<robot>/gripper/position_command</code>	ROS → GZ	Industrial scalar gripper command
<code>/<robot>/cmd_vel</code>	ROS → GZ	TIAGo variants only
<code>/<robot>/odom</code> and <code>/<robot>/tf</code>	GZ → ROS	TIAGo variants only

Each `robot_state_publisher` runs in the robot namespace, remaps `tf/tf_static`, and prefixes frames with `<robot>/`. Do not remove the prefix to make a visualization look shorter; duplicate robot links would then collide in the shared TF graph.

3.4 Canonical Rail API

Replace `<side>` with `right` or `left`. All entries below are public ROS interfaces unless marked simulation-only.

The exact declarations and publishers/subscribers are in `mfja_rail_interfaces/CMakeLists.txt`, the files under `mfja_rail_interfaces/msg/` and `mfja_rail_interfaces/srv/`, and `mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py`.

Table 3.4: Canonical public Room 315 rail endpoints

Endpoint	Type/direction	Purpose
<code>/room_315/rails/<side>/shuttles/command</code>	ShuttleCommand, subscribe	ON, OFF, RESET, or REMOVE a named shuttle
<code>/room_315/rails/<side>/shuttles/state</code>	ShuttleState, publish	One message per managed shuttle plus known-empty heartbeat
<code>/room_315/rails/<side>/shuttles/add</code>	AddShuttle, service	Validate and add/deploy a named shuttle
<code>/room_315/rails/<side>/switches/command</code>	SwitchCommand, subscribe	Request named switch states

Endpoint	Type/direction	Purpose
/room_315/rails/<side>/switches/state	SwitchState, publish	Delayed actual switch state
/room_315/rails/<side>/stoppers/command	StopperCommand, subscribe	Request named stopper states
/room_315/rails/<side>/stoppers/state	StopperState, publish	Delayed actual stopper state
/room_315/rails/<side>/sensors/feedback	SensorFeedback, publish	Binary device occupancy and simulation metadata
/room_315/rails/<side>/shuttles/pose_cmd	PoseStamped, publish	First shuttle's transformed Gazebo pose
/room_315/rails/<side>/shuttles/<safe-entity-name>/pose_cmd	PoseStamped, publish	Per-entity pose for additional/dynamic shuttles; the Gazebo entity name is sanitized for ROS
/room_315/rails/<side>/shuttles/pose_offset_command	String, subscribe	Expert JSON calibration offset; simulation-only
/room_315/rails/<side>/shuttles/payload_command	String, subscribe	Payload visual JSON; simulation/dataset-only
/room_315/rails/<side>/shuttles/payload_state	String, publish	Logical payload visual state JSON

Commands, state, and sensors normally use reliable volatile depth-10 QoS. The visual switch feedback subscription is the notable exception: reliable, transient-local, depth 1, so a late rail node receives the last latched visual summary. Camera images and rail-presence inputs to learned vision use sensor-data QoS. When debugging a missing sample, compare endpoint QoS before changing code.

QoS sources: `mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py` and `mfja_robot_control_config/scripts/room_315_visual_state_inference_node.py`.

3.4.1 Command-to-Effect Rule

The command path is defined by `mfja_rail_interfaces/msg/ShuttleCommand.msg`, `mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py`, and `mfja_robot_control_config/scripts/room_315_vla_supervisor.py`:

1. An operator or supervisor publishes a typed command.
2. The side-specific rail node validates identity, legal state, occupancy, topology, and delayed device rules.
3. Logical motion advances on simulation time and publishes fresh typed state and sensor feedback.
4. The node converts logical rail position to a world pose and calls Gazebo `SetEntityPose`.
5. The initiator verifies newer authoritative feedback; publication alone is never completion.

3.5 Switch Visual Synchronization

This path is implemented by `mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py` and `mfja_robot_control_config/scripts/conveyor_loop_mode_controller.py`.

The rail node schedules a typed actual switch change, publishes it, and mirrors the accepted state as a comma-separated `selector=EXTERIOR/INTERIOR` `std_msgs/msg/String` assignment list on `/mfja/conveyor/switch_cmd`. The visual controller animates/configures the Gazebo switch entity and publishes a latched assignment summary on `/mfja/conveyor/switch_states`. Matching feedback acknowledges an outstanding typed target; conflicting feedback is ignored until that target is acknowledged. When no typed target is outstanding and `sync_from_visual_switch_states=true`, an unmatched summary may schedule a delayed update of the typed actual-state registry. Because routing reads that registry, this synchronization path can affect topology; it is not merely a GUI presentation update.

3.6 Optional VLA and Task Topics

Topic creation and consumption are defined in `mfja_robot_control_config/launch/room_315_vla_supervisor.launch.py`, `mfja_robot_control_config/scripts/room_315_vla_supervisor.py`, `mfja_robot_control_config/scripts/room_315_visual_state_inference_node.py`, and `mfja_robot_control_config/scripts/room_315_task_execution_node.py`.

Table 3.5: Optional VLA, visual-state, diagnostics, and task endpoints

Endpoint	Type	Owner/consumer
<code>/room_315/vla/right_rail_rgb/image</code>	<code>sensor_msgs/Image</code>	Gazebo camera bridge → supervisor/vision/recorder
<code>/room_315/vla/left_rail_rgb/image</code>	<code>sensor_msgs/Image</code>	Gazebo camera bridge → supervisor/vision/recorder
<code>/room_315/vla/command</code>	<code>std_msgs/String</code> JSON	Operator/task gateway → supervisor
<code>/room_315/vla/status</code>	<code>std_msgs/String</code> JSON	Supervisor → gateway/operator/recorder
<code>/room_315/vla/emergency_stop</code>	<code>std_msgs/Bool</code>	Operator/safety source → supervisor
<code>/room_315/visual_state/raw</code>	<code>VisualStateObservation</code>	V4 inference, before acceptance
<code>/room_315/visual_state/raw_model_prediction</code>	<code>String</code> JSON	Diagnostic model output only
<code>/room_315/visual_state/validation</code>	<code>VisualStateObservation</code>	Validator decision/causes
<code>/room_315/visual_state/observed_state</code>	<code>VisualStateObservation</code>	Accepted V4 state → task gateway
<code>/room_315/task_goal</code>	<code>std_msgs/String</code> JSON	Confirmed English CLI/operator → gateway

Endpoint	Type	Owner/consumer
/room_315/task_goal/status	std_msgs/String JSON	Gateway lifecycle/result → operator
/diagnostics	DiagnosticArray	Visual runtime and task gateway health
/planner/get_plan	plansys2_msgs/ GetPlan	Gateway → PlanSys2 planner

The supervisor still subscribes to legacy `/room_315/rails/<side>/sensors/position_feedback`; the rail runtime does not publish that topic. It is compatibility scaffolding, not a missing required connection. Current consumers use `sensors/feedback`.

For the optional visual path, paired RGB images are synchronized and validated before an accepted observation may enter task grounding. For the optional task path, a confirmed `TaskGoal` is grounded, planned, fully translated, and executed one deterministic primitive at a time; every primitive requires newer effect evidence before replanning. The complete gates and their authoritative source paths are organized in chapter 9.

3.7 Adding or Changing an Interface

The interface build/install authority is `mfja_rail_interfaces/CMakeLists.txt`; runtime consumers are centered in `mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py`, `mfja_robot_control_config/scripts/room_315_vla_supervisor.py`, and `mfja_robot_control_config/scripts/room_315_task_execution_node.py`.

Contract-change checklist

1. Decide whether the value is a public typed fact, simulation-only JSON, or internal implementation detail. Prefer a typed interface for stable cross-process contracts.
2. Modify the owning `.msg/.srv` or versioned JSON schema; define units, legal values, empty/stale semantics, and authority.
3. Update every publisher, subscriber, bridge, launch/config default, recorder/serializer, test fixture, and PDDL/visual adapter that consumes it.
4. Rebuild `mfja_rail_interfaces` before dependent packages; source the new overlay in every terminal.
5. Test valid, invalid, empty, stale, duplicate, delayed, and restart cases.
6. Update this chapter, appendix B, and the Repository Source Index.

Part II

Environment and Runtime Foundation

Chapter 4

Installation and Build

Chapter guide

Purpose. Define the supported host, dependency layers, reproducible workspace setup, build lifecycle, and optional environment boundaries.

Recommended use. Setting up a new machine, rebuilding after a change, diagnosing a stale overlay, or preparing external learned-runtime artifacts.

4.1 Chapter Source Map

Table 4.1: Authoritative sources for installation and build procedures

Topic	Authoritative repository path(s)
Supported host and base setup	docs/INSTALLATION.md; README.md; mfja_rail_interfaces/package.xml; mfja_3rd_floor_description/package.xml; mfja_robot_control_config/package.xml; mfja_3rd_floor_bringup/package.xml
Build and installed surface	mfja_rail_interfaces/CMakeLists.txt; mfja_3rd_floor_description/CMakeLists.txt; mfja_robot_control_config/CMakeLists.txt; mfja_3rd_floor_bringup/CMakeLists.txt
Optional language environment	mfja_robot_control_config/scripts/setup_room315_intent_model.py; mfja_robot_control_config/config/room_315_vla/task_goal_understanding.yaml
Hybrid Nix shell	flake.nix; flake.lock
External V4 artifacts	mfja_robot_control_config/config/room_315_vla/visual_state_runtime.yaml; mfja_robot_control_config/config/room_315_vla/task_execution_runtime.yaml; mfja_robot_control_config/scripts/room_315_runtime_contracts.py

4.2 Supported Environment

The maintained combination is Ubuntu 24.04 LTS, ROS 2 Jazzy, Python 3.12, CMake 3.28 or newer, and Gazebo Harmonic (gz-sim 8) through `ros_gz`. Native Ubuntu is the supported end-to-end route. A VM may work when it provides adequate RAM, CPU, and 3D acceleration, but is not a validated target. Ubuntu 22.04, ROS 2 Humble, Gazebo Classic, Windows, macOS, NixOS, and pure-Nix ROS are not current supported configurations.

4.3 Dependency Layers

Table 4.2: Dependency layers and their supported resolution mechanisms

Layer	Required components	Resolution
Host	Compiler toolchain, CMake, Git, OpenGL, Python 3, pip/venv/pytest/YAML, colcon, rosdep	Ubuntu packages
ROS/Gazebo	Jazzy Desktop, Gazebo Harmonic, <code>ros_gz</code> , state publisher	Official ROS/Ubuntu packages
Interface/build	ament CMake, ROSIDL generator/runtime, Gazebo vendor libraries	Package manifests via rosdep
Runtime	rclpy, launch, standard messages, cv_bridge, message_filters, PlanSys2, lifecycle, ROS–Gazebo bridges	Package manifests via rosdep
Base Python	OpenCV, NumPy, Pillow, PyYAML	rosdep/Ubuntu
Optional learned runtime	Torch, TorchVision, CUDA where desired	Dedicated visual virtual environment
Optional language runtime	<code>llama-cpp-python==0.3.16</code> and GGUF	Dedicated intent virtual environment
Optional export	LeRobot	Separate environment; not required for operation

The four package manifests are the source of truth for the dependencies that the ROS packages declare and that `rosdep` can resolve. That authority is not a completeness guarantee: host graphics drivers, documentation tools, optional virtual environments, and external learned artifacts are outside the manifest contract. Whenever code gains an import, launch dependency, or linked library, update the owning manifest and verify it on a clean host; do not make a hand-written installation list the only record of that dependency.

4.4 Fresh Workspace Setup

4.4.1 Host preparation

The following assumes the official ROS 2 Jazzy apt repository is configured on Ubuntu 24.04. If it is not, configure it using the ROS Jazzy Ubuntu package installation procedure before continuing.

```

sudo apt update
sudo apt install -y \
  build-essential cmake git mesa-utils ninja-build pkg-config \
  poppler-utils ripgrep \
  python3-dev python3-colcon-common-extensions python3-pip \
  python3-pytest python3-rosdep python3-venv python3-yaml \
  ros-jazzy-desktop ros-jazzy-robot-state-publisher ros-jazzy-ros-gz

if [ ! -e /etc/ros/rosdep/sources.list.d/20-default.list ]; then
  sudo rosdep init
fi
rosdep update

```

After a kernel or graphics-driver update, reboot deliberately before debugging Gazebo. Then verify the local graphical shell:

```
source /opt/ros/jazzy/setup.bash
```



```
echo "$ROS_DISTRO"
gz sim --versions
cmake --version
python3 --version
glxinfo -B
```

4.4.2 Repository checkout

Create the workspace and reproduce the exact branch and commit documented by this manual release. The explicit destination and pre-existence check prevent Git from cloning over an operator's checkout.

```
export MFJA_WS="$HOME/mfja_ws"
export MFJA_REPO="$MFJA_WS/src/mfja_3rd_floor_gz"
export MFJA_RELEASE_BRANCH='INTERNSHIP-ALI-2026'
export MFJA_RELEASE_COMMIT='aadf967b1188e680256580408336e7703ff1cb5d'

(
  set -euo pipefail
  mkdir -p "$MFJA_WS/src"
  test ! -e "$MFJA_REPO"
  git clone --branch "$MFJA_RELEASE_BRANCH" --single-branch \
    https://github.com/aip-primeca-occitanie/mfja_3rd_floor_gz.git \
    "$MFJA_REPO"
  git -C "$MFJA_REPO" cat-file -e "$MFJA_RELEASE_COMMIT^{commit}"
  git -C "$MFJA_REPO" checkout --detach "$MFJA_RELEASE_COMMIT"
  test "$(git -C "$MFJA_REPO" rev-parse HEAD)" = \
    "$MFJA_RELEASE_COMMIT"
  test -f "$MFJA_REPO/docs/INSTALLATION.md"
  test -f "$MFJA_REPO/mfja_3rd_floor_bringup/launch/room_315_floor_common.py"
  git -C "$MFJA_REPO" status --short --branch
)
```

The detached checkout makes the reproduced snapshot unambiguous. Create a named maintenance branch from it before making source changes, and record the new commit in the next manual release.

4.4.3 Declared ROS dependencies

Install declared dependencies. Torch keys are skipped for the base path because the supported apt/rosdep environment may not provide suitable candidates and base simulation does not need them:

```
(
  set -euo pipefail
  source /opt/ros/jazzy/setup.bash
  rosdep install --from-paths \
    "$MFJA_REPO/mfja_rail_interfaces" \
    "$MFJA_REPO/mfja_3rd_floor_description" \
    "$MFJA_REPO/mfja_robot_control_config" \
    "$MFJA_REPO/mfja_3rd_floor_bringup" \
    --ignore-src --rosdistro jazzy -y \
    --skip-keys "python3-torch python3-torchvision"
)
```

The canonical dependency installation deliberately omits rosdep's `-r` continuation option: an unresolved declared dependency must stop the setup rather than leave a partially prepared host that appears successful.

4.5 Build and Verification

4.5.1 Canonical development build

Build from the workspace root and use explicit package paths. This prevents colcon from discovering a frozen package copy inside an extracted dataset or other evidence directory.

```
(
  set -euo pipefail
  cd "$MFJA_WS"
  source /opt/ros/jazzy/setup.bash
  colcon build --symlink-install --paths \
    "$MFJA_REPO/mfja_rail_interfaces" \
    "$MFJA_REPO/mfja_3rd_floor_description" \
    "$MFJA_REPO/mfja_robot_control_config" \
    "$MFJA_REPO/mfja_3rd_floor_bringup"
  test -f "$MFJA_WS/install/setup.bash"
  source "$MFJA_WS/install/setup.bash"
  ros2 pkg prefix mfja_robot_control_config
)
```

This fail-fast subshell never sources the install overlay after a failed build, which avoids accidentally validating stale output. Its environment is local to the block; source both layers again in every terminal used for operation.

Every new terminal must source both layers, in order:

```
source /opt/ros/jazzy/setup.bash
source "$HOME/mfja_ws/install/setup.bash"
```

Verify that the intended overlay is active:

```
ros2 pkg prefix mfja_robot_control_config
ros2 pkg executables mfja_robot_control_config | sort
ros2 interface show mfja_rail_interfaces/msg/ShuttleCommand
ros2 launch mfja_3rd_floor_bringup room_315_only.launch.py --show-args
```

4.5.2 Build scope by change type

Table 4.3: Minimum rebuild and installed-surface check by source-change category

Change	Minimum build	Installed-surface check
Control Python/config	Control package; bringup too if wrapper contract changes	Installed executable and configuration paths
Launch file	Owning package	Installed launch file and <code>--show-args</code> surface
World/model/URDF/C++ plugin	Description package; consumers when path/library contract changes	Installed assets/libraries and Gazebo loadability
Message/service	Interface package and every consumer (normally <code>--package s-up-to mfja_3rd_floor_bringup</code>)	<code>ros2 interface show</code> and installed bindings

Change	Minimum build	Installed-surface check
CMake/manifest	Affected packages with <code>--cmake-clean-cache</code>	<code>rosdepcheck</code> , package prefix, and install tree
Documentation only	No colcon build	Named PDF, links, language, and diff checks

This table selects only the rebuild and installed-surface scope. Table 11.2 in Chapter 11 is the canonical mapping from changed behavior to focused and broader tests.

Example control-only rebuild:

```
(
  set -euo pipefail
  cd "$MFJA_WS"
  source /opt/ros/jazzy/setup.bash
  colcon build --symlink-install \
    --paths "$MFJA_REPO/mfja_robot_control_config" \
    --packages-select mfja_robot_control_config
  test -f "$MFJA_WS/install/setup.bash"
  source "$MFJA_WS/install/setup.bash"
  ros2 pkg prefix mfja_robot_control_config
)
```

4.5.3 Install semantics and common stale-code traps

- Control-package scripts and many configs are installed through explicit CMake lists. A new file may exist in source yet be absent from `ros2run` or package share.
- Launch, model, world, and URDF directories are installed as directories, excluding Python caches where configured.
- ROS interfaces and C++ plugins always require regeneration/recompilation.
- Most YAML is read only at node startup; restart the affected launch.
- A terminal sourced before a rebuild can still resolve an older overlay.
- CMake decides whether Torch-gated tests are registered at configure time; changing environments can require a clean CMake configure.

4.6 Optional and External Environment Boundaries

4.6.1 Hybrid Nix boundary

`flake.nix` and `flake.lock` provide a hybrid auxiliary shell with pinned Bash, Ninja, Make, and Git. They do not provide ROS 2, Gazebo, CMake, the compiler, Python, colcon, rosdep, or ROS package dependencies. Install the complete Ubuntu ROS/toolchain first; use `nixdevelop` only around that host stack. Do not describe this repository as a pure-Nix ROS distribution.

4.6.2 External V4 and language artifacts

A clean build is not a complete learned runtime. Keep checkpoints, datasets, promotion bundles, task authorizations, GGUF files, and virtual environments outside the repository and the colcon source tree. The checked-in V4 YAML contains qualification-host paths as evidence/templates; create a host-local copy or pass explicit launch overrides. Never replace an approved expected hash with a hash calculated from an untrusted file merely to satisfy a check.

Chapter 5

Configuration and Deployment Inputs

Chapter guide

Purpose. Identifies which tracked file owns each configurable contract, which configuration families are operational, and which override wins at runtime.

Recommended use. Locate the owner and operational class before editing, apply the precedence rules to determine the effective value, then review artifact, installation, and dependency constraints before deployment.

5.1 Configuration Ownership Map

Table 5.1: Configuration files, primary consumers, and owned contracts

Source	Primary consumer	Owns
mfja_robot_control_config/config/robots.yaml	mfja_robot_control_config/launch/multi_robot_sim.launch.py	Full-floor instances, enable flags, model/variant selection, and spawn pose
mfja_robot_control_config/config/robots_room_315_only.yaml	Room-only and isolated launches	Focused-cell robot selection/defaults
mfja_robot_control_config/config/gripper_commands_defaults.yaml	Launch patching and gripper CLI	Per-instance jaw endpoints and default open/close percentages; fixed joint/controller mapping remains in mfja_robot_control_config/launch/gripper_range_config.py
mfja_robot_control_config/config/*.gui.conf	Gazebo GUI	Panel/layout profiles; runtime-safe and light are current profile defaults
mfja_robot_control_config/config/gazebo_parameters.yaml	No current runtime consumer	Installed legacy publish-rate parameter file; current launches do not select it
mfja_robot_control_config/config/room_315_kinematics/rail_network_side.yaml	Rail core/node	Directed nodes/segments, routing, switch guards, CSV references
mfja_robot_control_config/config/room_315_kinematics/raw_segments/*.csv	Rail path loader	Calibrated sample order and geometry

Source	Primary consumer	Owns
mfja_robot_control_config/config/room_315_kinematics/rail_devices_<side>.yaml	Rail device loader	Slots, sensors, stoppers, radii/default states
mfja_robot_control_config/config/room_315_vla/shuttle_identity.yaml	Identity tracking, recording/evaluation, and TaskGoal fusion	Canonical eight identities and associated visuals
mfja_robot_control_config/config/room_315_vla/vla_supervisor.yaml	Supervisor when explicitly selected	Primitive validation and recovery defaults
mfja_robot_control_config/config/room_315_vla/visual_state_runtime.yaml	Visual runtime launch/node	Active V4 manifest/hash, mode, topics, presence and fusion policy
mfja_robot_control_config/config/room_315_vla/visual_observed_state_calibration.yaml	Deterministic provider/smoke/training helpers	Geometry, block/slot mapping, thresholds and consistency policy; not the learned V4 node's runtime YAML
mfja_robot_control_config/config/room_315_vla/task_goal_understanding.yaml	English parser/semantic adapter	Allowed language contract and optional local model settings
mfja_robot_control_config/config/room_315_vla/task_execution_runtime.yaml	Task launch/gateway	Fail-closed authorization, topics, timeouts, limits, allowed V4 identity
mfja_robot_control_config/config/room_315_vla/task_execution_runtime_v4.yaml	Frozen qualification workflow	Versioned V4 runtime snapshot; not automatically selected
mfja_robot_control_config/config/room_315_vla/pddl/domain_room315_runtime.pddl	Task launch/PlanSys2	Live executable planning domain
mfja_robot_control_config/config/room_315_vla/pddl/domain_room315.pddl	Scenario/data tools	Broader expert/data-generation domain

Exact files and their short roles are also listed in the Repository Source Index in appendix D.

5.2 Active, Offline, and Frozen Families

The installed/current boundary is defined by `mfja_robot_control_config/CMakeLists.txt`, the active launch files under `mfja_robot_control_config/launch/`, and the versioned files under `mfja_robot_control_config/config/room_315_vla/`.

The flat control package contains multiple research generations. Classify a file before editing or invoking it:

Table 5.2: Operational classification of active, offline, frozen, and historical files

Class	Examples	Rule
Base active runtime	robot registries, rail network/devices/CSV, GUI profiles, supervisor	Maintained for ordinary simulation; changes need live smoke tests
Active learned runtime	<code>mfja_robot_control_config/config/room_315_vla/visual_state_runtime.yaml</code> , V4 visual contract/node, runtime PDDL, and <code>mfja_robot_control_config/config/room_315_vla/task_execution_runtime.yaml</code>	V4 only; requires exact external promoted/authorized artifacts
Offline current tooling	V4 scenario, dataset, training, evaluation, candidate, acceptance, fault tools	May create new artifacts; never mutates the current promoted bundle
Frozen evidence/config	V4 training/final-test JSON, versioned task YAML, sealed manifests/results	Immutable after decision; supersede with a new version
Historical research	V3, V3R1, Experiment-A, old final-test/compatibility adapters	Preserve for provenance; not the supported active runtime

File names alone are insufficient proof that an artifact is current. Follow the manifest chain and exact hashes. In particular, `final`, `v4`, or `runtime` in a filename does not replace promotion and authorization. Local build, runtime, and user-output classes are defined separately in chapter 12; they never become configuration authority.

5.3 Precedence Rules

Precedence and forwarding are implemented in `mfja_3rd_floor_bringup/launch/room_315_floor_common.py`, `mfja_robot_control_config/launch/room_315_visual_state_runtime.launch.py`, `mfja_robot_control_config/launch/room_315_task_execution.launch.py`, and the parameter declarations in their launched nodes.

Runtime values can come from several layers. For a given node, use this precedence from strongest to weakest:

1. an explicit launch argument that is forwarded as a ROS parameter;
2. a parameter override in the including launch;
3. the selected YAML parameter file;
4. the node's declared code default; and

5. a historical document/example, which has no runtime authority.

Not every launch argument becomes a node parameter. Some control launch conditions, select files, alter the environment, or cause a local cache action. Use `ros2launch...--show-args` for the public surface, then inspect the forwarding code and `ros2paramdump` for an effective running node.

5.4 Robot Registry Contract

Source files: `mfja_robot_control_config/config/robots.yaml`, `mfja_robot_control_config/config/robots_room_315_only.yaml`, and `mfja_robot_control_config/launch/multi_robot_sim.launch.py`.

The full registry defines six instances:

Table 5.3: Full robot-registry instances and room-only enablement

Name	Model	Spawn (x, y, z, yaw)	Room-only
kuka1	KUKA KR6	(−10.6366, −6.0, 1.0, 3.14)	enabled
staubli1	Stäubli TX2	(−15.1622, −6.0, 1.0, 1.57)	enabled
yaskawa_hc10_1	HC10	(−15.1622, −3.0, 0.62, 1.57)	enabled
yaskawa_hc10dt_1	HC10DT	(−10.4366, −3.0, 0.62, 1.57)	enabled
tiago1	TIAGo	(−3.0, −3.0, 0.0, 1.57)	disabled
tiago_base1	TIAGo base	(−6.2, −3.0, 0.0, 1.57)	absent (commented example)

When adding a robot, preserve uniqueness of instance name, Gazebo entity, namespace, and TF prefix. Confirm model and URDF joint names align; add bridge rules for only the capabilities actually implemented; and validate initial joints against plugin/model limits. A registry enable flag is used only when the launch selector is empty; `robots:=all` intentionally includes disabled entries.

5.5 Rail Network and Device Contract

Source files: `mfja_robot_control_config/config/room_315_kinematics/rail_network_right.yaml`, `mfja_robot_control_config/config/room_315_kinematics/rail_network_left.yaml`, `mfja_robot_control_config/config/room_315_kinematics/rail_devices_right.yaml`, `mfja_robot_control_config/config/room_315_kinematics/rail_devices_left.yaml`, and `mfja_robot_control_config/scripts/room_315_kinematic_shuttle.py`.

Network YAML schema 2 defines 12 nodes, 14 directed segments, four public switches, and explicit routing per side. A segment's `csv` path, direction, start/end node, and guarded successor are one atomic contract. Device YAML schema 1 defines four slots, 20 configured position sensors, and four stoppers per side; derived stopper sensors are created at runtime. The loader follows each explicit segment `csv` value; the current top-level `source_csv_dir` metadata is not used to resolve files.

Use this rule for changes:

Table 5.4: Rail configuration change types and mandatory validation

Intended change	Edit	Mandatory checks
Move rail visually as a whole	Calibration defaults/launch offsets	All device world poses, world clearance, both sides
Correct sampled path	Referenced CSV	Monotonic index, endpoints/tangents, length drift, topology and devices
Change connectivity	Network YAML routes/guards	All switch combinations, no-successor FALLING behavior, PDDL topology
Move/add sensor or slot	Device YAML ratios/points	Device loader, marker, occupancy crossing, identity DZI semantics
Rename public segment/switch	Network, defaults mapping, PDDL, configs, tests, datasets	Wire compatibility and historical adapter decision
Change speed/headway/delay	Launch/config parameters	Multi-shuttle, stopper/switch timing, task timeouts, acceptance

Never tune network connectivity to repair a visual coordinate offset. Use the device-position and calibration tools, then update the explicit numeric authority.

5.6 Visual and Task Parameter Precedence

Source files: `mfja_robot_control_config/launch/room_315_visual_state_runtime.launch.py`, `mfja_robot_control_config/launch/room_315_task_execution.launch.py`, `mfja_robot_control_config/config/room_315_vla/visual_state_runtime.yaml`, and `mfja_robot_control_config/config/room_315_vla/task_execution_runtime.yaml`.

For visual runtime, the selected YAML loads first. Launch always overrides `use_sim_time`, `device`, `runtime_generation`, `dry_run_state_fusion`, and `plansys2_update_enabled`. Nonempty `runtime_mode`, `promotion-manifest` path, and `manifest-hash` arguments override their YAML keys; empty values preserve YAML.

For task execution, YAML loads first. Launch always overrides `simulation_time`, `execution_enabled`, the installed runtime PDDL path, planner service, and external-obstacle policy. `enable_plansys2` controls whether the planner process is launched; it is not a gateway parameter. The planner is configured after 1 s and automatically activated from the inactive lifecycle state.

Two independent enable conditions

`execution_enabled:=true` is necessary but never sufficient. The gateway also requires exact immutable authorization/promotion files and matching schema/checkpoint hashes. Likewise, enabling the supervisor branch does not enable the task gateway. Preserve both separations.

5.7 Host-Specific Artifact Paths

The active tracked examples are `mfja_robot_control_config/config/room_315_vla/visual_state_runtime.yaml`, `mfja_robot_control_config/config/room_315_vl`

a/task_execution_runtime.yaml, and mfja_robot_control_config/config/room_315_vla/task_execution_runtime_v4.yaml; frozen evidence paths are recorded for provenance rather than rewritten.

A Git-tracked search of mfja_robot_control_config returns 301 lines that contain /home/tiago. They are concentrated in frozen experiment/dataset evidence and active V4 qualification configs. Do not blindly replace them: hashes and frozen evidence would change, and many paths intentionally record provenance. Other repository areas, especially report evidence, may preserve additional host paths for their own reproducibility records.

For a new machine:

1. inventory which referenced bundles exist and verify every recorded hash;
2. restore immutable bundles outside workspace src/, or create and qualify a new candidate;
3. make a host-local runtime YAML or pass supported launch overrides;
4. keep frozen evidence unchanged; and
5. run copied-install, shadow, acceptance, fault, and authorization checks before enabling execution.

5.8 Install Coverage and Portability Gaps

Source files: mfja_robot_control_config/CMakeLists.txt, mfja_robot_control_config/scripts/room_315_task_goal_semantic.py, mfja_robot_control_config/scripts/room_315_task_goal_benchmark.py, mfja_robot_control_config/scripts/room_315_task_execution_config.py, and mfja_robot_control_config/scripts/room_315_visual_observed_state_provider.py.

The CMake install surface is explicit and is not identical to Git:

- four tracked scripts are not listed for installation;
- 49 of the 112 tracked source scripts carry an executable source bit, while CMake lists 108 files; both dimensions matter in a symlink install;
- mfja_robot_control_config/config/room_315_kinematics/SEGMENT_CSV_MIGRATION.md and two V4 coverage JSON files are tracked but omitted from config installation;
- CMake installs the complete PDDL directory, so an ignored local mfja_robot_control_config/config/room_315_vla/pddl/domain_room315_guide_ar.html can leak into a local install even though a clean clone cannot reproduce it; and
- multiple Python utilities derive config paths using Path(__file__).resolve().parents[1]/config. This succeeds in the current symlink install but can fail in a normal copied install, where config belongs under package share. Use ament_index_python.

The affected copied-install pattern appears in task-goal semantic/benchmark, task-execution config, visual observed-state provider, V4 training, and related utilities. Treat copied-install validation as a release gate until all paths are centralized.

5.9 Runtime and Tooling Dependencies Beyond Package Manifests

Source files: mfja_robot_control_config/package.xml, mfja_robot_control_config/CMakeLists.txt, mfja_robot_control_config/scripts/setup_room315_intent_model.py, mfja_robot_control_config/scripts/room_315_vla_to_lerobot.py, and mfja_robot_control_config/launch/room_315_runtime_acceptance.launch.py.

The manifests cover ROS, Gazebo, OpenCV, NumPy, Pillow, YAML, Torch, and TorchVision. Selected optional workflows additionally require undeclared `llama-cpp-python==0.3.16`, an alternative `transformers` backend, `lerobot`, `rosbag2_py`, `rosidl_runtime_py`, and Python/pip for the intent setup tool. External program assumptions include `ros2bag` and a storage plugin, `gz`, `git`, `nvidia-smi`, `pskill`, and the ROS daemon. These are environment-specific, not base simulation requirements, but each installed workflow must state and validate them.

The runtime-acceptance launch also has the undeclared reverse dependency on `mfja_3rd_floor_bringup` described in chapter 2. Do not add that dependency blindly: it would create a control-bringup package cycle. Refactor the launch ownership or shared composition instead.

Conversely, Torch and TorchVision are unconditional manifest runtime dependencies even though base simulation does not import them and the base `rosdep` procedure skips their keys. This formal/-operational mismatch should be resolved by packaging the advanced learned stack separately or defining a supported optional-dependency policy.

5.10 Configuration Review Checklist

The executable `install/test` surface used by this checklist is declared in `mfja_robot_control_config/CMakeLists.txt`, `mfja_robot_control_config/package.xml`, and the registered tests under `mfja_robot_control_config/test/`.

Before merging a configuration change

1. Identify the owner and whether the file is active, offline, historical, frozen, or local output.
2. Validate syntax and schema with the owning loader/test, not only a YAML or JSON parser.
3. Search all consumers by filename, parameter key, topic, and semantic value; check left/right mapping separately.
4. Confirm launch precedence and installed-file coverage in both symlink and copied installs where relevant.
5. Run the smallest focused tests plus the affected end-to-end smoke path.
6. Update hashes/versioned manifests only by the approved candidate flow.
7. Update the launch reference, maintenance notes, and Repository Source Index.

Chapter 6

Launch and Runtime Composition

Chapter guide

Purpose. Describe the supported entry points, selected defaults, process ownership, startup sequence, isolation rules, runtime files, and clean shutdown order.

Recommended use. Changing launch composition, diagnosing a missing process or late topic, comparing profiles, or determining which process created a runtime artifact.

6.1 Chapter Source Map

Table 6.1: Authoritative sources for launch and runtime composition

Topic	Authoritative repository path(s)
Stable profiles and common floor policy	mfja_3rd_floor_bringup/launch/room_315_only.launch.py; mfja_3rd_floor_bringup/launch/full_floor.launch.py; mfja_3rd_floor_bringup/launch/single_industrial_robot. launch.py;mfja_3rd_floor_bringup/launch/room_315_floor _common.py
Base Gazebo and robot composition	mfja_robot_control_config/launch/multi_robot_sim.launc h.py;mfja_robot_control_config/launch/room_315_launch _utils.py;mfja_robot_control_config/launch/gripper_ran ge_config.py;mfja_robot_control_config/scripts/convey or_loop_mode_controller.py
Rail and optional VLA composition	mfja_robot_control_config/launch/room_315_dual_kinemat ic_shuttles.launch.py;mfja_robot_control_config/launch /room_315_vla_supervisor.launch.py;mfja_robot_control _config/launch/room_315_visual_state_runtime.launch.py; mfja_robot_control_config/launch/room_315_task_executi on.launch.py
Task-gateway singleton and shutdown	mfja_robot_control_config/scripts/room_315_task_execut ion_node.py;mfja_robot_control_config/scripts/room_315 _closed_loop_executive.py
Persistent dataset out- put	mfja_robot_control_config/scripts/room_315_vla_dataset _recorder.py

6.2 Stable Entry Points

Operators should normally start one of the bringup-package wrappers:

Table 6.2: Stable operator entry points and their default simulation profiles

Entry point	World	Initial state	Intended use
room_315_only.launch.py	room_315_only	Running	Rail, robot, camera, VLA, and maintenance work in the focused cell

Entry point	World	Initial state	Intended use
<code>full_floor.launch.py</code>	<code>mfja_3rd_floor</code>	Paused	Complete-floor demonstration and integration
<code>single_industrial_robot.launch.py</code>	<code>isolated_industrial_robot</code>	Running	Fast arm/gripper work with one selected industrial robot

Both floor wrappers call `mfja_3rd_floor_bringup/launch/room_315_floor_common.py`. Profile defaults differ, but the composition and public argument names stay aligned. The isolated wrapper delegates to `mfja_robot_control_config/launch/isolated_industrial_robot.launch.py`.

6.3 Launch Profile Defaults

Table 6.3: Default values selected by the two stable floor profiles

Setting	Room 315 only	Full floor
World file/entity	<code>room_315_only</code>	<code>mfja_3rd_floor</code>
Robot registry	<code>mfja_robot_control_config/room_315_obots_room_315_obots.nly.yaml</code>	<code>mfja_robot_control_config/room_315_obots.yaml</code>
GUI profile	<code>mfja_robot_control_config/room_315_runtime_safe.gui.config</code>	<code>mfja_robot_control_config/mfja_light.gui.config</code>
Start paused	false	true
Rails	both enabled	both enabled
Initial shuttle count	zero per side	zero per side
Initial shuttle drive	disabled	disabled
VLA camera/supervisor	disabled	disabled
Removable VLA obstacles	disabled	disabled
Obstacle cache cleanup	enabled	enabled
Device/payload markers	enabled	enabled

The wrapper and shared-common files in Table 6.1 own these defaults. If `robots` is empty, enabled flags in the selected registry decide what spawns. `robots:=all` includes disabled entries, `robots:=none` produces a rail-only run, and a comma list accepts exact names, model/short aliases, or one-based YAML indices. Ambiguous or unknown selectors fail before spawn; the selector implementation is `mfja_robot_control_config/launch/multi_robot_sim.launch.py`.

6.4 Base Gazebo Composition

`mfja_robot_control_config/launch/multi_robot_sim.launch.py` performs the following work in order:

1. Resolve the selected world and read its internal entity name.
2. Optionally write an obstacle-free temporary world without modifying the checked-in source.
3. Prepend the description model directory to `GZ_SIM_MODEL_PATH` and `GZ_SIM_RESOURCE_PATH`.
4. Set the selected Gazebo transport partition.
5. Start the Gazebo server; add `-r` only when the profile is not paused.
6. Start `mfja_robot_control_config/scripts/conveyor_loop_mode_controller.py` as the visual switch sink.
7. Optionally start the Gazebo GUI with the selected GUI config.
8. After three seconds, bridge `/clock` and the world `set_pose`, `create`, and `remove` services.
9. Load, validate, select, and spawn every requested robot; generate its bridge mapping and start its namespaced state publisher.

The full-floor profile starts paused. Simulation-time timers, trajectory controllers, rail motion, delayed switch/stopper updates, and many freshness checks do not advance until Gazebo is unpaused.

6.5 Runtime Process Inventory

Table 6.4: Processes participating in base and optional runtime compositions

Process	Started by	Responsibility	Launch or implementation authority
Gazebo server/GUI	Base simulation launch	World, entities, physics/rendering, Gazebo transport/services	<code>mfja_robot_control_config/launch/multi_robot_sim.launch.py</code>
<code>conveyor_loop_mode_controller</code>	Base simulation launch	Visual switch actuator and feedback adapter; synchronized feedback can affect routing	<code>mfja_robot_control_config/scripts/conveyor_loop_mode_controller.py</code>
Clock/world bridge	Delayed base actions	ROS clock and entity pose/create/remove services	<code>mfja_robot_control_config/launch/multi_robot_sim.launch.py</code>
Per-robot create node	Delayed base actions	Spawn one configured entity	<code>mfja_robot_control_config/launch/multi_robot_sim.launch.py</code>
Per-robot state publisher	Delayed base actions	Namespaced, instance-prefixed URDF TF tree	<code>mfja_robot_control_config/launch/multi_robot_sim.launch.py</code>
Per-robot parameter bridge	Delayed base actions	Joint/gripper/base commands and state feedback	<code>mfja_robot_control_config/launch/multi_robot_sim.launch.py</code>
Two rail nodes	Dual rail launch	Authoritative logical rail motion, lifecycle, devices, sensors, markers, pose calls	<code>mfja_robot_control_config/launch/room_315_dual_kinematic_shuttles.launch.py</code> ; <code>mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py</code>
VLA supervisor	Optional VLA launch	Primitive validation, safety gates, typed rail command publication	<code>mfja_robot_control_config/launch/room_315_vla_supervisor.launch.py</code> ; <code>mfja_robot_control_config/scripts/room_315_vla_supervisor.py</code>
VLA recorder	Optional VLA launch	Episode/camera/action recording	<code>mfja_robot_control_config/scripts/room_315_vla_dataset_recorder.py</code>

Process	Started by	Responsibility	Launch or implementation authority
Visual inference node	Separate visual launch	Verified V4 inference and accepted visual observations	<code>mfja_robot_control_config/launch/room_315_visual_state_runtime.launch.py</code> ; <code>mfja_robot_control_config/scripts/room_315_visual_state_inference_node.py</code>
PlanSys2 planner	Separate task launch	Lifecycle-managed PDDL planner service	<code>mfja_robot_control_config/launch/room_315_task_execution.launch.py</code>
Task gateway	Separate task launch	Authorization, grounding, one-step planning/execution/effect verification	<code>mfja_robot_control_config/scripts/room_315_task_execution_node.py</code>
English CLI	Separate operator terminal	Parse, clarify, confirm, publish TaskGoal, display result	<code>mfja_robot_control_config/scripts/room_315_task_goal_cli.py</code>

6.6 Floor Launch Sequence

Table 6.5: Ordered floor-startup stages and their implementation sources

Delay	Stage	Work started	Source file(s)
0 s	Host guard	Acquire the advisory floor lock and apply obstacle-cache policy	<code>mfja_3rd_floor_bringup/launch/room_315_floor_common.py</code>
0 s	Base simulation	Start Gazebo and the visual switch controller	<code>mfja_robot_control_config/launch/multi_robot_sim.launch.py</code>
3 s	ROS integration	Start clock/world bridges, robot spawners, per-robot bridges, and TF publishers	<code>mfja_robot_control_config/launch/multi_robot_sim.launch.py</code>
4 s	Rail runtime	Start the right and left authoritative rail nodes	<code>mfja_3rd_floor_bringup/launch/room_315_floor_common.py</code> ; <code>mfja_robot_control_config/launch/room_315_dual_kinematic_shuttles.launch.py</code>

Delay	Stage	Work started	Source file(s)
5 s, optional	VLA branch	Start camera bridges, deterministic supervisor, and optional recorder	mfja_3rd_floor_bringup/launch/room_315_floor_common.py; mfja_robot_control_config/launch/room_315_vla_supervisor.launch.py

Treat topics and services that appear during this five-second sequence as delayed startup unless the relevant stage has exceeded its configured gate.

What the high-level VLA switch does not start

`enable_room315_vla:=true` starts camera bridges and the deterministic primitive supervisor; it can also start the recorder. It does not start the learned V4 inference node, PlanSys2 planner, English CLI, or task-execution gateway. Those remain separate, artifact-dependent launches listed in Table 6.4.

6.7 Singleton Locks and Namespace Isolation

The floor composition locks `/tmp/mfja_room315_floor_runtime.lock` with `fcntl.flock`. Only one high-level Room 315/full-floor process is supported on one host because the rail ROS names and world services are fixed. A unique Gazebo `GZ_PARTITION` isolates transport, but does not rename the ROS graph.

A lock pathname may remain after a clean shutdown; the operating-system lock is released when the descriptor closes. Do not delete or bypass the lock while an owner is alive. Stop the owner. The task-execution node has a second singleton lock at `/tmp/mfja_room315_task_execution_node.lock` to prevent concurrent gateways. The owning sources are `mfja_3rd_floor_bringup/launch/room_315_floor_common.py` and `mfja_robot_control_config/scripts/room_315_task_execution_node.py`.

6.8 Temporary and Persistent Runtime Files

Table 6.6: Generated runtime files, user data, and their maintenance rules

Location	Contents	Rule	Creating source
<code>/tmp/<robot>_bridge.yaml</code>	Generated per-robot bridge map	Inspect for diagnosis; never edit as source	<code>mfja_robot_control_config/launch/multi_robot_sim.launch.py</code>
<code>/tmp/<robot>_mobile_model.sdf</code>	Instance-specific mobile topic/frame substitutions	Temporary	<code>mfja_robot_control_config/launch/multi_robot_sim.launch.py</code>
<code>/tmp/mfja*_gripper_range*.sdf</code>	Launch-patched gripper limits	Temporary	<code>mfja_robot_control_config/launch/gripper_range_config.py</code>
<code>/tmp/*_without_room315_vla_obstacles*.world</code>	Filtered world copy	Temporary	<code>mfja_robot_control_config/launch/multi_robot_sim.launch.py</code>
<code>~/ros/log/</code>	ROS logs	Preserve when diagnosing; otherwise disposable	ROS 2 runtime

Location	Contents	Rule	Creating source
<code>~/.ros/room315_vla_obstacles.json</code>	Optional obstacle pose cache	Cleared by default; override only to an intended file	<code>mfja_3rd_floor_bringup/launch/room_315_floor_common.py</code>
<code>~/.ros/room315_visual_state_datasets/<run>/</code>	Recorded episodes and frames	User data outside Git; back up deliberately	<code>mfja_robot_control_config/scripts/room_315_vla_dataset_recorder.py</code>
External selected directories	Datasets, checkpoints, manifests, authorization, reports	Preserve hashes/provenance; never edit promoted artifacts in place	Runtime paths in the V4 configuration files

6.9 Shutdown Semantics

For an actuating rail task, send a controlled `OFF` or supervisor `stop_all`, wait for explicit `DISABLED` feedback, stop interactive helpers, then press `Ctrl-C` in the high-level launch. The task gateway publishes `stop_all` during exception or shutdown handling when an active goal exists. `Ctrl-C` should propagate so the floor lock releases and child processes exit. The owning implementation is `mfja_robot_control_config/scripts/room_315_task_execution_node.py`, `mfja_robot_control_config/scripts/room_315_closed_loop_executive.py`, and `mfja_3rd_floor_bringup/launch/room_315_floor_common.py`.

Part III

Subsystem Design and Closed-Loop Execution

Chapter 7

Simulation and Robot Control

Chapter guide

Purpose. Explain how Gazebo assets are resolved, how robot instances are selected and isolated, and how validated commands cross the ROS–Gazebo boundary.

Recommended use. Adding or selecting a robot, diagnosing missing assets or topics, changing arm/gripper behavior, or evaluating simulation-specific limitations.

7.1 Chapter Source Map

Table 7.1: Authoritative sources for simulation and robot control

Topic	Authoritative repository path(s)
World and model resolution	<code>mfja_3rd_floor_description/CMakeLists.txt</code> ; <code>mfja_3rd_floor_description/worlds/</code> ; <code>mfja_3rd_floor_description/models/</code> ; <code>mfja_robot_control_config/launch/multi_robot_sim.launch.py</code>
Robot registry and selection	<code>mfja_robot_control_config/config/robots.yaml</code> ; <code>mfja_robot_control_config/config/robots_room_315_only.yaml</code> ; <code>mfja_robot_control_config/launch/multi_robot_sim.launch.py</code>
Joint command and trajectory control	<code>mfja_robot_control_config/scripts/robot_joint_command.py</code> ; <code>mfja_3rd_floor_description/src/SmoothJointTrajectoryController.cc</code> ; industrial and TIAGo model.sdf plugin declarations
Gripper configuration and control	<code>mfja_robot_control_config/config/gripper_command_defaults.yaml</code> ; <code>mfja_robot_control_config/launch/gripper_range_config.py</code> ; <code>mfja_robot_control_config/scripts/robot_gripper_command.py</code> ; <code>mfja_3rd_floor_description/src/SymmetricGripperController.cc</code>
TIAGo frame isolation	<code>mfja_robot_control_config/launch/multi_robot_sim.launch.py</code> ; <code>mfja_3rd_floor_description/models/tiago/model.sdf</code> ; <code>mfja_3rd_floor_description/models/tiago_base/model.sdf</code> ; <code>mfja_3rd_floor_description/urdf/tiago.urdf</code> ; <code>mfja_3rd_floor_description/urdf/tiago_base.urdf</code>
Industrial collision boundary	<code>mfja_3rd_floor_description/models/kuka_kr6r900sixx/model.sdf</code> ; <code>mfja_3rd_floor_description/models/staubli_tx2_60l/model.sdf</code> ; <code>mfja_3rd_floor_description/models/yaskawa_hc10/model.sdf</code> ; <code>mfja_3rd_floor_description/models/yaskawa_hc10dt/model.sdf</code>

7.2 Asset and Model Resolution

Every maintained reusable Gazebo model normally has this form:

```
mfja_3rd_floor_description/models/<model_name>/
```

```
model.config
model.sdf
meshes/... # where required
```

World SDF includes models with `model://...` URIs. The launch prepends the description model directory to Gazebo's model and resource paths while preserving any pre-existing entries. A missing model/mesh error usually means the description package was not built/sourced, a URI does not match its model directory, or a local environment overwrote the resource path.

The main worlds include static room/furniture/cell assets. Robots are not fixed world composition: they are selected from YAML and spawned after Gazebo starts. Eight identity-specific Room 315 shuttle models are preloaded below the scene and are revealed or managed by the rail nodes.

7.3 Robot Instances and Runtime Boundary

7.3.1 Robot registry

The authoritative instance, model, spawn-pose, and profile-default mapping is kept once in Table 5.3. Its owning configuration files are `mfja_robot_control_config/config/robots.yaml` and `mfja_robot_control_config/config/robots_room_315_only.yaml`; this chapter focuses on how those selected instances cross the runtime boundary.

The full-floor registry enables all six listed instances. TIAGo aliases include `tiago`, `tiago_with_arm`, and `tiago_arm`; base aliases include `tiago_base`, `tiago_no_arm`, and `tiago_mobile_base`. A mobile entry may also use `variant/type` fields or an explicit `arm` boolean. Names must be unique because each name becomes a Gazebo entity, ROS namespace, bridge-file name, and TF prefix.

7.3.2 Per-robot data flow

Table 7.2: Per-instance robot command and feedback boundaries

Stage	Direction	Responsibility	Source file(s)
Validated helper	Operator → ROS	Resolve one registry instance, validate the request, and publish only to its namespace	<code>mfja_robot_control_config/scripts/robot_joint_command.py</code> ; <code>mfja_robot_control_config/scripts/robot_gripper_command.py</code>
Generated bridge	ROS ↔ Gazebo	Map instance-specific command, state, odometry, and TF topics	<code>mfja_robot_control_config/launch/multi_robot_sim.launch.py</code>

Stage	Direction	Responsibility	Source file(s)
Model controller	Gazebo transport → model	Apply bounded joint, gripper, or mobile-base motion inside Gazebo	mfja_3rd_floor_description/src/SmoothJointTrajectoryController.cc; mfja_3rd_floor_description/src/SymmetricGripperController.cc; selected model model.sdf
Feedback lane	Gazebo → ROS	Return joint state, progress, odometry, and TF through the same instance boundary	mfja_robot_control_config/launch/multi_robot_simulation.launch.py; selected URDF/model SDF

There is no global broadcast joint-command topic.

7.3.3 Canonical robot ROS topics

Replace `<name>` with the registry instance name.

Table 7.3: Canonical namespaced ROS topics exposed for robot instances

ROS name/type	Direction	Availability
/<name>/joint_trajectory — trajectory_msgs/JointTrajectory	ROS → Gazebo	Every robot
/<name>/joint_trajectory_progress — std_msgs/Float32	Gazebo → ROS	Every robot using the smooth plugin
/<name>/joint_states — sensor_msgs/JointState	Gazebo → ROS	Every robot
/<name>/tf, /<name>/tf_static	State publisher/Gazebo → ROS	Every robot; mobile TF also comes from DiffDrive
/<name>/gripper/position_command — std_msgs/Float64	ROS → Gazebo	Industrial robots
/<name>/cmd_vel — geometry_msgs/Twist	ROS → Gazebo	TIAGo variants
/<name>/odom — nav_msgs/Odometry	Gazebo → ROS	TIAGo variants

7.4 Industrial Arm Control

7.4.1 Joint command helper

`robot_joint_command.py` resolves the robot profile, validates joint names/counts/limits, waits for a complete live joint state, and publishes once a dense quintic trajectory sampled at 100–200 Hz to the selected namespace. Inspect its live help because supported options are executable authority:

```
ros2 run mfja_robot_control_config robot_joint_command.py --help
ros2 run mfja_robot_control_config robot_joint_command.py --list
```

Use its dry-run/validation path before a new pose. The helper’s dense message is interpolated as a multi-point trajectory by the smooth controller, which retimes motion when necessary to remain within configured joint velocity limits. The controller rejects unknown/duplicate joints, non-finite values, mismatched point sizes, non-monotonic times, and positions outside SDF limits.

7.4.2 Trajectory controller semantics

`SmoothJointTrajectoryController.cc` is deterministic and simulation-time driven. It uses joint position/velocity reset components on every step, then rewrites public joint state after physics so gravity/solver residue does not leak into feedback.

- One-point trajectories are goals: current state to target through a quintic minimum-jerk blend.
- Multi-point trajectories are timestamped samples and interpolate linearly between points after velocity-limit retiming.
- A newer valid message replaces the active trajectory.
- Commands received while paused are retained; progress starts when simulation time advances.
- Progress is published on the Gazebo trajectory topic plus `_progress`, at no more than one update per 0.05 simulated seconds.

7.5 Gripper Configuration and Control

`gripper_command_defaults.yaml` declares, per industrial instance, the per-jaw position at 0% and 100%, plus default open/close percentages. Current 100% per-jaw travels are 0.030 m (KUKA), 0.010 m (Stäubli), 0.040 m (HC10), and 0.010 m (HC10DT).

At launch, `gripper_range_config.py` validates the selected robots and config, then materializes temporary SDF/URDF content with consistent joint and plugin limits. `robot_gripper_command.py` maps an operator percentage or bounded position into `/<name>/gripper/position_command`. Inspect before actuation:

```
ros2 run mfja_robot_control_config robot_gripper_command.py --help
```

Gripper realism

The symmetric plugin animates two opposite-axis jaws at bounded velocity. It does not create a grasp constraint, attach a payload, or validate contact force. A visually enclosed object is not a physically held object.

7.6 TIAGo Frame Isolation

For each mobile instance, the launch creates an instance-specific SDF copy that rewrites `cmd_vel`, odometry, TF, frame, and child-frame names. Gazebo DiffDrive publishes `<name>/odom-><name>/base_link`; the namespaced state publisher adds the remaining tree with `frame_pref`

`ix=<name>/`. Multiple TIAGo variants can therefore share a graph without colliding on `odom`, `base_link`, or downstream frames.

7.7 Robot Model Limitations

Industrial assets are optimized for visual integration and intentionally use collision simplification; some are collision-free. Visual overlap is not proof of safe physical clearance. Commands are not ROS 2 control actions and there is no hardware controller manager in this stack. Before adapting a robot to hardware, replace the simulation bridge/controller boundary and establish hardware limits, collision monitoring, e-stop behavior, and vendor-specific safety integration independently.

Chapter 8

Room 315 Rail Transport

Chapter guide

Purpose. Explain the rail model from its configuration inputs and coordinate transforms through routing, devices, shuttle lifecycle, motion guards, and simulation-only presentation.

Recommended use. Changing rail geometry or devices, diagnosing route or pose disagreement, extending shuttle behavior, or verifying which state is authoritative.

8.1 Chapter Source Map

Table 8.1: Authoritative sources for the Room 315 rail transport system

Topic	Authoritative repository path(s)
Rail launch and runtime integration	<code>mfja_robot_control_config/launch/room_315_dual_kinematic_shuttles.launch.py</code> ; <code>mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py</code> ; <code>mfja_robot_control_config/scripts/room_315_kinematic_shuttle.py</code>
Directed network and sampled geometry	<code>mfja_robot_control_config/config/room_315_kinematics/rail_network_right.yaml</code> ; <code>mfja_robot_control_config/config/room_315_kinematics/rail_network_left.yaml</code> ; <code>mfja_robot_control_config/config/room_315_kinematics/raw_segments/</code> ; <code>mfja_robot_control_config/scripts/room_315_kinematic_shuttle.py</code>
Calibration and public-name mapping	<code>mfja_robot_control_config/scripts/room_315_rail_defaults.py</code> ; <code>mfja_robot_control_config/scripts/room_315_device_position_tool.py</code> ; <code>mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py</code>
Slots, switches, stoppers, and sensors	<code>mfja_robot_control_config/config/room_315_kinematics/rail_devices_right.yaml</code> ; <code>mfja_robot_control_config/config/room_315_kinematics/rail_devices_left.yaml</code> ; <code>mfja_robot_control_config/scripts/room_315_rail_devices.py</code> ; <code>mfja_robot_control_config/scripts/conveyor_loop_mode_controller.py</code>
Identity and lifecycle	<code>mfja_robot_control_config/scripts/room_315_multi_shuttle.py</code> ; <code>mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py</code> ; <code>mfja_rail_interfaces/srv/AddShuttle.srv</code> ; <code>mfja_rail_interfaces/msg/ShuttleState.msg</code>
Commands, states, and headway	<code>mfja_robot_control_config/scripts/room_315_kinematic_shuttle.py</code> ; <code>mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py</code> ; <code>mfja_rail_interfaces/msg/ShuttleCommand.msg</code> ; <code>mfja_3rd_floor_bringup/launch/room_315_floor_common.py</code>
Markers and payload visuals	<code>mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py</code> ; <code>mfja_robot_control_config/scripts/room_315_rail_defaults.py</code> ; <code>mfja_3rd_floor_description/models/room315_vla_overhead_devices/model.sdf</code> ; <code>mfja_3rd_floor_description/models/room315_vla_payload_medium_box/model.sdf</code> ; <code>mfja_3rd_floor_description/models/room315_vla_removeable_obstacle_marker/model.sdf</code>

8.2 System Model and Authorities

8.2.1 Modelling decision

Shuttle propulsion is kinematic. Each rail node loads directed topology and calibrated CSV samples, advances a logical shuttle by arc length, transforms the logical pose into the Gazebo frame, and calls the world `SetEntityPose` service. No wheel torque or rail-contact dynamics propel a shuttle.

This design makes routing, devices, datasets, and effect checks deterministic, but it has a crucial consequence: logical rail state may continue advancing if a pose service is missing or times out. Monitor diagnostics and Gazebo together with ROS state when visual/logical agreement matters.

8.2.2 Authoritative input files

Each rail side loads five source layers in the following order:

1. `mfja_robot_control_config/config/room_315_kinematics/rail_network_<side>.yaml`: nodes, 14 forward-only segments, switches, fixed transitions, and the routing table;
2. `mfja_robot_control_config/config/room_315_kinematics/raw_segments/*.csv`: index, x, y, z samples for each public segment filename;
3. `mfja_robot_control_config/config/room_315_kinematics/rail_devices_<side>.yaml`: slots, binary sensors, stoppers, radii, and default states;
4. `mfja_robot_control_config/scripts/room_315_rail_defaults.py`: public topics, identity/entity defaults, left-name mapping, Gazebo calibration, and marker styles; and
5. identity and model conventions from `mfja_robot_control_config/scripts/room_315_multi_shuttle.py` and the identity-bearing shuttle models in `mfja_3rd_floor_description/models/`.

The explicit `csv:raw_segments/<segment>.csv` field is authoritative. Do not derive filenames from keys even though normalized live files currently match. A narrow compatibility table exists only for immutable legacy snapshots. The loaders are `mfja_robot_control_config/scripts/room_315_kinematic_shuttle.py`, `mfja_robot_control_config/scripts/room_315_rail_devices.py`, and `mfja_robot_control_config/scripts/room_315_rail_defaults.py`.

8.3 Static Rail Definition

8.3.1 Geometry and path sampling

`mfja_robot_control_config/scripts/room_315_kinematic_shuttle.py` provides two backends:

- `cubic_hermite` (default): continuous interpolation from sampled positions and tangents, reparameterized by numerically sampled arc length; and
- `polyline`: direct piecewise-linear interpolation for comparison and calibration debugging.

Each segment stores length L ; devices use normalized $s_{ratio} \in [0, 1]$ and resolve $s = s_{ratio}L$. Motion advances $\Delta s = v\Delta t$, consuming residual distance across valid successor segments. Every segment follows CSV index 0 to the final row. Reverse traversal is not supported.

8.3.2 Calibration into Gazebo coordinates

Logical CSV coordinates are first transformed by a side-specific affine map, then scaled around a configured origin, rotated around a configured origin, and offset. In compact form:

$$\begin{bmatrix} x_b \\ y_b \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}, \quad \mathbf{p}_{gz} = R(\theta) S(\mathbf{p}_b - \mathbf{o}_s) + \mathbf{o}_r + \mathbf{t}.$$

Exact coefficients live in `mfja_robot_control_config/scripts/room_315_rail_defaults.py`. Runtime scale, rotation, and offsets are node parameters. Use `mfja_robot_control_c`

`onfig/scripts/room_315_device_position_tool.py` to convert a Gazebo point into a candidate segment+s_ratio; never tune topology to compensate for a calibration error.

8.3.3 Directed topology and switch guards

The right public cycle has two guarded routing stages:

Table 8.2: Right-rail directed routing stages and matching convergence guards

Stage	Diverging path	Required convergence
A3/A4 half	A23 → A3E/A3I → A34E/A34I	Matching A4 exterior/interior guard → A4E/A4I → A14
A1/A2 half	A14 → A1E/A1I → A12E/A12I	Matching A2 exterior/interior guard → A2E/A2I → A23

A diverging switch selects an outgoing branch. A converging gate must match the branch on which the shuttle arrives. An unknown state, missing successor, or mismatched guard has no valid successor and sets `FALLING`; the runtime does not teleport or silently select another path.

8.3.3.1 Left internal versus public names

The left rail reuses the raw geometry but rotates its public station/switch vocabulary. Keep conversion direction explicit:

Table 8.3: Mapping between raw/internal and public left-rail names

Internal/raw group	Public left group	Meaning
A3E/A3I, switch at raw A3	A1E/A1I, public A1	First public left diverger
A4E/A4I, raw A4	A2E/A2I, public A2	First public left converger
A1E/A1I, raw A1	A3E/A3I, public A3	Second public left diverger
A2E/A2I, raw A2	A4E/A4I, public A4	Second public left converger
A34E/A34I	A12E/A12I	Upper/lower exterior/interior long branch mapping
A12E/A12I	A34E/A34I	Opposite long branch mapping
A14	A23	Trunk mapping
A23	A14	Opposite trunk mapping

Use `internal_rail_segment_name_to_public` when publishing internal state and `public_rail_segment_name_to_internal` when resolving public input. The current map happens to be an involution in several pairs; that does not make the two functions interchangeable. The mapping authorities are `mfja_robot_control_config/config/room_315_kinematics/rail_network_left.yaml` and `mfja_robot_control_config/scripts/room_315_rail_defaults.py`.

8.4 Devices and Occupancy

8.4.1 Slots and identity-bearing DZI sensors

Table 8.4: Configured slot and identity-bearing DZI sensor locations

Side	Slot/sensor	Raw segment	s_{ratio}	Radius
Right	slot 1 / DZI1R	A12E	0.411866742	0.09 m
Right	slot 2 / DZI2R	A12E	0.653073633	0.09 m
Right	slot 3 / DZI3R	A34E	0.447469343	0.09 m
Right	slot 4 / DZI4R	A34E	0.683726992	0.09 m
Left	slot 1 / DZI1L	raw A34E	0.428330934	0.09 m
Left	slot 2 / DZI2L	raw A34E	0.674370792	0.09 m
Left	slot 3 / DZI3L	raw A12E	0.427586575	0.09 m
Left	slot 4 / DZI4L	raw A12E	0.668424664	0.09 m

The device YAML also defines DA approach/branch sensors around every switch. Single-point devices store `segment` and `s_ratio`; multi-point devices store a `points` list. Each stopper has a derived `A<n>_STOPPER_SENSOR` located 0.10 m upstream along each stopper point. Only segment, ratio/points, and radius determine occupancy. Descriptive `switch` or `branch` labels do not move a detector.

8.4.2 Switches, stoppers, and delayed state

Typed switch and stopper commands schedule actual-state changes at current simulation time plus configured delay (0.3 s switch, 0.1 s stopper by high-level default). State topics report the delayed actual value. Routing uses actual switch state, not the requested command.

Stopper state 0 means open/pass; 1 means closed/stop. A closed stopper clamps an enabled shuttle at its trigger and places it in `WAITING`; it resumes when the guard clears. Sensor output is binary occupancy, including swept crossing between ticks. The additional `segment`, `s`, and `s_ratio` fields are privileged simulator metadata for debugging/evaluation, not extra physical measurements.

Accepted delayed switch actual state is mirrored as a comma-separated `selector=EXTERIOR/INTERIOR` string on `/mfja/conveyor/switch_cmd`. The visual switch controller updates blade models and publishes a latched summary on `/mfja/conveyor/switch_states`. While a typed target is outstanding, matching feedback is only an acknowledgement and conflicting feedback is ignored as stale. With `sync_from_visual_switch_states=true`, however, an unmatched visual summary can schedule a delayed update of the logical switch registry and therefore affect routing. Treat this subscription as configured state synchronization, not presentation-only telemetry; Chapter 3 documents the cross-boundary authority in detail.

8.5 Shuttle Runtime

8.5.1 Identity and lifecycle

Canonical identities are `R1--R4` and `L1--L4`, with Gazebo entities `room315_right_shuttle_1--4` and `room315_left_shuttle_1--4`. Startup selection modes are:

- `count_prefix`: count n selects the first n identities; and

- **explicit**: exact comma-separated identities select an arbitrary subset and must agree with the declared count.

Initial start slots/positions must match the selected count and be unique when occupancy rejection is enabled. Zero is a valid count and produces a heartbeat with an empty name so the presence provider can distinguish a known-empty rail from a missing or stale rail.

The `AddShuttle` service resolves an empty name to the next unused entity and an empty slot to the next available configured slot. A requested speed that compares greater than zero is used; otherwise the configured default is substituted, with no separate finiteness validation. Duplicate entities, invalid slots, and—when enabled—occupied start slots are rejected. A non-preloaded model is spawned asynchronously; a successful service reply means the logical shuttle was accepted, not that Gazebo has already rendered it. `REMOVE` removes logical management immediately and requests asynchronous Gazebo deletion; a failed delete may leave a visual orphan.

8.5.2 State machine and commands

The command-driven transitions are separated from autonomous guard and target transitions to make command verification unambiguous.

Table 8.5: Command-driven shuttle transitions and required verification

Command	Applicable state	Result	Verification and semantic
ON	DISABLED, WAITING, or FALLING	MOVING with positive retained or supplied speed	Require a newer state; the current code can clear FALLING without homing
OFF	Any managed mode	DISABLED	Clear the exact target, retain configured speed, and require newer explicit feedback
RESET	Any managed mode with a free start slot	MOVING, WAITING, or DISABLED	Verify the start pose and resulting mode; enabled state and retained speed are preserved
REMOVE	Any managed mode	Removed from registry	Check both logical registry/state and Gazebo entity deletion

Table 8.6: Autonomous shuttle transitions caused by guards and targets

Current state	Condition	Result	Semantic
MOVING	Stopper or headway guard	WAITING	Hold position until the active guard clears
WAITING	Guard clear with positive speed	MOVING	Resume the retained command
MOVING	Exact target reached	DISABLED	Clamp to target and populate <code>reached_target_slot</code>
MOVING	No valid guarded successor	FALLING	Stop core stepping; do not choose another successor

`ShuttleState.speed` is the retained commanded travel setting, not instantaneous velocity. It may remain non-zero in `DISABLED` or `WAITING`. `reached_target_slot` is populated only when the low-level controller clamps to an explicit target and disables the shuttle.

8.5.3 Headway control and failure behavior

Before accepting a motion step, the node predicts center separation among managed shuttles and uses bounded search to clamp progress before the configured minimum (0.33 m by default). A held shuttle reports `WAITING` and resumes when clear. This is a software center-distance rule, not certified block protection or complete rigid-body collision avoidance.

Fail-closed rail conditions

Unknown or mismatched route state enters `FALLING` and stops core stepping, but the current positive-speed `ON` path can clear that mode without a reset. Treat that behavior as a safety defect and recover only through a verified supervisor procedure. An occupied start can reject add/reset; stale typed feedback must not be replaced with a fixed delay. Low-level typed topics can bypass planning, so manual publishers remain an expert interface.

8.6 Simulation-only Markers and Payloads

When enabled, the rail node spawns collision-free visual markers gradually via the world create service. Sensors are blue/inactive and green/active; stoppers are amber/open and red/closed; switches use green/interior and orange/exterior in debug mode; a shuttle is red in `FALLING`. Dynamic remove/recreate refresh is disabled by default to avoid races with pending entity creation.

Payload command/state topics use JSON because payload visuals are privileged simulation/dataset controls, not the public physical rail contract. A payload box follows its shuttle with configured offsets. Spawn/delete failure does not change logical payload state, and no payload physics or grasp is simulated.

Chapter 9

Visual State, Planning, and Closed-Loop Execution

Chapter guide

Purpose. Explains the optional learned-visual and task-execution stack, including its deterministic safety boundary and immutable deployment lifecycle.

Recommended use. Follow the live path first—visual observation, confirmed goal, planning, supervised primitive, and verified effect—then use the offline sections for dataset, promotion, authorization, and controlled change work. Base simulation and direct rail control do not require this optional stack.

9.1 Layering and Safety Intent

The optional research stack deliberately separates perception, language, planning, supervision, and actuation. No learned component publishes a typed rail command. The deterministic supervisor is the only component in this stack that does so.

Table 9.1: Closed-loop runtime stages and authority boundaries

Stage	Exchange	Authority/gate	Source file(s)
Observation readiness	Gazebo effect → paired images/presence → accepted V4 state	Promoted model, validator, presence/fusion readiness	mfja_robot_control_config/scripts/room_315_visual_state_inference_node.py; mfja_robot_control_config/scripts/room_315_visual_runtime_validation.py
Goal intake	Constrained English → confirmed TaskGoal	Parser, validation, explicit operator confirmation	mfja_robot_control_config/scripts/room_315_task_goal_cli.py; mfja_robot_control_config/scripts/room_315_task_goal_builder.py
Grounding	Fresh accepted observation → concrete task facts	Task gateway schema/checkpoint/freshness gates	mfja_robot_control_config/scripts/room_315_task_execution.py

Stage	Exchange	Authority/gate	Source file(s)
Planning	Fresh PDDL problem ↔ PlanSys2 plan	Runtime domain plus whole-plan validation	mfja_robot_control_config/config/room_315_vla/pddl/domain_room315_runtime.pddl;mfja_robot_control_config/scripts/room_315_pddl_plan_translator.py
Primitive execution	First translated primitive ↔ supervisor status	Gateway authorization and deterministic supervisor	mfja_robot_control_config/scripts/room_315_task_execution.py;mfja_robot_control_config/scripts/room_315_vla_supervisor.py
Typed actuation	Shuttle/switch/stopper command ↔ actual state	Rail controller and typed interfaces	mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py;mfja_rail_interfaces/CMakeLists.txt
Effect verification	Newer visual/controller evidence → success or replan	Gateway effect predicates and bounded retries; the loop returns to observation readiness	mfja_robot_control_config/scripts/room_315_task_execution.py;mfja_robot_control_config/scripts/room_315_visual_state_inference_node.py

9.2 V4 Visual-State Runtime

Source files: mfja_robot_control_config/launch/room_315_visual_state_runtime.launch.py, mfja_robot_control_config/scripts/room_315_visual_state_inference_node.py, mfja_robot_control_config/scripts/room_315_visual_runtime_v4.py, and mfja_robot_control_config/config/room_315_vla/visual_state_runtime.yaml.

room_315_visual_state_runtime.launch.py starts the separate learned runtime. Only generation V4 is admitted to the active path. A promoted runtime must supply the exact manifest and checkpoint identities expected by config; missing files, wrong hashes, wrong schema, stale inputs, rejected predictions, or unsafe supervisor status fail closed.

The inference node consumes:

- synchronized left/right RGB images;
- each rail's ShuttleState only for identity presence, timestamp, receive time, and known-empty heartbeat; and
- deterministic supervisor safety status.

It must not use controller segment, arc length, world pose, slot, payload state, or privileged identity-tracker geometry as learned inputs. This separation prevents evaluation leakage. Presence requires both rail heartbeats, a 0.5 s warm-up, and freshness no older than 1.0 s under current defaults. A blank-name heartbeat is known-empty; no heartbeat is unknown/stale.

The node publishes typed raw prediction, validation, and accepted observation streams plus diagnostic JSON/model output. The default runtime mode is `active`, but `dry_run_state_fusion=true` and `plansys2_update_enabled=false`; therefore merely starting visual runtime does not mutate PlanSys2 knowledge. The dry-run flag gates external predicate writes; it does not suppress internal fusion or accepted-observation publication. `shadow` mode writes under `/room_315/visual_state/shadow_v4/...` and is observation-only.

9.3 Visual Observation Contract

Source files: `mfja_rail_interfaces/msg/VisualStateObservation.msg`, `mfja_rail_interfaces/msg/VisualShuttleState.msg`, `mfja_robot_control_config/scripts/room_315_visual_runtime_validation.py`, and `mfja_robot_control_config/scripts/room_315_task_execution.py`.

For every canonical identity `L1--L4`, `R1--R4`, the model/validator may report presence, side/block, bounding box, normalized/metric segment position, loaded state, and confidence. The enclosing observation records source image stamps, schema/checkpoint, acceptance/stabilization/readiness flags, rejection reasons, clamps, latency, and counters.

Consumers must apply all of the following rather than checking only accepted:

1. message and both source images are fresh;
2. schema and checkpoint hash exactly match the authorized values;
3. model/input/presence/fusion readiness is true;
4. the observation is accepted and not stale;
5. facts required by the requested task are valid and sufficiently confident; and
6. a newer observation follows any commanded action before effect success.

`stabilized` reports that the optional temporal filter was applied. It is normally false when `temporal_filter_enabled=false`, including on valid accepted observations, and the task gateway does not require it. If a promoted runtime enables temporal filtering, validate that filter's authorized contract rather than imposing `stabilized=true` on every deployment.

9.4 English TaskGoal Understanding

Source files: `mfja_robot_control_config/scripts/room_315_task_goal_cli.py`, `mfja_robot_control_config/scripts/room_315_task_goal_dialogue.py`, `mfja_robot_control_config/scripts/room_315_task_goal_parsers.py`, and `mfja_robot_control_config/config/room_315_vla/task_goal_understanding.yaml`.

The language layer combines a deterministic parser with an optional local semantic model. The deterministic path remains available without the large model. It produces a draft, asks targeted clarification for ambiguity, and requires an explicit yes/confirm step before publishing JSON to `/room_315/task_goal`.

Supported intent is deliberately narrow: one transport or inspection request, explicit/any/nearest identity where valid, loaded/empty/any payload qualification, and an exact slot or supported station target. Compound plans, swaps, "move all", load/unload operations, arbitrary coordinates, deadlines, and speed-optimization objectives are rejected rather than guessed.

The optional model requires the separately managed `llama-cpp-python==0.3.16` environment and a roughly 1.04 GiB GGUF file. `mfja_robot_control_config/scripts/setup_room315_intent_model.py` can download/install dependencies and may use `user-site` or `--break-system-packages`; prefer the documented isolated virtual environment and inspect its planned actions before running it.

9.5 PDDL Authorities and Translation

Source files: `mfja_robot_control_config/config/room_315_vla/pddl/domain_room315_runtime.pddl`, `mfja_robot_control_config/config/room_315_vla/pddl/domain_room315.pddl`, `mfja_robot_control_config/scripts/room_315_pddl_plan_translator.py`, and `mfja_robot_control_config/scripts/room_315_pddl_validation_gate.py`.

`domain_room315_runtime.pddl` is the live execution domain. `domain_room315.pddl` is the broader expert/data-generation domain. Do not replace the runtime file with the research domain merely because it contains more actions.

The current runtime domain defines these 21 actions:

```
prepare_switches open_stoppers
restore_normal_route move_shuttle_to_slot
prepare_topology_route move_shuttle_from_segment_to_slot
prepare_slot_topology_route move_shuttle_via_topology_to_slot
begin_route_clearance relocate_blocker_to_interior
stage_selected_to_interior finish_route_clearance
begin_segment_route_clearance relocate_segment_blocker_to_interior
stage_selected_segment_to_interior
finish_segment_route_clearance pause_route_clearance
stop_shuttle finish_task
finish_candidate_task inspect_state
```

The expert and runtime domains currently expose the same action names, while their predicates/preconditions remain separate versioned contracts. The checked runtime file is always the machine authority. The translator reduces the supported plan surface to deterministic primitives: `SET_SWITCHES`, `SET_STOPPERS`, `SHUTTLE_ON`, `STOP_NOW`, and `DONE`. The entire returned plan is parsed and validated before its first primitive is sent.

9.6 Closed-Loop Task Gateway

Source files: `mfja_robot_control_config/scripts/room_315_task_execution_node.py`, `mfja_robot_control_config/scripts/room_315_task_execution.py`, `mfja_robot_control_config/scripts/room_315_task_execution_config.py`, and `mfja_robot_control_config/config/room_315_vla/task_execution_runtime.yaml`.

`room_315_task_execution_node.py` owns the host singleton lock and combines the accepted visual observation, supervisor status, and both typed sensor feeds. It ignores privileged `segment`, `s`, and `s_ratio` fields in sensor readings; only active state, sensor name, and shuttle identity may anchor exact DZI occupancy.

For each confirmed goal the gateway:

1. verifies execution enablement, Gazebo-only policy, immutable task authorization, promotion manifest, schema, and checkpoint hash;
2. waits for fresh visual/supervisor/sensor inputs;

3. grounds identity and payload constraints; payload requests require five agreeing accepted frames within at most 15 observations;
4. generates a new problem and calls `/planner/get_plan`;
5. validates/translate the complete response;
6. executes only the first primitive through the supervisor;
7. waits for a newer visual/controller effect, then replans; and
8. succeeds only when the exact goal holds, or publishes `stop_all` and aborts on bounded failure.

An exact final slot requires both identity-bearing DZI occupancy and explicit controller `DISABLED`. Current defaults include 20 s planner timeout, 1.5 s visual/supervisor freshness, 1.0 s sensor freshness, 2.0 s observation wait, two confirming sensor frames, 0.2 m/s motion, 32 maximum steps, eight replans, three unknown-effect retries, 5 s supervisor timeout, 30 s ordinary effect timeout, and 60 s clearance timeout. Long-route arrival uses $1.25 \times$ estimated travel time plus 5 s margin.

9.7 Primitive Supervisor

Source files: `mfja_robot_control_config/scripts/room_315_vla_supervisor.py`, `mfja_robot_control_config/launch/room_315_vla_supervisor.launch.py`, and `mfja_robot_control_config/config/room_315_vla/vla_supervisor.yaml`.

`room_315_vla_supervisor.py` is deterministic. It consumes JSON primitives on `/room_315/vla/command`, reads typed rail/device state and camera availability, and publishes typed shuttle, switch, and stopper commands. It reports structured status on `/room_315/vla/status`.

Important invariants are:

- arbitrary learned action vectors are rejected;
- identities, sides, slots, switch/stopper states, speeds, readiness, and incompatible simultaneous actions are validated before publication;
- `stop_all` is the normal fail-safe primitive;
- an emergency stop closes stoppers and prevents continued actuation; clearing the Boolean emergency-stop input does not automatically restart previously moving shuttles; and
- bounded recovery is attempted at most twice by default, after which the supervisor enters emergency-stop/manual-recovery behavior.

The supervisor's legacy position-feedback subscriptions are not current state authority. Typed shuttle, switch, stopper, and `sensors/feedback` topics are the live feedback surface.

9.8 Dataset Lifecycle and Leakage Control

Source files: `mfja_robot_control_config/scripts/room_315_vla_dataset_recorder.py`, `mfja_robot_control_config/scripts/room_315_visual_state_dataset.py`, `mfja_robot_control_config/scripts/room_315_grouped_visual_splits.py`, and `mfja_robot_control_config/scripts/room_315_visual_dataset_audit.py`.

A recorded episode contains `images/<camera>/`, `data.jsonl`, `events.jsonl`, `episode.json`, and, after validation, `validation.json`. The supported lifecycle is summarized below.

Table 9.2: Dataset-to-candidate lifecycle and owning tools

Stage	Required transition/output	Source file(s)
Capture	Scenario manifest → guarded episode images/events/data	mfja_robot_control_config/scripts/room_315_vla_dataset_recorder.py; mfja_robot_control_config/scripts/room_315_visual_state_capture.py
Validate/label	Episode → validation record and permitted labels/export	mfja_robot_control_config/scripts/room_315_visual_state_dataset.py; mfja_robot_control_config/scripts/room_315_visual_label_exporter.py
Split/audit	Related captures remain grouped; leakage and role constraints are audited	mfja_robot_control_config/scripts/room_315_grouped_visual_splits.py; mfja_robot_control_config/scripts/room_315_visual_dataset_audit.py
Train/evaluate	Frozen training selection → sealed evaluation evidence	mfja_robot_control_config/scripts/room_315_visual_training_v4.py; mfja_robot_control_config/scripts/room_315_visual_v4_final_test.py
Package	Verified model/config/evidence → immutable candidate	mfja_robot_control_config/scripts/room_315_build_runtime_candidate_v4.py

Privileged identity tracks and controller coordinates may be used as label or evaluation oracle data only when the applicable contract permits them. They must remain separate from model inputs. Split tools group related captures so variants of the same scene do not cross train/validation/test boundaries.

The frozen V4 training contract uses a 50/50 legacy/hard-data mixture, a split-rail ResNet-18 model at 320×240 , 12 epochs, and seed 31520260811. The associated evidence records these frozen roles.

The tracked role/count authorities are `mfja_robot_control_config/config/room_315_vla/visual_state_training_v4.json`, `mfja_robot_control_config/config/room_315_vla/visual_state_final_test_v4.json`, and `mfja_robot_control_config/config/room_315_vla/visual_state_final_test_v4_coverage_extension.json`.

Table 9.3: Frozen V4 dataset roles, counts, and permitted use

Dataset role	Count	Use
Legacy training	1,528	Historical training component
Historical predecessor validation	256	Provenance/comparison; not reusable as a new final test

Dataset role	Count	Use
Consumed historical test	256	Already consumed; never tune against it again
Hard training	4,000	V4 hard-case training component
V4 validation	512	Model/epoch selection under frozen rules
Canary	256	Guarded pre-deployment check
Sealed final-test coverage	1,040	Frozen extension of the 1,024-scene base final-test contract

Treat those values as the current contract, not a target to reproduce by silently adding files. Any changed dataset requires a new version, immutable manifest, provenance, split audit, and qualification decision.

9.9 Promotion and Authorization

Source files: `mfja_robot_control_config/scripts/room_315_build_runtime_candidate_v4.py`, `mfja_robot_control_config/scripts/room_315_promote_runtime_v4.py`, `mfja_robot_control_config/scripts/room_315_authorize_runtime_v4_closed_loop.py`, `mfja_robot_control_config/scripts/room_315_visual_runtime_v4.py`, and `mfja_robot_control_config/scripts/room_315_task_execution_config.py`.

The intended chain is:

1. build an immutable candidate with dataset, config, code, and checkpoint hashes;
2. run validation and CPU/runtime compatibility checks;
3. deploy in shadow mode and record acceptance/fault evidence;
4. manually promote the exact candidate, producing a signed/hashed promotion manifest;
5. separately authorize closed-loop task execution for that exact manifest/checkpoint/code contract; and
6. point runtime config at those immutable files without editing the candidate in place.

Promotion permits inference; task authorization permits a specific Gazebo closed-loop configuration. They are intentionally separate. Rollback selects a previous complete authorized bundle and restores all matching hashes/config; never copy only an older checkpoint into a newer manifest.

Task authorization is verified only when `execution_enabled=true`; a disabled gateway requires no authorization artifact. For enabled execution, the authorization JSON, promotion manifest, manual decision, and every bound artifact must be regular files whose final path component is not a symbolic link and whose Unix mode has no write bits. Hash agreement does not replace these immutability checks.

Clean-clone limitation

The active checked-in V4 visual and task-execution YAML files reference absolute `/home/tiago/...` promotion and authorization artifacts that are not stored in Git. A clean clone can run simulation, rails, cameras, recording, and deterministic supervision, but active learned

inference and authorized execution require the complete external bundle to be restored or a new bundle to be qualified. Do not weaken hash checks to make startup succeed.

9.10 Changing the Learned/Planning Stack

The coupled change surface is implemented and tested across `mfja_robot_control_config/scripts/room_315_visual_model_v4.py`, `mfja_robot_control_config/scripts/room_315_visual_runtime_validation.py`, `mfja_robot_control_config/scripts/room_315_pddl_plan_translator.py`, `mfja_robot_control_config/scripts/room_315_task_execution.py`, and `mfja_robot_control_config/test/test_room315_task_execution.py`.

Safe change sequence

1. Version the contract; never edit a sealed dataset, checkpoint, manifest, authorization, or final-test result in place.
2. Keep controller/oracle data out of learned inputs and record the exact feature boundary in tests.
3. Update model, validator, typed observation schema, state builder, PDDL generator/domain, translator, executive, config, and fault tests as one reviewed dependency set.
4. Train/select only on allowed splits; preserve the final test until the decision rule is frozen.
5. Build a new immutable candidate, run shadow/acceptance/fault campaigns, promote manually, and issue a separate execution authorization.
6. Retain a complete prior bundle and rehearse rollback before enabling execution.

Part IV

Operations and Project Lifecycle

Chapter 10

Gazebo Operator Runbook

Chapter guide

Purpose. Provides repeatable Gazebo-only procedures for starting, checking, commanding, extending, and stopping the supported runtime profiles.

Recommended use. Proceed in dependency order: verify the overlay, prove the base floor, operate robots or rails, add deterministic supervision, add visual state, enable authorized task execution, and shut down cleanly. Stop at the first unmet readiness condition.

10.1 Scope and Safety Preconditions

This runbook applies only to the Gazebo simulation described by this repository. It does not authorize connection to physical rails, PLCs, robots, or grippers; the complete deployment boundary is defined in chapter 13. Follow the sections in dependency order. Do not begin an optional layer until the preceding layer's clock, services, topics, and authoritative feedback are healthy.

Completion requires feedback

A published command is a request, not proof of completion. Require a newer typed actual-state sample or a newer verified task effect before the next mutation. The direct rail commands in this chapter bypass the planner and deterministic supervisor and therefore require an explicitly controlled, empty-rail procedure.

10.2 Runbook Source Map

The following map identifies the implementation and interface sources behind each operational group. Commands later in this chapter are operator recipes; the listed files remain authoritative when behavior or interfaces change.

Table 10.1: Operator-runbook groups and their authoritative source files

Runbook group	Source file(s)
Terminal setup and pre-flight	mfja_3rd_floor_bringup/package.xml;mfja_3rd_floor_bringup/launch/room_315_floor_common.py.
Base headless smoke and Room 315 profile	mfja_3rd_floor_bringup/launch/room_315_only.launch.py;mfja_3rd_floor_bringup/launch/room_315_floor_common.py;mfja_robot_control_config/launch/room_315_dual_kinematic_shuttles.launch.py.
Full-floor profiles	mfja_3rd_floor_bringup/launch/full_floor.launch.py;mfja_3rd_floor_bringup/launch/room_315_floor_common.py;mfja_robot_control_config/launch/multi_robot_sim.launch.py;mfja_robot_control_config/config/robots.yaml.

Runbook group	Source file(s)
Isolated industrial robot	<code>mfja_3rd_floor_bringup/launch/single_industrial_robot.launch.py</code> ; <code>mfja_robot_control_config/launch/isolated_industrial_robot.launch.py</code> .
Robot joint and gripper operation	<code>mfja_robot_control_config/scripts/robot_joint_command.py</code> ; <code>mfja_robot_control_config/scripts/robot_gripper_command.py</code> ; <code>mfja_robot_control_config/config/gripper_command_defaults.yaml</code> .
Shuttle creation and life-cycle	<code>mfja_rail_interfaces/srv/AddShuttle.srv</code> ; <code>mfja_rail_interfaces/msg/ShuttleCommand.msg</code> ; <code>mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py</code> ; <code>mfja_robot_control_config/scripts/room_315_multi_shuttle.py</code> .
Switch operation	<code>mfja_rail_interfaces/msg/SwitchCommand.msg</code> ; <code>mfja_rail_interfaces/msg/SwitchState.msg</code> ; <code>mfja_robot_control_config/scripts/room_315_rail_devices.py</code> .
Stopper and sensor operation	<code>mfja_rail_interfaces/msg/StopperCommand.msg</code> ; <code>mfja_rail_interfaces/msg/StopperState.msg</code> ; <code>mfja_rail_interfaces/msg/SensorFeedback.msg</code> ; <code>mfja_robot_control_config/scripts/room_315_rail_devices.py</code> .
Camera bridge and deterministic supervisor	<code>mfja_3rd_floor_bringup/launch/room_315_floor_common.py</code> ; <code>mfja_robot_control_config/launch/room_315_vla_supervisor.launch.py</code> ; <code>mfja_robot_control_config/scripts/room_315_vla_supervisor.py</code> .
V4 visual runtime	<code>mfja_robot_control_config/launch/room_315_visual_state_runtime.launch.py</code> ; <code>mfja_robot_control_config/scripts/room_315_visual_state_inference_node.py</code> ; <code>mfja_robot_control_config/scripts/room_315_visual_runtime_v4.py</code> ; <code>mfja_robot_control_config/config/room_315_vla/visual_state_runtime.yaml</code> .
Planning and closed-loop task execution	<code>mfja_robot_control_config/launch/room_315_task_execution.launch.py</code> ; <code>mfja_robot_control_config/scripts/room_315_task_execution_node.py</code> ; <code>mfja_robot_control_config/scripts/room_315_task_execution.py</code> ; <code>mfja_robot_control_config/config/room_315_vla/task_execution_runtime.yaml</code> .
Controlled shutdown	<code>mfja_3rd_floor_bringup/launch/room_315_floor_common.py</code> ; <code>mfja_robot_control_config/scripts/room_315_vla_supervisor.py</code> ; <code>mfja_robot_control_config/scripts/room_315_task_execution_node.py</code> .

10.3 Terminal Setup and Preflight

Use the same workspace variables in every terminal. These commands do not build or change runtime state:

```
export MFJA_WS="${MFJA_WS:-$HOME/mfja_ws}"
export MFJA_REPO="${MFJA_REPO:-$MFJA_WS/src/mfja_3rd_floor_gz}"

cd "$MFJA_WS"
source /opt/ros/jazzy/setup.bash
source "$MFJA_WS/install/setup.bash"

git -C "$MFJA_REPO" status --short --branch
ros2 pkg prefix mfja_3rd_floor_bringup
ros2 pkg prefix mfja_robot_control_config
```

Before launching, stop any previous high-level floor with Ctrl-C, confirm the desired overlay, and decide whether startup may clear the configured obstacle cache. The high-level default is destructive for that one cache file; the runbook commands below explicitly preserve it with `room315_clear_vla_obstacle_pose_cache:=false`.

10.4 Base Headless Smoke

Terminal A:

```
ros2 launch mfja_3rd_floor_bringup room_315_only.launch.py \
  robots:=none \
  gui:=false \
  start_paused:=false \
  enable_room315_kinematic_shuttles:=true \
  room315_clear_vla_obstacle_pose_cache:=false \
  room315_right_shuttle_count:=1 \
  room315_left_shuttle_count:=0 \
  room315_shuttles_start_enabled:=false
```

After five seconds, Terminal B:

```
ros2 topic echo --once /clock
ros2 service list | grep '^/world/room_315_only/'
ros2 topic echo --once /room_315/rails/right/shuttles/state \
  mfja_rail_interfaces/msg/ShuttleState
ros2 topic echo --once /room_315/rails/right/sensors/feedback \
  mfja_rail_interfaces/msg/SensorFeedback
```

Expect `create`, `remove`, and `set_pose` world services plus a stopped right-shuttle state. `gui:=false` also omits the Gazebo client; camera rendering can still require working server-side EGL/graphics support.

10.5 Supported Simulation Profiles

10.5.1 Room 315, empty rails

```
ros2 launch mfja_3rd_floor_bringup room_315_only.launch.py \
  robots:=none gui:=true start_paused:=false \
  room315_clear_vla_obstacle_pose_cache:=false \
  room315_right_shuttle_count:=0 \
  room315_left_shuttle_count:=0
```


10.5.2 Lightweight full floor

```
ros2 launch mfja_3rd_floor_bringup full_floor.launch.py \
  robots:=none gui:=true start_paused:=false \
  room315_clear_vla_obstacle_pose_cache:=false \
  room315_right_shuttle_count:=0 \
  room315_left_shuttle_count:=0
```

The unmodified full-floor default is paused. This example explicitly starts running so simulation-time controllers advance.

10.5.3 Full floor with selected robots

```
ros2 launch mfja_3rd_floor_bringup full_floor.launch.py \
  robots:=kukal,staubli1,yaskawa_hc10_1,yaskawa_hc10dt_1,tiago1 \
  gui:=true start_paused:=false \
  room315_clear_vla_obstacle_pose_cache:=false
```

10.5.4 Isolated industrial robot

```
ros2 launch mfja_3rd_floor_bringup single_industrial_robot.launch.py \
  robot:=kuka gui:=true start_paused:=false
```

Valid short selectors are kuka, staubli, hc10, and hc10dt. Use this profile for fast joint/gripper changes.

10.6 Robot Joint and Gripper Operation

Discover the current installed profiles before commanding:

```
ros2 run mfja_robot_control_config robot_joint_command.py --list
ros2 run mfja_robot_control_config robot_gripper_command.py --list
```

Preview first:

```
ros2 run mfja_robot_control_config robot_joint_command.py \
  kuka --unit rad \
  --positions 0.6 -1.0 1.1 0.0 0.6 0.0 \
  --dry-run

ros2 run mfja_robot_control_config robot_gripper_command.py \
  kuka open --dry-run
```

Publish the verified command:

```
ros2 run mfja_robot_control_config robot_joint_command.py \
  kuka --unit rad \
  --positions 0.6 -1.0 1.1 0.0 0.6 0.0

ros2 run mfja_robot_control_config robot_gripper_command.py kuka open
ros2 run mfja_robot_control_config robot_gripper_command.py kuka close
ros2 run mfja_robot_control_config robot_gripper_command.py kuka open 75
ros2 run mfja_robot_control_config robot_gripper_command.py \
  hc10 --position 0.012
```

The selected robot must already exist and expose the expected subscriber. Check `/<robot>/joint_states`, command topic type, and trajectory progress. Industrial grippers only animate jaws; they do not attach payloads.

10.7 Direct Rail Operation

The following expert examples target the right rail. For the left rail, replace `right` and the entity/identity consistently. Begin with an empty-rail profile. Direct typed publication bypasses the planner and supervisor.

10.7.1 Add and inspect a stopped shuttle

```
ros2 service call /room_315/rails/right/shuttles/add \
  mfja_rail_interfaces/srv/AddShuttle \
  "{name: 'room315_right_shuttle_1', start_slot: '2', \
    speed: 0.2, start_enabled: false}"

ros2 topic echo --once /room_315/rails/right/shuttles/state \
  mfja_rail_interfaces/msg/ShuttleState
```

A successful add reply accepts the logical shuttle; visual spawn is asynchronous. Verify both state and Gazebo rendering when the visual matters.

10.7.2 Start, target, stop, reset, and remove

The mutation commands below are mutually exclusive examples, not one script to paste from top to bottom. Choose the applicable ON form, OFF scope, or lifecycle operation, then require a newer typed actual-state sample before issuing another mutation.

To start on the retained/default route without an exact target, run only:

```
ros2 topic pub --once /room_315/rails/right/shuttles/command \
  mfja_rail_interfaces/msg/ShuttleCommand \
  "{name: 'room315_right_shuttle_1', command: 'ON', speed: 0.2}"
```

Alternatively, to start toward one exact reachable target, run only:

```
ros2 topic pub --once /room_315/rails/right/shuttles/command \
  mfja_rail_interfaces/msg/ShuttleCommand \
  "{name: 'room315_right_shuttle_1', command: 'ON', \
    speed: 0.2, target_slot: '3'}"
```

After either ON form, require a newer MOVING or guarded WAITING typed state. To stop one identity, run:

```
ros2 topic pub --once /room_315/rails/right/shuttles/command \
  mfja_rail_interfaces/msg/ShuttleCommand \
  "{name: 'room315_right_shuttle_1', command: 'OFF'}"
```

Use the following as an alternative emergency scope, not immediately after the single-identity command:

```
ros2 topic pub --once /room_315/rails/right/shuttles/command \
  mfja_rail_interfaces/msg/ShuttleCommand \
  "{name: 'ALL', command: 'OFF'}"
```

Require a newer DISABLED state after either OFF scope. To reset a faulted or stopped identity to its allowed origin, first prove that origin unoccupied, then run:

```
ros2 topic pub --once /room_315/rails/right/shuttles/command \
mfja_rail_interfaces/msg/ShuttleCommand \
"{name: 'room315_right_shuttle_1', command: 'RESET'}"
```

Removal is a separate lifecycle operation. After confirming that removal is intended, run:

```
ros2 topic pub --once /room_315/rails/right/shuttles/command \
mfja_rail_interfaces/msg/ShuttleCommand \
"{name: 'room315_right_shuttle_1', command: 'REMOVE'}"
```

Nonzero speed is retained configuration, not proof of motion. A RESET or OFF shuttle still occupies the rail; verify typed state after every command.

10.7.3 Switches

Only change a coordinated switch pair after proving the rail is empty. Then:

```
ros2 topic pub --once /room_315/rails/right/switches/command \
mfja_rail_interfaces/msg/SwitchCommand \
"{switches: [{name: 'A1', state: 'INTERIOR'}, \
              {name: 'A2', state: 'INTERIOR'}]}"
```

Alternatively, and only after the same empty-rail proof, request EXTERIOR for every switch:

```
ros2 topic pub --once /room_315/rails/right/switches/command \
mfja_rail_interfaces/msg/SwitchCommand \
"{switches: [{name: 'ALL', state: 'EXTERIOR'}]}"
```

After the selected request, inspect the typed actual state:

```
ros2 topic echo --once /room_315/rails/right/switches/state \
mfja_rail_interfaces/msg/SwitchState
```

The pair request and ALL request are alternatives. A command contains requested state only: after either example, wait until a newer typed state reports every requested switch at its delayed actual value. Never issue the next mutation after a fixed sleep or from GUI animation alone.

10.7.4 Stoppers and sensors

To close stopper A1, run:

```
ros2 topic pub --once /room_315/rails/right/stoppers/command \
mfja_rail_interfaces/msg/StopperCommand \
"{stoppers: [{name: 'A1', state: '1'}]}"

ros2 topic echo --once /room_315/rails/right/stoppers/state \
mfja_rail_interfaces/msg/StopperState
```

Only after a newer typed state confirms the close, open the stopper when the procedure calls for clear passage:

```
ros2 topic pub --once /room_315/rails/right/stoppers/command \
mfja_rail_interfaces/msg/StopperCommand \
"{stoppers: [{name: 'A1', state: '0'}]}"

ros2 topic echo --once /room_315/rails/right/stoppers/state \
mfja_rail_interfaces/msg/StopperState
```

Inspect occupancy separately:

```
ros2 topic echo --once /room_315/rails/right/sensors/feedback \
mfja_rail_interfaces/msg/SensorFeedback
```

Close and open are separate mutations. Confirm a newer typed stopper state of 1 before relying on the guard, and confirm 0 before relying on clear passage; sensor feedback does not replace the stopper actual-state check.

10.8 Camera Bridge and Deterministic Supervisor

Start the integrated branch with the floor:

```
ros2 launch mfja_3rd_floor_bringup room_315_only.launch.py \
  robots:=none gui:=true start_paused:=false \
  enable_room315_kinematic_shuttles:=true \
  enable_room315_vla:=true \
  enable_room315_vla_camera_bridge:=true \
  enable_room315_vla_obstacles:=false \
  room315_clear_vla_obstacle_pose_cache:=false \
  room315_right_shuttle_count:=1 \
  room315_left_shuttle_count:=0 \
  room315_shuttles_start_enabled:=false
```

Inspect status and issue the ordinary supervised stop:

```
ros2 topic echo --once /room_315/vla/status std_msgs/msg/String
ros2 topic pub --once /room_315/vla/command std_msgs/msg/String \
  "{data: '{\"action\":\"stop_all\"}'}"
```

This does not require a learned model and does not start visual inference or task execution. If attaching to a floor that was launched without VLA, start `room_315_vla_supervisor.launch.py` directly. Do not do that when the integrated branch is already active; it would duplicate the supervisor and camera bridge.

10.9 V4 Visual Runtime

External artifact prerequisite

Do not run the checked-in visual YAML blindly: it names a host-specific external bundle. Obtain a complete independently approved promotion bundle, its JSON promotion manifest, the independently recorded manifest SHA-256, and the intended visual virtual environment. Never compute a new “approved” hash from an untrusted file merely to pass the check or edit a promoted bundle in place.

With the integrated VLA camera bridge already running, the safe shadow template is:

```
export ROOM315_SHADOW_V4_BUNDLE="$HOME/room315_artifacts/visual_v4_shadow"
ROOM315_SHADOW_V4_MANIFEST="$ROOM315_SHADOW_V4_BUNDLE"
ROOM315_SHADOW_V4_MANIFEST+=/runtime_promotion_manifest.json
export ROOM315_SHADOW_V4_MANIFEST
export ROOM315_SHADOW_V4_MANIFEST_SHA256='<approved-manifest-sha256>'

source "$HOME/.venvs/mfja-visual/bin/activate"
test -f "$ROOM315_SHADOW_V4_MANIFEST"
printf '%s %s\n' "$ROOM315_SHADOW_V4_MANIFEST_SHA256" \
  "$ROOM315_SHADOW_V4_MANIFEST" | sha256sum --check -

ros2 launch mfja_robot_control_config \
```

```
room_315_visual_state_runtime.launch.py \
use_sim_time:=true enable_camera_bridge:=false \
runtime_mode:=shadow \
v4_promotion_manifest:="$ROOM315_SHADOW_V4_MANIFEST" \
v4_promotion_manifest_sha256:="$ROOM315_SHADOW_V4_MANIFEST_SHA256" \
device:=auto dry_run_state_fusion:=true \
plansys2_update_enabled:=false
```

Inspect:

```
ros2 topic list | sort | rg '/room_315/visual_state/shadow_v4'
ros2 topic echo --once \
  /room_315/visual_state/shadow_v4/validation \
  mfja_rail_interfaces/msg/VisualStateObservation
ros2 topic echo \
  /room_315/visual_state/shadow_v4/diagnostics \
  diagnostic_msgs/msg/DiagnosticArray
```

Stop the shadow node before starting the independently approved active bundle. With the integrated camera bridge still running, use this exact first-readiness template:

```
export ROOM315_ACTIVE_V4_BUNDLE="$HOME/room315_artifacts/visual_v4_active"
ROOM315_ACTIVE_V4_MANIFEST="$ROOM315_ACTIVE_V4_BUNDLE"
ROOM315_ACTIVE_V4_MANIFEST+=/runtime_promotion_manifest.json
export ROOM315_ACTIVE_V4_MANIFEST
export ROOM315_ACTIVE_V4_MANIFEST_SHA256='<approved-manifest-sha256>'

source "$HOME/.venvs/mfja-visual/bin/activate"
test -f "$ROOM315_ACTIVE_V4_MANIFEST"
printf '%s %s\n' "$ROOM315_ACTIVE_V4_MANIFEST_SHA256" \
  "$ROOM315_ACTIVE_V4_MANIFEST" | sha256sum --check -

ros2 launch mfja_robot_control_config \
  room_315_visual_state_runtime.launch.py \
  use_sim_time:=true enable_camera_bridge:=false \
  runtime_mode:=active \
  v4_promotion_manifest:="$ROOM315_ACTIVE_V4_MANIFEST" \
  v4_promotion_manifest_sha256:="$ROOM315_ACTIVE_V4_MANIFEST_SHA256" \
  device:=auto dry_run_state_fusion:=true \
  plansys2_update_enabled:=false
```

In another sourced terminal, require an accepted active observation and inspect the visual-runtime diagnostic:

```
ros2 topic echo --once \
  /room_315/visual_state/observed_state \
  mfja_rail_interfaces/msg/VisualStateObservation
ros2 topic echo --once /diagnostics \
  diagnostic_msgs/msg/DiagnosticArray
```

The observation must have stage fused_observed_state, current source stamps, matching schema/checkpoint, accepted, model_ready, input_ready, presence_ready, and state_fusion_ready true, and stale false. stabilized=false is valid while the optional temporal filter is disabled.

Both supervisor and visual launches default to creating RGB bridges. When the integrated supervisor already owns them, visual runtime must use enable_camera_bridge:=false.

10.10 Planning and Closed-Loop Task Execution

Planning process/gateway startup with actuation disabled:

```
ros2 launch mfja_robot_control_config \
  room_315_task_execution.launch.py \
  use_sim_time:=true execution_enabled:=false \
  enable_plansys2:=true external_obstacles_disabled:=true

ros2 lifecycle get /planner
```

With `execution_enabled=false`, authorization verification is deliberately bypassed and no authorization or promotion artifact is required. The planner may be active while the disabled gateway still reports that live visual, supervisor, or sensor inputs are not ready; that is independent of artifact availability. For actuation, first create a host-local YAML and edit only its external paths/hashes:

```
export ROOM315_TASK_EXEC_CONFIG="$HOME/.config/mfja/task_execution_runtime.yaml"
mkdir -p "$(dirname "$ROOM315_TASK_EXEC_CONFIG")"
cp "$MFJA_REPO/mfja_robot_control_config/config/room_315_vla/task_execution_runtime.
  yaml" \
  "$ROOM315_TASK_EXEC_CONFIG"
${EDITOR:-nano} "$ROOM315_TASK_EXEC_CONFIG"
```

Set literal absolute paths for `task_execution_authorization_path` and `task_execution_promotion_manifest_path`, plus the independently approved `task_execution_authorization_sha256`. Verify that the visual schema and checkpoint hash match the active observation. Before enabling execution, the authorization JSON, promotion manifest, manual decision, and every artifact bound from that manifest must be a regular file, not a symbolic link at its final path component, and have all user/group/other write bits cleared. The host-local runtime YAML remains configuration; do not make writable copies of the immutable artifacts merely to change its paths.

Only after the floor, supervisor, active accepted vision, planner, and artifact checks are ready:

```
ros2 launch mfja_robot_control_config \
  room_315_task_execution.launch.py \
  use_sim_time:=true \
  runtime_config="$ROOM315_TASK_EXEC_CONFIG" \
  execution_enabled:=true enable_plansys2:=true \
  external_obstacles_disabled:=true

ros2 lifecycle get /planner
ros2 topic echo /diagnostics diagnostic_msgs/msg/DiagnosticArray
```

`execution_enabled:=true` causes supervised Gazebo actuation. The current gateway has no external-obstacle perception provider and supports only the paired policy `enable_room315_vla_obstacles:=false` with `external_obstacles_disabled:=true`. Setting the gateway value false fails closed with `external_obstacle_perception_not_configured`; task execution is unavailable while the floor obstacles are enabled.

TaskGoals should normally be published by the confirmatory English CLI. If a developer publishes JSON directly, first validate it with the same TaskGoal builder/validator; do not invent fields from memory.

10.11 Controlled Shutdown

Shutdown order

1. Stop the active goal or publish supervisor `stop_all`.
2. Require newer DISABLED feedback for every managed shuttle.
3. Stop the English CLI and separately launched task/visual processes with Ctrl-C.
4. Press Ctrl-C once in the high-level floor terminal and allow children to exit; preserve logs if a service/entity error occurred.
5. Confirm the relevant ROS nodes/Gazebo processes are gone before starting another floor. A leftover lock pathname alone is harmless.

Avoid `pkill` in normal operation. The payload batch/review utilities use broad `pkill-f` patterns and restart the ROS daemon; run them only on a dedicated simulation host after reviewing side effects. Their reset options can also delete a selected dataset directory. The intent-model setup downloads about 1.04 GiB and installs packages; keep it inside its isolated environment.

Source files: `mfja_robot_control_config/scripts/room_315_payload_case_batch_runner.py`, `mfja_robot_control_config/scripts/room_315_payload_case_review.py`, and `mfja_robot_control_config/scripts/setup_room315_intent_model.py`.

Chapter 11

Verification Strategy and Evidence

Chapter guide

Purpose. Maps each source change to the least expensive valid verification layers and defines what evidence is required before a result can support release.

Recommended use. Choose the scope from the change-to-test matrix, run static, focused, registered, and runtime checks in increasing cost, then record exact commands, counts, environment, exclusions, and retained evidence.

11.1 What Each Test Layer Proves

Table 11.1: Testing layers and their assurance boundaries

Layer	Proves	Does not prove
Static/source pytest	Parser, schema, topology, contracts, deterministic cores, source-tree integration	Installation, compiled plugin loading, live ROS/Gazebo timing
Ament/colcon tests	CMake-registered package tests in a built workspace	Nine unregistered source tests or unavailable optional-dependency cases
Copied-install audit	Package-share lookup, program modes, explicit install lists	Live runtime behavior
Headless smoke	Launch composition, graph/services, clock, basic rail state/sensors	GUI rendering, all robots, long routing, learned qualification
GUI/system scenario	Visual assets, robot/rail/camera alignment and operator flow	Certified physics or physical-machine safety
V4 unit/offline qualification	Dataset/model/schema/hash metrics under frozen contracts	Live timing and fault behavior
Shadow, acceptance, and fault campaign	Integrated visual/runtime failure handling for an exact candidate	Authorization for another artifact or real hardware

Source paths: `mfja_3rd_floor_description/CMakeLists.txt`, `mfja_3rd_floor_description/test/`, `mfja_robot_control_config/CMakeLists.txt`, `mfja_robot_control_config/test/`, and `mfja_robot_control_config/scripts/room_315_runtime_acceptance_campaign_v4.py`.

A passing lower layer never substitutes for a required upper layer. Conversely, a missing external model should be reported as an unmet prerequisite, not silently turned into either a pass or a code failure.

11.2 Change-to-Test Matrix

Table 11.2: Minimum validation by changed source area

Change area	Minimum focused validation before broader regression
Robot selector/registry/launch	Full-package static and multi-shuttle launch-argument tests; CLI <code>--list</code> ; <code>--show-args</code> ; headless spawn
World, model, URDF, plugin	Seven description tests as relevant; compiled install; Gazebo load, joint, and gripper smoke
Rail CSV/topology/names	Kinematic core, segment names/normalization, route topology, device and collision tests, switch authority
Device/slot/stopper	Rail-device loader, crossing/marker lifecycle, live sensor and delayed actual-state verification
Identity, payload, headway	Identity config, assets, and tracker; payload visual and occlusion; keepout, block reservation, multi-shuttle safety
Message/service	Interface rebuild plus all consumers, contract tests, <code>ros2interface show</code> , live type/service call
Launch argument/composition	Static launch tests, singleton tests, <code>--show-args</code> , default and override headless runs
Supervisor/safety	Supervisor, fleet safety, headway/block reservation, model-input boundary, emergency-stop/recovery tests
TaskGoal, PDDL, executive	Builder, semantic, validation, scenario gate, problem generator, planner/translator, authorization, and closed-loop tests
V4 visual, model, artifact	Contract, model, calibration, runtime, acceptance, shadow, fault, and promotion tests; CPU smoke with hashes
Dataset, split, evaluation	Recorder, extractor, validator, split, leakage, audit, and role tests; verify output manifest and no input mutation
Documentation and Source Index	Documentation-build consistency, LaTeX validation, links, English scan, and <code>git diff --check</code>

Validation source map: tests are under `mfja_3rd_floor_description/test/` and `mfja_robot_control_config/test/`; package test registration is in their respective `CMakeLists.txt`; documentation checks are defined in `docs/project_documentation/Makefile`.

11.3 Dependency and Static Checks

From the workspace root:

```
source /opt/ros/jazzy/setup.bash
source "$MFJA_WS/install/setup.bash"

rosdep check --from-paths \
  "$MFJA_REPO/mfja_3rd_floor_bringup" \
  "$MFJA_REPO/mfja_3rd_floor_description" \
  "$MFJA_REPO/mfja_rail_interfaces" \
  "$MFJA_REPO/mfja_robot_control_config" \
  --ignore-src --rosdistro jazzy \
  --skip-keys "python3-torch python3-torchvision"
```

```
git diff --check
python3 -m py_compile \
  "$MFJA_REPO"/mfja_robot_control_config/scripts/*.py \
  "$MFJA_REPO"/mfja_robot_control_config/launch/*.py \
  "$MFJA_REPO"/mfja_3rd_floor_bringup/launch/*.py
```

The `rosdep skip` is for the base path only. Record and verify the isolated Torch environment when running learned-runtime tests. A CMake cache created without Torch registers only 84 cases even if Torch is installed later; rerun configure with `--cmake-clean-cache`.

Dependency and build sources: the four package `package.xml` manifests and the four package `CMakeLists.txt` files, all at the repository package roots listed in Table 3.

11.4 Focused Source Tests

Start with the exact file related to a change:

```
cd "$MFJA_REPO"
python3 -m pytest -q \
  mfja_robot_control_config/test/test_room315_rail_devices.py

python3 -m pytest -q \
  mfja_3rd_floor_description/test/test_room315_vla_camera_model.py
```

Use an exact test node or `-k` expression while iterating. Some modules perform module-level optional-dependency skips. Without Torch, a broad direct invocation of the entire control test directory can terminate with `pytest` exit code 5 (no tests collected) after collection behavior; that is not a pass. Prefer explicit families/files and record counts.

11.5 Registered Package Tests

After a fresh build and overlay source:

```
cd "$MFJA_WS"
colcon test --packages-select \
  mfja_3rd_floor_description \
  mfja_robot_control_config \
  --event-handlers console_cohesion+
colcon test-result --verbose
```

Do not report only the command's shell exit status. Capture passed, failed, errored, skipped, and not-run counts, the configured optional dependencies, and the Git commit. Remove/rebuild stale test metadata if a result unexpectedly references a deleted test.

11.6 Runtime Smoke Protocol

Launch the base headless procedure from chapter 10, wait at least five seconds, then verify:

```
ros2 topic echo --once /clock
ros2 service list | rg '^/world/room_315_only/(set_pose|create|remove)$'
ros2 topic echo --once \
  /room_315/rails/right/shuttles/state \
  mfja_rail_interfaces/msg/ShuttleState
ros2 topic echo --once \
  /room_315/rails/right/sensors/feedback \
  mfja_rail_interfaces/msg/SensorFeedback
ros2 topic info --verbose /mfja/conveyor/switch_states
```

Confirm the known-empty heartbeat when count is zero or the expected identity when count is one. For motion tests, publish ON, observe MOVING/WAITING and changing arc length, then publish OFF and require a newer DISABLED message. Preserve logs for any pose-service or entity divergence.

Smoke-protocol sources: `mfja_3rd_floor_bringup/launch/room_315_only.launch.py`, `mfja_3rd_floor_bringup/launch/room_315_floor_common.py`, `mfja_robot_control_config/launch/multi_robot_sim.launch.py`, and `mfja_robot_control_config/launch/room_315_dual_kinematic_shuttles.launch.py`.

11.7 Current Test Inventory and Registration Gaps

The description package contains 7 test modules, all registered by CMake. The control package contains 98 `test_*.py` modules. Its CMake registers 84 without Torch and 89 when Torch is importable at configure time; five are Torch-gated.

These nine control tests are source-only and therefore absent from ordinary `colcon test`:

- `mfja_robot_control_config/test/test_room315_floor_runtime_singleton.py`;
- `mfja_robot_control_config/test/test_room315_pddl_scenario_validation_gate.py`;
- `mfja_robot_control_config/test/test_room315_rail_segment_names.py`;
- `mfja_robot_control_config/test/test_room315_route_topology.py`;
- `mfja_robot_control_config/test/test_room315_runtime_contracts.py`;
- `mfja_robot_control_config/test/test_room315_switch_feedback_authority.py`;
- `mfja_robot_control_config/test/test_room315_task_execution_singleton.py`;
- `mfja_robot_control_config/test/test_room315_visual_v4_final_test_coverage_compat.py`; and
- `mfja_robot_control_config/test/test_room315_visual_v4_final_test_coverage_extension.py`.

Register them when they are deterministic and suitable for the ament environment, or retain a written reason and execute them explicitly in the release procedure. The Repository Source Index in appendix D classifies every test by family.

Registration sources: `mfja_3rd_floor_description/CMakeLists.txt` and `mfja_robot_control_config/CMakeLists.txt`. The complete source inventory is taken from the two package `test/` directories above.

11.8 Evidence Recorded for This Manual

During the repository audit at `aadf967b1188`, `rosdepcheck` passed on the audited host for the declared base dependency surface. A selected 148-test source run completed with zero failures in 5.06 s, and ten selected V4 coverage tests completed with zero failures in 7.91 s. These were deliberately selected scopes, not a claim that every package, Torch, ROS-integration, artifact-dependent, or live Gazebo test passed.

The exact source-smoke command, run from the repository root, was:

```
PYTHONDONTWRITEBYTECODE=1 python3 -m pytest -q -p no:cacheprovider \
  mfja_3rd_floor_description/test \
  mfja_robot_control_config/test/test_room315_full_package_static.py \
  mfja_robot_control_config/test/test_room315_multi_shuttle_launch_args.py \
  mfja_robot_control_config/test/test_room315_floor_runtime_singleton.py \
  mfja_robot_control_config/test/test_room315_rail_devices.py \
  mfja_robot_control_config/test/test_room315_rail_segment_names.py \
  mfja_robot_control_config/test/test_room315_route_topology.py \
  mfja_robot_control_config/test/test_room315_switch_feedback_authority.py \
  mfja_robot_control_config/test/test_room315_runtime_contracts.py \
  mfja_robot_control_config/test/test_room315_task_execution_singleton.py \
  mfja_robot_control_config/test/test_room315_pddl_scenario_validation_gate.py
```

The exact selected V4 coverage command was:

```
PYTHONDONTWRITEBYTECODE=1 python3 -m pytest -q -p no:cacheprovider \
  mfja_robot_control_config/test/test_room315_visual_v4_final_test_coverage_compat.py \
  mfja_robot_control_config/test/test_room315_visual_v4_final_test_coverage_extension.py
```

No portable, repository-controlled log was retained for these two runs. Consequently, the commands, counts, and timings are audit context rather than release evidence. A release rerun must save its own pytest output and environment record under the release evidence directory.

A stored older colcon result reports 1,748 tests, zero failures, and 11 skips, but predates the documented commit and is historical evidence only. Do not use it as current release proof. It is an aggregate of XML files under `/home/tiago/mfja_3rd_floor_ros2_ws/build/*/test_results/`, not a sealed single-commit result bundle. Re-run the appropriate suite in the target environment and preserve the resulting `log/test_<timestamp>/` directory with the commit and dependency record.

11.9 CI Gap and Recommended Baseline

No checked-in continuous-integration workflow, `pyproject.toml`, `pytest.ini`, or lint configuration currently enforces the test surface. The minimum future CI should contain:

1. a clean Ubuntu 24.04/ROS Jazzy base dependency and four-package build;
2. source tests including the nine unregistered files;
3. ament tests with and without the optional Torch environment recorded;
4. copied-install and installed-surface audits;
5. headless Room 315 smoke with topic/service assertions;
6. Repository Source Index and documentation validation with `makecheck`; and
7. separately gated artifact-dependent V4 jobs that verify immutable bundle hashes before running.

Repository basis: the repository root (absence of a checked-in CI workflow and the named Python test configuration files), the four package `CMakeLists.txt` files, and `docs/project_documentation/Makefile`.

11.10 Test Result Record

Every handover/release result should state:

- commit, branch, date, supported OS/ROS/Gazebo, and workspace overlay;
- exact command and selected test files/packages;
- optional dependencies and external artifact identifiers/hashes;
- pass/fail/error/skip/uncollected counts and elapsed time;
- whether GUI/live Gazebo, copied install, and rollback were exercised;
- log/result locations and any known exclusions.

Never summarize “tests pass” when only a cached, selected, skipped, or historical result exists.

Chapter 12

Maintenance, Troubleshooting, and Extension

Chapter guide

Purpose. Defines the complete change cycle, package impact rules, extension recipes, diagnosis workflow, data policy, and rollback discipline.

Recommended use. Start with the standard change cycle, select the matching extension recipe, diagnose with read-only probes before mutation, and finish with verified documentation, evidence, and rollback.

12.1 Standard Change Cycle

One complete maintenance loop

1. Record branch, commit, worktree state, environment, and the command or observation that demonstrates the issue.
2. Classify the owner: interface, description/plugin, control/config, or bringup policy. Classify the data as product source, local output, external, or immutable.
3. Find all consumers with `rg`; include filenames, parameter keys, message fields, topic names, PDDL symbols, hashes, and left/right aliases.
4. Make the smallest coherent source change. Never repair `install/` or mutate a promoted/frozen artifact.
5. Run syntax/schema and focused source tests, then inspect the diff.
6. Build the affected package dependency closure and source the new overlay.
7. Verify the installed surface, registered tests, and relevant headless/ GUI/runtime scenario.
8. Update public documentation, this manual, and the Repository Source Index.
9. Record exact evidence, exclusions, rollback, and any migration action.

Before editing, treat all existing worktree changes as maintainer-owned. Do not discard or reformat unrelated work. Use version-control review to separate the intended patch from local build/capture output.

12.2 Change Impact by Package

Table 12.1: Package-level change impact and compatibility risk

Owner changed	Rebuild/review downstream	Typical compatibility risk
<code>mfja_rail_interfaces</code>	Both control and bringup dependency closure	Generated bindings, serializers, recordings, field semantics
<code>mfja_3rd_floor_description</code>	Description; control/bringup when path/plugin contract changes	SDF ABI, world service name, joint/-topic alignment, asset URI/license

Owner changed	Rebuild/review downstream	Typical compatibility risk
mfja_robot_control_config	Control plus bringup when wrapper-visible	Install list, parameter precedence, runtime contracts, research artifacts
mfja_3rd_floor_bringup	Bringup	Public arguments, profile defaults, singleton/startup order

Governing paths: each package’s root `package.xml` and `CMakeLists.txt`, plus dependency declarations in the downstream package manifests. Exact package roots are listed in Table 3.

This matrix identifies downstream ownership and compatibility risk; it does not replace the canonical change-to-test matrix in Table 11.2.

Shared Python foundations deserve broader regression. The dependency hot-spot table in the Repository Source Index (appendix D) identifies modules with many in-repository importers. In particular, treat rail defaults/fleet, visual dataset/contracts, runtime contracts, PDDL scenario generation, and task execution as cross-cutting.

12.3 Extension Recipes

Table 12.2 is the source registry for every recipe in this section. It keeps paths in one place so the procedural text does not repeat them inconsistently.

Table 12.2: Extension recipe source-path registry

Change type	Authoritative source paths
Python tool/node	mfja_robot_control_config/scripts/;mfja_robot_control_config/CMakeLists.txt;mfja_robot_control_config/package.xml;mfja_robot_control_config/test/
Launch surface	mfja_3rd_floor_bringup/launch/;mfja_robot_control_config/launch/;mfja_3rd_floor_bringup/CMakeLists.txt;mfja_robot_control_config/CMakeLists.txt
Installed configuration	mfja_robot_control_config/config/;mfja_robot_control_config/CMakeLists.txt
ROS interface	mfja_rail_interfaces/msg/;mfja_rail_interfaces/srv/;mfja_rail_interfaces/CMakeLists.txt; consumer code under mfja_robot_control_config/
Gazebo model/-world/robot	mfja_3rd_floor_description/models/;mfja_3rd_floor_description/worlds/;mfja_3rd_floor_description/urdf/;mfja_3rd_floor_description/src/;mfja_3rd_floor_description/THIRD_PARTY.md
Rail geometry/routing/devices	mfja_robot_control_config/config/room_315_kinematics/;mfja_robot_control_config/scripts/room_315_kinematic_shuttle.py;mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py

Change type	Authoritative source paths
TaskGoal, PDDL, and execution	<code>mfja_robot_control_config/config/room_315_vla/pddl/;mfja_robot_control_config/scripts/room_315_task_goal*.py;mfja_robot_control_config/scripts/room_315_pddl*.py;mfja_robot_control_config/scripts/room_315_task_execution*.py</code>
Dataset, model, and deployment	<code>mfja_robot_control_config/config/room_315_visual_state/;mfja_robot_control_config/config/room_315_vla/;relevant mfja_robot_control_config/scripts/room_315_vla*.py;mfja_robot_control_config/scripts/room_315_build_runtime_candidate_v4.py;mfja_robot_control_config/scripts/room_315_promote_runtime_v4.py;mfja_robot_control_config/scripts/room_315_authorize_runtime_v4_closed_loop.py</code>

12.3.1 Add a Python tool or ROS node

1. Put the file under `mfja_robot_control_config/scripts`, with a Python 3 shebang, concise module docstring, validated inputs, meaningful exit status, and safe `--help/--dry-run` where it mutates.
2. Separate ROS-free logic from the node adapter when practical.
3. Add it to the explicit CMake `install(PROGRAMS)` list only when it should be an installed/public program; set the source executable bit.
4. Declare required package/system dependencies; optional research imports must fail with an actionable prerequisite message.
5. Add focused tests and register suitable deterministic tests in CMake.
6. Rebuild and verify `ros2pkgexecutables`, copied install, help, failure cases, and output/side effects.

Do not flatten another large multi-purpose module into the already broad script directory without a clear family/contract. Its Repository Source Index category and public CLI status must remain understandable.

12.3.2 Add a launch file or argument

Define units, legal choices, default, and side effects. Trace it through every wrapper/include; test default, explicit override, false/zero/empty values, and parameter type. Use `--show-args` after rebuild. If the argument deletes a file, downloads data, starts actuation, or alters an authorization boundary, state that before the operator command.

When both the VLA supervisor and visual runtime are composed, assign exactly one camera bridge. When overriding VLA command/status topics, change gateway topics to match. Keep obstacle flags semantically consistent across floor and gateway.

12.3.3 Add an installed config

Adding a file to Git does not guarantee CMake installs it. Add the appropriate install rule, validate with its owning loader, rebuild, then inspect `ros2pkgprefix--sharemfja_robot_control_config`. Test a normal copied install as well as `--symlink-install`. Do not install a whole directory when ignored/local files could leak into the artifact.

12.3.4 Add or change a ROS interface

Define unit, authority, legal/empty/stale semantics, and compatibility policy. Update every consumer and recorded representation. Rebuild the interface first, then dependent packages; verify generated definitions with `ros2interfaceshow`. String-valued modes/actions should be centralized and validated; for a future breaking version, consider message constants or enums rather than another unrestricted string.

12.3.5 Add a Gazebo model, world asset, or robot

Preserve upstream license/provenance and exact license text. Validate model URI, mesh scale/axis, pose, material, rendering cost, and collision intent. Add `model.config`; include the model from the appropriate world; update the Repository Source Index. For a robot, align SDF/URDF joint names, bridge topics, plugin limits, registry aliases, TF prefix, state feedback, and commands.

Keep visual-only assets explicitly collision-free. Do not silently add heavy mesh collisions to industrial robots; that changes performance and behavior and needs a deliberate collision/control validation plan.

12.3.6 Change rail geometry, routing, or devices

Use the mapping in chapter 5. Preserve CSV originals and migrate explicitly; compare length, endpoint, tangent, route cycles, guard failure, and all device ratios. Test both sides and left public/internal conversion. Verify Gazebo world alignment with markers and representative shuttles, including headway, stopper crossing, target clamp, occupied reset, and pose-service fault.

12.3.7 Change task semantics or PDDL

Treat this as one dependency chain:

```
TaskGoal schema/validation -> grounding -> problem generator
-> expert and runtime domains -> PlanSys2 response
-> full-plan translator/validator -> supervisor primitive
-> fresh effect verification -> tests/documentation
```

Unsupported compound meaning must still clarify or reject. Add invalid and adversarial cases, not only a new happy path. Preserve first-primitive execution and replan-after-effect behavior.

12.3.8 Change datasets, models, or deployment policy

Create new versioned directories/manifests. Never overwrite frozen roles, sealed final tests, checkpoint sidecars, promotion manifests, authorizations, or evidence. Run split/leakage audit, record seeds/tool commit, qualify on CPU and target device, shadow, acceptance, and fault campaigns, then perform manual promotion and separate task authorization. Retain and test the complete prior bundle for rollback.

12.4 Troubleshooting by Symptom

Use read-only observation first. The source groups below are ordered by startup dependency: floor, robots, rails, camera/visual state, planning, then installation and retained data.

Table 12.3: Troubleshooting source-group registry

Key	Subsystem	Authoritative source paths
D1	Floor launch/world services	mfja_3rd_floor_bringup/launch/room_315_floor_common.py;mfja_robot_control_config/launch/multi_robot_sim.launch.py;mfja_robot_control_config/launch/room_315_launch_utils.py
D2	Robots and plugins	mfja_robot_control_config/config/robots.yaml;mfja_robot_control_config/launch/multi_robot_sim.launch.py;mfja_3rd_floor_description/src/
D3	Rail runtime	mfja_robot_control_config/launch/room_315_dual_kinematic_shuttles.launch.py;mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py;mfja_robot_control_config/config/room_315_kinematics/
D4	Camera and visual state	mfja_robot_control_config/launch/room_315_vla_supervisor.launch.py;mfja_robot_control_config/scripts/room_315_visual_runtime_v4.py;mfja_robot_control_config/config/room_315_vla/visual_state_runtime.yaml
D5	Planning and task gateway	mfja_robot_control_config/launch/room_315_task_execution.launch.py;mfja_robot_control_config/scripts/room_315_task_execution_node.py;mfja_robot_control_config/config/room_315_vla/task_execution_runtime.yaml
D6	Install, tests, and retained data	mfja_3rd_floor_bringup/CMakeLists.txt;mfja_3rd_floor_description/CMakeLists.txt;mfja_rail_interfaces/CMakeLists.txt;mfja_robot_control_config/CMakeLists.txt;mfja_robot_control_config/test/;mfja_robot_control_config/scripts/room_315_vla_dataset_recorder.py;docs/project_documentation/Makefile

Table 12.4: Troubleshooting symptoms, checks, actions, and source groups

Symptom	Check	Likely cause/action	Source
Second floor launch refuses	Owning process and lock; ROS node list	Supported singleton is active. Stop it cleanly; do not delete/bypass a live lock	D1
No /clock	Gazebo server output, bridge process, sourced domain	Server/bridge failed, wrong overlay/domain, or delayed startup	D1

Symptom	Check	Likely cause/action	Source
Clock present but motion frozen	Clock stamp changing; Gazebo pause state	Full floor starts paused; unpause or relaunch running	D1
World services absent	Internal world name and /world/... list	Filename/entity mismatch, bridge delay/failure, wrong world	D1
Robot absent	Registry selector, create response, Gazebo entity list	Unknown/disabled selector, bad model URI, spawn failure	D2
Joint command has no effect	Exact namespace/type/-subscriber, simulation time, limits	Wrong selector/overlay, plugin not loaded, paused world, invalid target	D2
Rail topic absent	Four-second delay, side enable/count, rail-node logs	High-level rail disabled, node validation failed, stale overlay	D3
Shuttle is WAITING	stopper actual state, headway, route guard	Expected guard, not an OFF acknowledgement; clear safely or command OFF	D3
Shuttle is FALLING	current segment and actual switches; network successor	Missing/mismatched guarded transition; stop, clear occupancy, repair/reset deliberately	D3
Logical state moves, visual does not	Set-pose service/client diagnostics and entity name	Service timeout/failure or missing entity; preserve logs, reconcile/restart	D3
Visual orphan after REMOVE	Gazebo entity and delete response	Logical removal preceded failed asynchronous delete; restart/explicit expert cleanup	D3
No camera image	bridge count, Gazebo sensors/rendering/EGL, topic type	Duplicate/missing bridge, renderer failure, wrong world/topic	D4
Visual runtime rejects startup	Manifest path/hash/schema/deployment mode/environment	Missing/incomplete/wrong external bundle; do not weaken validation	D4
Observations stay unready	Both images, both rail heartbeats, freshness, supervisor status, diagnostics	Paused clock, missing known-empty heartbeat, stale sync, safety gate	D4
Planner inactive	ros2lifecyleget/planner, launch logs	Delayed configure/activation, missing PlanSys2 dependency, disabled bundled planner	D5
Gateway cannot plan	GetPlan service, runtime PDDL, fresh observation/status	Planner absent, invalid problem/goal, unready authorized inputs	D5
Execution remains disabled	Config, exact authorization/promotion/checkpoint hashes	Intended fail-closed behavior; restore/qualify exact bundle	D5

Symptom	Check	Likely cause/action	Source
Executable/config missing after build	CMake install list, source mode, package share, overlay prefix	Source-only file, non-executable symlink, copied-install path bug, stale overlay	D6
Test count unexpectedly small	CMake cache/Torch import, registered list, pytest exit code	Optional test registration or source-only tests; reconfigure and record scope	D6
Disk usage grows	Workspace/repo log/, datasets, external bundles	Preserve needed evidence, then apply project retention policy; never delete unknown user data	D6

Useful read-only probes include:

```

ros2 node list | sort
ros2 topic list -t | sort
ros2 service list -t | sort
ros2 topic info --verbose <topic>
ros2 param dump <node>
ros2 lifecycle get /planner
ros2 doctor --report
gz service --list

```

Compare types and QoS at both ends before adding sleeps or changing reliability. Use timestamps to distinguish stale feedback from an incorrect value.

12.5 Build, Temporary, and User Data

Table 12.5: Data classes, locations, and maintenance policy

Class	Examples	Maintenance action
Product source	package code/config/assets/-docs	Review, test, version
Build output	build/, install/, log/, Python/pytest caches	Recreate; never hand-edit or treat as only copy
Runtime temporary	bridge YAML, patched SDF, filtered world under /tmp	Inspect during diagnosis; process-owned; may remain after shutdown
User captures	~/ .ros/... datasets and reports	Back up/retain by experiment policy; do not delete implicitly
External runtime	checkpoints, GGUF, promotion/authorization bundles	Verify hashes/provenance; immutable after promotion
Frozen research	split/final-test/evidence manifests	Supersede with new version, never rewrite
Third-party asset	CAD/meshes/model derivatives	Preserve exact terms, attribution, originals

Classification sources: package source roots listed in Table 3, temporary-path generation in `mfja_robot_control_config/launch/multi_robot_sim.launch.py`, dataset output handling

in `mfja_robot_control_config/scripts/room_315_vla_dataset_recorder.py`, and third-party provenance in `mfja_3rd_floor_description/THIRD_PARTY.md`.

Large local logs currently exist both inside this checkout and at workspace level; neither is a source tree. Establish a documented retention policy before cleanup, because failed-run logs and research evidence may be needed.

12.6 Documentation Maintenance

Keep detailed truth in one chapter and cross-reference it. Update this manual when any package, executable, launch argument/default, public topic/service, message field, config schema, authority rule, artifact gate, operator command, or safety boundary changes.

From `docs/project_documentation/`:

```
make check
```

The Repository Source Index covers Git-tracked source, so untracked experiments do not silently become documented product. Inspect the PDF table of contents, List of Tables, row separators, command wrapping, cross-references, and source classification. The documentation build and validation rules are defined in `docs/project_documentation/Makefile`.

12.7 Release and Rollback

For code/config rollback, apply a reviewed version-control revert to source, rebuild the same dependency closure, source the intended overlay, and rerun the failure-reproducing test plus smoke checks. Never roll back by editing an installed file.

For learned runtime rollback, select the previous complete immutable promoted bundle and its matching task authorization, update the host-local config, verify all expected hashes, run shadow/readiness, and only then re-enable. A checkpoint alone is not a rollback unit.

Preserve failing logs, manifests, input identity, commands, and results until the cause and corrective action are reviewed.

Chapter 13

Safety Boundary, Risk Register, and Handover

Chapter guide

Purpose. Consolidates the non-negotiable deployment boundary, prioritized unresolved risks, redistribution constraints, and evidence required for handover.

Recommended use. Review the safety boundary before any deployment decision, assign owners to the risk register, then use the acceptance checklist and completion record for a traceable sender–receiver handover.

13.1 Authoritative Source Registry

The source keys used below keep safety and maintenance claims traceable without repeating long paths throughout the prose. Each key identifies the primary tracked evidence for the associated claim; the Repository Source Index in Appendix D provides the package-oriented source index.

Table 13.1: Authoritative source registry for safety boundaries and known gaps

Key	Subject	Authoritative tracked source files
S1	Deployment boundary	README.md;mfja_robot_control_config/config/room_315_vla/task_execution_runtime.yaml
S2	Rail motion, topology, devices, and headway	mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py;mfja_robot_control_config/launch/room_315_dual_kinematic_shuttles.launch.py;mfja_robot_control_config/config/room_315_kinematics/rail_network_left.yaml;mfja_robot_control_config/config/room_315_kinematics/rail_network_right.yaml;mfja_robot_control_config/config/room_315_kinematics/rail_devices_left.yaml;mfja_robot_control_config/config/room_315_kinematics/rail_devices_right.yaml
S3	Robot collision geometry boundary	mfja_3rd_floor_description/test/test_industrial_robot_collision_free.py;mfja_3rd_floor_description/models/kuka_kr6r900sixx/model.sdf;mfja_3rd_floor_description/models/staubli_tx2_60l/model.sdf;mfja_3rd_floor_description/models/yaskawa_hc10/model.sdf;mfja_3rd_floor_description/models/yaskawa_hc10dt/model.sdf
S4	Payload, device, and obstacle presentation	mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py;mfja_3rd_floor_description/models/room315_vla_removable_obstacle_marker/model.sdf;mfja_3rd_floor_description/models/room315_vla_payload_medium_box/model.sdf;mfja_3rd_floor_description/models/room315_vla_overhead_devices/model.sdf
S5	Learned-state validation and supervised execution	mfja_robot_control_config/scripts/room_315_visual_state_inference_node.py;mfja_robot_control_config/scripts/room_315_task_execution.py;mfja_robot_control_config/scripts/room_315_vla_supervisor.py;mfja_robot_control_config/config/room_315_vla/visual_state_runtime.yaml;mfja_robot_control_config/config/room_315_vla/task_execution_runtime.yaml
S6	Build, installation, and test registration	mfja_3rd_floor_bringup/CMakeLists.txt;mfja_3rd_floor_description/CMakeLists.txt;mfja_rail_interfaces/CMakeLists.txt;mfja_robot_control_config/CMakeLists.txt;mfja_robot_control_config/config/room_315_vla/pddl/
S7	Asset provenance and declared package licences	mfja_3rd_floor_description/THIRD_PARTY.md;mfja_3rd_floor_description/package.xml;mfja_3rd_floor_bringup/package.xml;mfja_rail_interfaces/package.xml;mfja_robot_control_config/package.xml

Key	Subject	Authoritative tracked source files
S8	Batch/review process and temporary-file behavior	mfja_robot_control_config/scripts/room_315_payload_case_batch_runner.py;mfja_robot_control_config/scripts/room_315_payload_case_review.py;mfja_robot_control_config/scripts/room_315_task_execution_node.py;mfja_robot_control_config/scripts/room_315_v4_closed_loop_fault_campaign.py
S9	Host-bound active artifact locations	mfja_robot_control_config/config/room_315_vla/visual_state_runtime.yaml;mfja_robot_control_config/config/room_315_vla/task_execution_runtime.yaml;mfja_robot_control_config/config/room_315_vla/visual_state_training_v4.json;mfja_robot_control_config/config/room_315_vla/visual_state_final_test_v4.json
S10	Copied-install path handling	mfja_robot_control_config/scripts/room_315_task_goal_semantic.py;mfja_robot_control_config/scripts/room_315_task_goal_benchmark.py;mfja_robot_control_config/scripts/room_315_task_execution_config.py;mfja_robot_control_config/scripts/room_315_visual_observed_state_provider.py;mfja_robot_control_config/scripts/room_315_vla_train_v4.py
S11	Active and historical visual-runtime families	mfja_robot_control_config/CMakeLists.txt;mfja_robot_control_config/config/room_315_vla/visual_state_experiment_a_dataset_v3.yaml;mfja_robot_control_config/config/room_315_vla/visual_state_experiment_a_dataset_v3r1.yaml;mfja_robot_control_config/config/room_315_vla/visual_state_runtime.yaml

13.2 Non-Negotiable Deployment Boundary

Simulation and research only

This repository is not a safety-certified controller and is not authorized for physical machinery. Its current task authorization is Gazebo-only. Do not connect its command outputs to a real rail, PLC, robot, or gripper without a new approved hardware architecture, independent risk assessment, trusted sensors, hard real-time/safety functions, physical commissioning, and organizational authorization.

The limitation is architectural, not a disclaimer that can be removed after a successful demo:

- shuttle propulsion uses pose resets, not motor/contact dynamics;
- headway is a software center-distance check, not certified block protection;
- KUKA, Stäubli, HC10, and HC10DT models intentionally lack collision geometry;
- industrial grippers articulate visually and do not grasp/attach physics payloads;
- removable obstacle markers and several payload/device assets are visual-only;
- asynchronous Gazebo service failure can leave logical/visual divergence;
- learned position/load facts are probabilistic and are accepted only through a specific guarded V4 contract; and
- direct typed ROS topics can bypass high-level planning/supervision.

Source keys: S1–S5 in Table 13.1.

13.3 Known-Gap Register

Table 13.2: Prioritized known-gap register and current containment

Priority	Gap	Required direction	Current containment	Source
P0	Active visual/task YAML uses external <code>/home/tiago/...</code> artifacts	Host-local deployment config or portable bundle locator; preserve immutable hashes	Fails closed when files/hashes are absent	S9
P0	Physical use is not validated/certified	Separate hardware/safety project and authorization	Gazebo-only policy and documentation	S1
P1	Copied-install path assumptions in several Python tools	Centralize package-share lookup with <code>ament_index_python</code> ; add copied-install CI	Symlink development install masks issue	S10
P1	No checked-in CI; nine source tests unregistered; five Torch-gated	Add clean base/optional pipelines and explicit complete-source job	Documented commands and Repository Source Index	S6
P1	Mixed third-party asset licenses while manifests say Apache-2.0	Vendor exact upstream license texts in <code>LICENSES/</code> ; review redistribution	<code>mfja_3rd_floor_description/THIRD_PARTY.md</code> records provenance	S7
P1	PDDL directory install can include ignored local HTML	Install an explicit tracked-file set	Clean clone omits the local file	S6
P2	Only 49 source scripts executable versus 108 CMake-listed	Separate support libraries/public programs and normalize modes/install tests	Repository Source Index records both dimensions	S6
P2	V3/V3R1/V4 and compatibility tools share a flat package	Archive/name/version active versus historical surfaces more strongly	V4-only active validation and this classification	S11
P2	Large multi-responsibility maintenance hot spots	Refactor behind existing contracts with characterization tests	Broad regression matrix	Table 13.3
P2	Temporary files use predictable <code>/tmp</code> names and often remain	Unique scoped temp directories and cleanup handlers	High-level singleton prevents common collision	S8
P2	Wire semantics use unrestricted strings for many modes/states	Central constants or versioned typed constants/enums	Runtime validators/tests reject illegal values	S2, S5
P2	Broad <code>pkill</code> /ROS-daemon behavior in batch/review tools	Replace with process-group/session ownership and explicit confirmation	Dedicated-host-only warning	S8
P3	Maintained Markdown test counts are stale	Reconcile counts with CMake registration and the tracked test tree during documentation review	This manual records audited counts	S6
P3	Large local build/log/cache trees obscure clean source	Adopt retention/cleanup policy after evidence review	Ignored and explicitly non-authoritative	S6

13.4 High-Risk Maintenance Hot Spots

The following current files combine enough policy and behavior that a local edit can have system-wide effects:

Table 13.3: High-risk maintenance hot spots

File	Approx. lines	Principal risk
<code>mfja_robot_control_config/scripts/room_315_pddl_scenario_generator.py</code>	11,150	Problem construction, planner client, scenario/evidence coupling
<code>mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py</code>	5,053	ROS, Gazebo, fleet, device, and lifecycle integration
<code>mfja_robot_control_config/scripts/room_315_vla_train_v4.py</code>	4,891	Dataset, model, training, and evaluation contract

File	Approx. lines	Principal risk
<code>mfja_robot_control_config/scripts/room_315_closed_loop_executive.py</code>	4,595	Planning, action, effect, and recovery policy
<code>mfja_robot_control_config/scripts/room_315_task_execution.py</code>	3,537	Grounding, authorization and task orchestration
<code>mfja_robot_control_config/scripts/room_315_vla_supervisor.py</code>	3,408	Primitive validation and safety recovery

Before refactoring, create characterization tests at public/pure-core boundaries. Extract one responsibility at a time and preserve message, manifest, PDDL, and failure semantics. A lower line count is not success if it duplicates policy or weakens fail-closed behavior.

13.5 Third-Party and Redistribution Boundary

The description install includes imported KUKA, Stäubli, Motoman, TIAGo, SCHUNK, Zimmer, and other model/CAD material under Apache-2.0, BSD, and CC BY-ND terms. `mfja_3rd_floor_description/THIRD_PARTY.md` is the current provenance record. Before external redistribution, add exact upstream license texts under a maintained `LICENSES/` directory and have the intended distribution reviewed. Preserve attribution when converting or optimizing an asset; do not overwrite the upstream original silently.

13.6 What Must Accompany a Handover

The receiving maintainer needs more than this PDF:

- repository URL, branch, exact commit, intentional unmerged changes, and release/tag policy;
- supported host/ROS/Gazebo versions and a reproducible four-package build;
- current test evidence with commands, exclusions, environment, and logs;
- inventory/location/owner/backup policy for datasets, GGUF files, checkpoints, promoted bundles, authorizations, and sealed evidence;
- independent expected hashes and the prior complete rollback bundle;
- known hardware/GPU/display constraints and any host-local configuration;
- third-party attribution/license material and redistribution decision;
- operational ownership/contact escalation and data-retention policy; and
- open issues mapped to the known-gap register above.

Secrets, credentials, or private artifact URLs must be transferred through an approved secure channel, not embedded into YAML, this PDF, shell history, or Git.

13.7 Release/Handover Acceptance Checklist

Technical acceptance

- ☐ The checked-out commit and intentional worktree changes are recorded.
- ☐ A clean dependency check and four-package build succeed on the supported host.
- ☐ Interfaces, package prefixes, executable/config install surface, and launch arguments match this manual.
- ☐ Focused tests, registered tests, nine source-only cases, and relevant optional tests have results or explicit exclusions.
- ☐ Headless Room 315 smoke passes; full-floor/isolated/GUI checks pass where affected.
- ☐ Rail add, ON/OFF, delayed devices, sensors, headway, target arrival, and fault behavior are verified for the changed surface.
- ☐ External artifact locations, exact hashes, provenance, and backups are verified independently.
- ☐ V4 shadow/readiness/acceptance/fault evidence corresponds to the exact candidate; execution has separate authorization.
- ☐ Rollback to the previous complete bundle has been rehearsed.
- ☐ No build output, cache, local dataset, virtual environment, model, credential, or unintended ignored file is staged/installed.
- ☐ Third-party provenance and exact license obligations are complete for the intended distribution.
- ☐ The Repository Source Index and PDF validation succeed, and the final PDF was visually inspected.
- ☐ Known limitations, unverified claims, owners, and next actions are signed off by sender and receiver.

13.8 Recommended First Maintenance Milestone

Before adding another research feature, the highest-value stabilization work is:

1. make active artifact paths host-portable without weakening hash identity;
2. fix package-share lookup and prove a copied install;
3. register or explicitly schedule all source tests and add clean CI;
4. restrict directory install rules to tracked intended files;
5. complete third-party license texts for the planned distribution; and
6. split public executables from import-only modules and begin tested extraction of the largest integration hot spots.

These actions reduce handover risk while preserving the existing runtime contract. Feature work can then proceed against a reproducible baseline.

13.9 Handover Completion Record

Use the following fields in the project issue/release record; do not fill them only inside a printed copy:

Table 13.4: Handover completion record

Field	Recorded value
Repository/commit	aadf967b1188e680256580408336e7703ff1cb5d on INTERNSHIP-ALI-2026
Manual revision/date	1.0 / 28 August 2026
Build/test evidence	Location, command, counts, environment, exclusions
External artifact register	Owner-controlled location plus independent hashes
Rollback bundle	Exact prior manifest/authorization identity and verification date
Open risks/owners	Issue identifiers mapped to priorities in this chapter
Sender/receiver approval	Names, roles, date, and explicit scope accepted

The project is maintainable when a new engineer can reproduce the base system, locate every authority, explain each data boundary, make a bounded change, prove its effect, and roll it back without relying on undocumented personal state.

Part V

Reference Appendices

Appendix A

Glossary and Naming Conventions

A.1 Source Domains

The glossary associates each term with a readable source domain. The table below maps those domains to the primary owning files without repeating long paths in every definition.

Table A.1: Glossary source domains and owning files

Source domain	Responsibility	Primary source path(s)
Build and install	Package build, dependencies, and installed surface	mfja_3rd_floor_bringup/CMakeLists.txt;mfja_3rd_floor_description/CMakeLists.txt;mfja_rail_interfaces/CMakeLists.txt;mfja_robot_control_config/CMakeLists.txt
Repository	Source and non-source boundaries	.gitignore;README.md;mfja_3rd_floor_bringup/package.xml;mfja_3rd_floor_description/package.xml;mfja_rail_interfaces/package.xml;mfja_robot_control_config/package.xml
Simulation assets	Gazebo worlds and models	mfja_3rd_floor_description/worlds/room_315_only.world;mfja_3rd_floor_description/worlds/mfja_3rd_floor.world;mfja_3rd_floor_description/models/room315_shuttle/model.sdf
Historical visual research	Earlier visual-state generations	mfja_robot_control_config/config/room_315_v1a/visual_state_experiment_a_dataset_v3.yaml;mfja_robot_control_config/config/room_315_v1a/visual_state_experiment_a_dataset_v3r1.yaml
ROS interfaces	Message and service contracts	mfja_rail_interfaces/CMakeLists.txt;mfja_rail_interfaces/msg/ShuttleState.msg;mfja_rail_interfaces/msg/VisualStateObservation.msg
Launch composition	Runtime composition and arguments	mfja_3rd_floor_bringup/launch/room_315_floor_common.py;mfja_robot_control_config/launch/room_315_dual_kinematic_shuttles.launch.py;mfja_robot_control_config/launch/room_315_visual_state_runtime.launch.py;mfja_robot_control_config/launch/room_315_task_execution.launch.py
Rail runtime	Rail state, geometry, and devices	mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py;mfja_robot_control_config/config/room_315_kinematics/rail_network_left.yaml;mfja_robot_control_config/config/room_315_kinematics/rail_network_right.yaml;mfja_robot_control_config/config/room_315_kinematics/rail_devices_left.yaml;mfja_robot_control_config/config/room_315_kinematics/rail_devices_right.yaml
Task execution	Grounding, planning, and authorization	mfja_robot_control_config/scripts/room_315_task_execution.py;mfja_robot_control_config/config/room_315_v1a/task_execution_runtime.yaml;mfja_robot_control_config/config/room_315_v1a/pddl/domain_room315_runtime.pddl
Visual runtime	Active learned-state contract	mfja_robot_control_config/scripts/room_315_visual_state_inference_node.py;mfja_robot_control_config/scripts/room_315_v1a_train_v4.py;mfja_robot_control_config/scripts/room_315_promote_runtime_v4.py;mfja_robot_control_config/config/room_315_v1a/visual_state_runtime.yaml
VLA supervision	Primitive supervision and recovery	mfja_robot_control_config/scripts/room_315_vla_supervisor.py;mfja_robot_control_config/config/room_315_vla/vla_supervisor.yaml;mfja_robot_control_config/launch/room_315_vla_supervisor.launch.py

Table A.2: Project glossary and source domains

Term	Meaning in this project	Source domain
Ament	ROS 2 build/test conventions used by all four packages; packages use <code>ament_cmake</code>	Build and install
Arc length s	Metres travelled from the beginning of the current directed rail segment	Rail runtime; ROS interfaces
Block/segment	One of 14 public rail classes: A12E/I, A14, A1E/I, A23, A2E/I, A34E/I, A3E/I, A4E/I	Rail runtime
Candidate	Immutable proposed learned-runtime bundle before manual promotion	Visual runtime
Checkpoint hash	SHA-256 identity of the exact learned model admitted by the visual/-task contract	Visual runtime; Task execution
Clean clone	Git-tracked repository content without local ignored build output, caches, datasets, or external models	Repository
Colcon overlay	Workspace <code>install/setup.bash</code> layered after the base ROS distribution	Build and install
Copied install	Normal CMake/colcon installation containing copied scripts/assets; exposes source-relative path bugs hidden by symlinks	Build and install
DA sensor	Configured rail approach/branch detector near a switch	Rail runtime
DZI sensor	Identity-bearing slot detector used for exact slot occupancy/arrival	Rail runtime; Task execution
Effect verification	Requiring fresh post-command observation/controller evidence before declaring a primitive complete	Task execution; VLA supervision
FALLING	Latched logical rail fault caused by the absence of a valid directed successor/guard	Rail runtime
Gazebo authority	World/entity/plugin/service state owned by Gazebo, distinct from logical Python state	Simulation assets; Rail runtime
GGUF	External local-language-model file format used by the optional English semantic backend	Task execution
GZ partition	Gazebo transport isolation name; does not isolate fixed ROS topic/service names	Launch composition
Headway	Software minimum center-distance rule between managed shuttles	Rail runtime
Known-empty heart-beat	Blank-name ShuttleState proving a live side currently manages no shuttle	Rail runtime; ROS interfaces
Manifest	Versioned metadata tying code/config/data/model/output identities and hashes into one artifact	Visual runtime; Task execution
PlanSys2	ROS 2 planning framework providing the lifecycle planner and <code>/planner/get_plan</code>	Task execution; Launch composition
Presence	Deterministic present/absent/unknown identity state derived only from rail heartbeats/freshness for learned-runtime gating	Visual runtime; Task execution
Primitive	Bounded deterministic supervisor action such as switches, stoppers, shuttle ON, stop, or done	VLA supervision
Promotion	Manual decision that admits one exact immutable candidate to a stated inference deployment mode	Visual runtime
Public/internal segment name	Left rail's published vocabulary versus rotated raw-geometry vocabulary; right names currently coincide	Rail runtime
QoS	ROS 2 delivery/history/durability policy; compatibility must be checked at both endpoints	ROS interfaces; Rail runtime
Rail authority	Side-specific node state based on topology, CSV, devices, commands, and simulation time	Rail runtime
Runtime authorization	Separate hash-bound permission for a promoted bundle to participate in Gazebo closed-loop task actuation	Task execution; Visual runtime
s_{ratio}	Dimensionless normalized segment coordinate s/L in $[0, 1]$	Rail runtime; ROS interfaces
Sealed final test	Frozen evaluation input whose results must not be used for further tuning and then rerun as if unseen	Visual runtime
Shadow mode	Learned inference publishes under a separate topic root without active fusion/planning mutation	Visual runtime
Source of truth	The owning machine-readable definition used at runtime/build, not an installed copy or historical explanation	Repository; Build and install
Stopper	Delayed binary device; 0 open/pass and 1 closed/stop	Rail runtime; ROS interfaces

Term	Meaning in this project	Source domain
Supervisor	Deterministic VLA primitive validator/recovery node; only optional-stack component that publishes typed rail commands	VLA supervision
Symlink install	Development colcon mode linking many installed programs back to source; convenient but can hide portability/mode issues	Build and install
TaskGoal	Confirmed constrained JSON goal grounded into a fresh symbolic problem	Task execution
Typed feedback	ROS message from the <code>mfja_rail_interfaces</code> package, contrasted with simulation-only JSON diagnostics/control	ROS interfaces
V3/V3R1	Historical visual/dataset research generations retained for provenance/offline reproduction	Historical visual research
V4	Sole learned visual generation admitted to the current active planning path	Visual runtime
VLA	Project term for the optional vision/language/action research stack; learned components do not directly actuate	VLA supervision; Visual runtime; Task execution
Visual facts	Learned segment/position/bounding-box/load outputs after validation, distinct from controller geometry	Visual runtime; ROS interfaces
World internal name	SDF <code><worldname></code> used to construct Gazebo service paths; must match launch expectation	Simulation assets; Launch composition

A.2 Case, Side, and Identity Conventions

ROS endpoints are lowercase absolute names rooted at `/room_315`. Rail side is lowercase `left/right`. Public switches use uppercase `A1--A4`; segments use uppercase `A/E/I` names. Visual identities are short `L1--L4`, `R1--R4`; Gazebo/rail entity names are `room315_left_shuttle_1--4` and `room315_right_shuttle_1--4`. Keep identity and entity distinct in APIs.

Naming is shared by the ROS interfaces, rail runtime, visual runtime, and launch composition domains in Table A.1. Review those owning files together when migrating a public name.

Appendix B

ROS Messages and Services Reference

This appendix documents every message and service defined by `mfja_rail_interfaces`. Table B.1 connects each interface family to its definition and primary runtime implementation. Rebuild the interface package and all consumers after changing any definition.

B.1 Interface Ownership and Source Paths

The definition column in Table B.1 governs the wire layout. The implementation column identifies the primary runtime source for the semantics described in the sections that follow. Interface registration and installation are maintained in `mfja_rail_interfaces/CMakeLists.txt`.

Table B.1: ROS interface ownership and implementation paths

Interface family	Definition path(s)	Runtime implementation path(s)
Shared named state	<code>mfja_rail_interfaces/msg/NamedState.msg</code>	<code>mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py</code>
Shuttle command and state	<code>mfja_rail_interfaces/msg/ShuttleCommand.msg</code> ; <code>mfja_rail_interfaces/msg/ShuttleState.msg</code>	<code>mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py</code>
Switch command and state	<code>mfja_rail_interfaces/msg/SwitchCommand.msg</code> ; <code>mfja_rail_interfaces/msg/SwitchState.msg</code> ; <code>mfja_rail_interfaces/msg/NamedState.msg</code>	<code>mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py</code>
Stopper command and state	<code>mfja_rail_interfaces/msg/StopperCommand.msg</code> ; <code>mfja_rail_interfaces/msg/StopperState.msg</code> ; <code>mfja_rail_interfaces/msg/NamedState.msg</code>	<code>mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py</code>
Sensor reading and feedback	<code>mfja_rail_interfaces/msg/SensorReading.msg</code> ; <code>mfja_rail_interfaces/msg/SensorFeedback.msg</code>	<code>mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py</code> ; <code>mfja_robot_control_config/scripts/room_315_task_execution.py</code>
Visual shuttle state	<code>mfja_rail_interfaces/msg/VisualShuttleState.msg</code>	<code>mfja_robot_control_config/scripts/room_315_visual_state_inference_node.py</code> ; <code>mfja_robot_control_config/scripts/room_315_task_execution.py</code>
Visual observation	<code>mfja_rail_interfaces/msg/VisualStateObservation.msg</code> ; <code>mfja_rail_interfaces/msg/VisualShuttleState.msg</code>	<code>mfja_robot_control_config/scripts/room_315_visual_state_inference_node.py</code> ; <code>mfja_robot_control_config/scripts/room_315_task_execution.py</code>
Add-shuttle service	<code>mfja_rail_interfaces/srv/AddShuttle.srv</code>	<code>mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py</code>

B.2 Shared Named State

B.2.1 NamedState.msg

```
string name
string state
```


Switch command input accepts E/EXTERIOR or I/INTERIOR; canonical rail state is E or I. Stopper input accepts canonical 0 (open/pass) or 1 (closed/stop), with documented textual aliases normalized by the node. Published state is canonical. Selectors may name an individual configured device or the supported group ALL; switch groups also include the runtime's documented side/group selectors.

B.3 Shuttle Command and State

B.3.1 ShuttleCommand.msg

```
std_msgs/Header header
string name
string command
string start_slot
float64 speed
string target_slot
```

Table B.2: Shuttle command field contract

Field	Contract
header	Command stamp/frame metadata; the rail validates payload fields rather than using the header as completion evidence
name	Exact Gazebo entity name, empty only for the supported single-shuttle convenience, or ALL
command	Canonical operator values ON, OFF, RESET, REMOVE; aliases normalize internally but should not be used in durable clients
start_slot	Lifecycle/start metadata retained for compatibility; add uses the service request; reset uses configured origin
speed	Positive configured speed applied only to ON; zero means pre-preserve current ON setting for applicable commands
target_slot	Optional slot 1–4, legal only with ON; empty means continuous motion

B.3.2 ShuttleState.msg

```
std_msgs/Header header
string name
string mode
string current_segment
float64 s
float64 x
float64 y
float64 z
float64 yaw
float64 speed
string reached_target_slot
```

Legal modes are MOVING, WAITING, DISABLED, and FALLING. *s* is arc length in metres on *current_segment*; *x*, *y*, *z*, *yaw* are transformed Gazebo pose values. *speed* is retained configured travel speed, not instantaneous velocity. *reached_target_slot* is nonempty only after the controller clamps an explicit target and disables the shuttle. With no managed shuttle, the node publishes a blank-name heartbeat to mean known-empty.

B.4 Switch and Stopper Arrays

B.4.1 SwitchCommand.msg and SwitchState.msg

```
std_msgs/Header header
NamedState[] switches
```

Command and state share the same structure but not the same meaning: command is a requested target; state is delayed actual rail authority.

B.4.2 StopperCommand.msg and StopperState.msg

```
std_msgs/Header header
NamedState[] stoppers
```

Canonical state 0 permits passage and 1 stops/blocks. Published state changes only when the scheduled delay expires in simulation time.

B.5 Sensor Feedback

B.5.1 SensorReading.msg

```
string name
string sensor_type
uint8 active
string shuttle_name
string segment
float64 s
float64 s_ratio
```

`sensor_type` is currently always `sensor`; purpose is encoded by the configured name/documentation. `active` is 0 clear or 1 occupied. When active, `shuttle_name` identifies the occupant. Segment and position are privileged simulator metadata. The task gateway deliberately ignores those three position fields and uses only fresh active/name/identity for exact DZI slot grounding.

B.5.2 SensorFeedback.msg

```
std_msgs/Header header
SensorReading[] readings
```

Each side publishes its complete configured/derived reading array at the configured sensor rate.

B.6 Visual Shuttle State

B.6.1 VisualShuttleState.msg

```
string identity
string presence_state
bool visual_facts_valid
string side
string block
float32[4] bbox_xywh
float64 s_m
float64 s_ratio
float64 segment_length_m
```

```
string loaded_state
float64 segment_confidence
float64 loaded_confidence
```

Canonical identities are L1--L4, R1--R4. Presence is present, absent, or unknown. Valid learned side is left/right; block is one of the 14 public segment classes; loaded state is empty/loaded. The bounding box order is (x, y, w, h) according to the versioned visual contract. Consumers must honor `visual_facts_valid`; absent/unknown entries do not carry usable learned facts merely because numeric fields exist.

B.7 Visual Observation

B.7.1 VisualStateObservation.msg

```
std_msgs/Header header
builtin_interfaces/Time left_image_stamp
builtin_interfaces/Time right_image_stamp
string schema_version
string checkpoint_sha256
string stage
bool accepted
bool stabilized
bool stale
bool model_ready
bool input_ready
bool presence_ready
bool state_fusion_ready
string[] validation_reasons
string[] clamped_fields
float64 preprocessing_latency_ms
float64 inference_latency_ms
float64 validation_latency_ms
float64 cycle_latency_ms
uint64 accepted_frame_count
uint64 rejected_frame_count
uint64 stale_frame_count
VisualShuttleState[] shuttles
```

stage distinguishes raw/validation/accepted runtime output. An execution consumer verifies timestamps, schema, checkpoint, readiness, acceptance, staleness, and task-required shuttle facts together. `stabilized` is a diagnostic that says the optional temporal filter was applied; it is normally false on valid accepted observations when that filter is disabled and is not a task-gateway required-true field. Counters are process-lifetime diagnostics and are not task sequence numbers.

B.8 Add Shuttle Service

B.8.1 AddShuttle.srv

```
string name
string start_slot
float64 speed
bool start_enabled
---
bool success
string message
string name
```

An empty name selects the next unused entity and an empty slot selects the next available configured slot. A speed that compares greater than zero is used; otherwise the configured default is substituted, with no separate finiteness validation. Duplicate entities, invalid slots, and—when configured—occupied start slots are rejected. A successful response means the logical runtime accepted the shuttle. Gazebo spawn remains asynchronous, so visual existence requires separate verification. Removal, reset, ON, and OFF are ShuttleCommand actions, not services.

B.9 Compatibility Rules

Adding a field is still a ROS interface change and requires dependent rebuilds. Renaming, removing, retyping, reordering fixed arrays, or changing units/legal strings is a contract migration. Record a schema/version strategy for persisted JSON/datasets separately; ROS message compatibility alone does not migrate recorded episodes or manifests.

Interface registration and dependency declarations are maintained in `mfja_rail_interfaces/CMakeLists.txt`; field definitions are maintained at the paths in Table B.1.

Appendix C

Launch Argument Reference

This appendix documents the public and lower-level launch arguments used by the supported simulation, visual-runtime, and task-execution profiles. Launch values remain strings until launch or node conversion. Confirm the effective arguments in an installed workspace with:

```
ros2 launch <package> <file.launch.py> --show-args
```

C.1 Launch Surfaces and Source Paths

Table C.1 is the source registry for this appendix. Each argument table uses the same readable launch-surface name. Verify defaults and forwarding behavior in the corresponding source files after a launch change.

Table C.1: Launch surfaces and owning source files

Launch surface	Source path(s)
Stable floor core and base	mfja_3rd_floor_bringup/launch/room_315_floor_common.py; mfja_3rd_floor_bringup/launch/room_315_only.launch.py; mfja_3rd_floor_bringup/launch/full_floor.launch.py;mfja_robot_control_config/launch/multi_robot_sim.launch.py
Stable floor rail and shuttle	mfja_3rd_floor_bringup/launch/room_315_floor_common.py; mfja_robot_control_config/launch/room_315_dual_kinematic_shuttles.launch.py
Stable floor optional VLA	mfja_3rd_floor_bringup/launch/room_315_floor_common.py; mfja_robot_control_config/launch/room_315_vla_supervisor.launch.py
Stable isolated robot	mfja_3rd_floor_bringup/launch/single_industrial_robot.launch.py;mfja_robot_control_config/launch/isolated_industrial_robot.launch.py
VLA supervisor and recorder	mfja_robot_control_config/launch/room_315_vla_supervisor.launch.py
V4 visual runtime	mfja_robot_control_config/launch/room_315_visual_state_runtime.launch.py
Task execution	mfja_robot_control_config/launch/room_315_task_execution.launch.py
Observation-only acceptance	mfja_robot_control_config/launch/room_315_runtime_acceptance.launch.py
Lower-level arguments	mfja_robot_control_config/launch/multi_robot_sim.launch.py; mfja_robot_control_config/launch/room_315_dual_kinematic_shuttles.launch.py

C.2 Stable Floor Wrappers (49 Arguments)

mfja_3rd_floor_bringup/launch/room_315_only.launch.py and mfja_3rd_floor_bringup/launch/full_floor.launch.py contain no local declarations; both use mfja_3rd_floor_bringup/launch/room_315_floor_common.py. Their public names are identical and their profile defaults differ only where shown.

C.2.1 Core and base arguments

Table C.2: Stable floor core and base arguments

Argument	Room default	Full default	Meaning
world_name	room_315_only	mfja_3rd_floor	World file/stem and expected internal name
robots	empty	empty	Empty uses enabled YAML entries; none, all, or selector list
robot_config	mfja_robot_control_config/room_315_only.yaml	mfja_robot_control_config/room_315_only.yaml	Registry path
gripper_config	mfja_robot_control_config/gripper_command_defaults.yaml	mfja_robot_control_config/gripper_command_defaults.yaml	Gripper range/default path
gz_partition	room_315_only_<PID>	mfja_3rd_floor_<PID>	Gazebo transport partition
use_sim_time	true (true/false)		Use /clock
gui	true (true/false)		Start Gazebo client
gui_config	mfja_robot_control_config/room315_runtime_safe_gui.config	mfja_robot_control_config/mfja_light_gui.config	GUI profile
start_paused	false	true	Gazebo initial pause state
initial_loop_mode	auto		Initial visual conveyor loop mode
pause_during_switch_update	false (true/false)		Visual controller pause policy
room315_visual_debug_colors	true (true/false)		Switch/device debug colouring
enable_room315_vla_obstacles	false (true/false)		Include removable VLA obstacle markers

C.2.2 Rail and shuttle arguments

Table C.3: Stable floor rail and shuttle arguments

Argument	Both defaults	Meaning
enable_room315_kinematic_shuttles	true	Include delayed dual-rail launch
enable_room315_right_rail	true	Start right side
enable_room315_left_rail	true	Start left side
room315_right_start_slot	2	Scalar fallback start slot
room315_right_start_slots	empty	Per-selected-shuttle slot list
room315_right_start_positions	empty	Per-selected-shuttle segment/position overrides
room315_left_start_slot	2	Scalar fallback start slot
room315_left_start_slots	empty	Per-selected-shuttle slot list
room315_left_start_positions	empty	Per-selected-shuttle segment/position overrides
room315_shuttle_speed	0.2	Configured speed, m/s
room315_falling_stop_offset_m	0.0	Fault presentation offset, m
room315_shuttle_collision_distance_m	0.33	Software headway, m
room315_shuttles_start_enabled	false	Begin selected shuttles moving
room315_right_shuttle_count	0	Initial right identity count
room315_right_active_identities	empty	Explicit right identities
room315_left_shuttle_count	0	Initial left identity count
room315_left_active_identities	empty	Explicit left identities
room315_identity_selection_mode	count_prefix	count_prefix or explicit
room315_switch_motion_delay_s	0.3	Requested-to-actual switch delay, s

Argument	Both defaults	Meaning
room315_stopper_motion_delay_s	0.1	Requested-to-actual stopper delay, sim s
room315_sensor_publish_rate_hz	30.0	Feedback frequency
room315_sync_sensor_feedback_to_motion_tick	true	Align feedback to motion tick
room315_gazebo_set_pose_rate_hz	30.0	Gazebo pose call rate
room315_show_device_markers	true	Spawn device markers
room315_enable_payload_visuals	true	Spawn/follow logical payload visuals
room315_payload_pose_x_offset_m	-0.08	Payload longitudinal visual offset
room315_right_loaded_shuttles	empty	Initially loaded right identities
room315_left_loaded_shuttles	empty	Initially loaded left identities
room315_sensor_marker_visual_hold_s	0.35	Active-colour visual hold, s

C.2.3 Optional VLA arguments

Table C.4: Stable floor optional VLA arguments

Argument	Both defaults	Meaning
enable_room315_vla	false	Include supervisor/camera/optional recorder at 5 s
room315_clear_vla_obstacle_pose_cache	true	Unlink configured cache before includes
room315_vla_obstacle_pose_file	~/ .ros/room315_vla_obstacles.json	Exact cache target
enable_room315_vla_camera_bridge	true	Bridge paired RGB-D topics
enable_room315_vla_dataset_recorder	false	Start recorder
room315_vla_dataset_dir	~/ .ros/room315_visual_state_datasets/demo	Recorder output
room315_vla_dataset_sample_period_s	0.2	Recorder sampling period

C.3 Stable Isolated-Robot Wrapper (9 Arguments)

Table C.5: Stable isolated-robot arguments

Argument	Default	Meaning
robot	kuka	Bounded short selector
robot_config	mfja_robot_control_config/config/robots_room_315_only.yaml	Robot registry
gripper_config	mfja_robot_control_config/config/gripper_command_defaults.yaml	Gripper config
world_name	isolated_industrial_robot	Minimal world
gz_partition	isolated_industrial_robot_<PID>	Transport partition
use_sim_time	true	Simulation clock

Argument	Default	Meaning
gui	true	Gazebo client
gui_config	mfja_robot_control_config/config/mfja_light.gui.config	Gazebo GUI configuration
start_paused	false	Initial pause

The stable wrapper explicitly declares and forwards the other eight values. The included lower launch declares `gui_config`; ROS launch therefore exposes it as the ninth effective argument in `--show-args`, and a top-level override is inherited by the included launch. Recheck this inherited surface whenever the include is refactored.

C.4 VLA Supervisor/Recorder Launch (20 Arguments)

Table C.6: VLA supervisor and recorder arguments

Argument	Default	Meaning
use_sim_time	true	Simulation time
enable_camera_bridge	true	Paired RGB-D bridge
enable_supervisor	true	Deterministic primitive supervisor
enable_dataset_recorder	false	Episode recorder
config_path	empty	Optional supervisor YAML; empty uses code defaults
command_topic	/room_315/vla/command	Primitive JSON input
status_topic	/room_315/vla/status	Supervisor JSON status
user_goal_topic	/room_315/vla/user_goal	Recorder-only goal input
episode_control_topic	/room_315/vla/episode_control	Recorder lifecycle input
dataset_status_topic	/room_315/vla/dataset_status	Recorder status output
dataset_dir	~/ros/room315_visual_state_datasets/demo	Recorder output
dataset_sample_period_s	0.2	Recorder period
dataset_auto_start_on_goal	true	Recorder policy
emergency_stop_topic	/room_315/vla/emergency_stop	Boolean emergency input
image_topic	empty	Optional legacy primary image
right_image_topic	/room_315/vla/right_rail_rgbd/image	Right RGB
left_image_topic	/room_315/vla/left_rail_rgbd/image	Left RGB
camera_info_topic	empty	Optional legacy camera info
right_camera_info_topic	/room_315/vla/right_rail_rgbd/camera_info	Right intrinsics
left_camera_info_topic	/room_315/vla/left_rail_rgbd/camera_info	Left intrinsics

The high-level floor passes six arguments total: five wrapper values (simulation time, camera bridge, recorder enablement, dataset directory, and sample period) plus `enable_supervisor=true`. The other values retain these defaults. `user_goal_topic` does not feed the supervisor.

C.5 V4 Visual Runtime Launch (11 Arguments)

Table C.7: V4 visual runtime arguments

Argument	Default	Meaning
<code>use_sim_time</code>	<code>true</code>	Simulation time
<code>enable_camera_bridge</code>	<code>true</code>	RGB-only bridge; disable if supervisor owns it
<code>runtime_config</code>	<code>installed mfja_robot_control_config/config/room_315_vla/visual_state_runtime.yaml</code>	Parameter YAML
<code>node_name</code>	<code>room_315_visual_state_inference_node</code>	ROS node name
<code>runtime_generation</code>	<code>v4 (only choice)</code>	Active generation
<code>runtime_mode</code>	<code>empty</code>	Preserve YAML (currently active); or active/shadow
<code>v4_promotion_manifest</code>	<code>empty</code>	Optional immutable JSON promotion-manifest path override
<code>v4_promotion_manifest_sha256</code>	<code>empty</code>	Optional expected-manifest-hash override
<code>device</code>	<code>auto</code>	<code>auto/cpu/cuda</code>
<code>dry_run_state_fusion</code>	<code>true</code>	Gate external PlanSys2 predicate mutation; fusion/observation still run
<code>plansys2_update_enabled</code>	<code>false</code>	Prevent predicate-service calls

YAML loads first. The launch always overrides time, device, generation, dry-run, and PlanSys2-update values; the mode/manifest/hash override only when nonempty.

C.6 Task Execution Launch (6 Arguments)

Table C.8: Task-execution arguments

Argument	Default	Meaning
<code>use_sim_time</code>	<code>true</code>	Simulation time
<code>execution_enabled</code>	<code>false</code>	False needs no authorization artifact; true requests authorized supervised actuation
<code>enable_plansys2</code>	<code>true</code>	Start bundled lifecycle planner

Argument	Default	Meaning
planner_service	/planner/get_plan	GetPlan endpoint
external_obstacles_disable	true	Only supported value; false fails because obstacle perception is not configured
runtime_config	installed mfja_robot_control_config/config/room_315_vla/task_execution_runtime.yaml	Parameter/authorization YAML

Launch forces the installed runtime PDDL path. `enable_plansys2=false` only suppresses the bundled planner; an external compatible service must then exist. Enabling execution additionally requires the authorization, promotion manifest, manual decision, and bound evidence to be non-writable regular files whose final path components are not symbolic links.

C.7 Observation-Only Runtime Acceptance (11 Arguments)

Table C.9: Observation-only runtime acceptance arguments

Argument	Default	Meaning
candidate_directory	required	Immutable candidate with scenario/config/state/checkpoint files
scenario_id	required	Exact unique row in candidate scenario manifest
output_root	required	New output root; existing path is refused
gui	true	Gazebo GUI
world_readiness_timeout_s	45.0	World gate deadline
scene_readiness_timeout_s	45.0	Rail/scenario gate deadline
camera_readiness_timeout_s	45.0	Camera gate deadline
runtime_readiness_timeout_s	120.0	Verified V4 runtime gate deadline
record_duration_s	60.0	Observation recording duration
enable_task_execution	false	Compatibility guard; true is rejected
execution_enabled	false	Second compatibility guard; true is rejected

The launch verifies candidate schema/hash before starting Gazebo, then gates world → scene → camera → runtime → recorder → report. It forces no robots, stopped exact scenario shuttles, no external obstacles/markers/debug colours, dry-run fusion, no PlanSys2 mutation, and no task execution. It is not an actuation acceptance test.

C.8 Lower-Level-Only Arguments

All 13 `mfja_robot_control_config/launch/multi_robot_sim.launch.py` arguments are exposed by the two stable floor wrappers (`visual_debug_colors` is renamed to `room315_visual_debug_colors`). Of 34 dual-rail arguments, only these three are not exposed at the stable floor level:

Table C.10: Lower-level-only launch arguments

Argument	Default	Meaning
<code>path_backend</code>	<code>cubic_hermite</code>	<code>cubic_hermite</code> or <code>diagnostic polyline interpolation</code>
<code>payload_type</code>	<code>box</code>	Structured payload visual type
<code>payload_pose_z_offset_m</code>	<code>0.0</code>	Additional payload vertical visual offset

Use lower-level launches only when their narrower lifecycle is intentional. Direct concurrent use bypasses the high-level singleton/startup policy and can collide on fixed ROS names or predictable temporary files.

C.9 Composition Consistency Rules

These rules apply to the public and optional launch surfaces in Table C.1. Use that registry to locate each owning include and argument declaration.

- Floor bringup never includes visual inference or task execution.
- Assign one RGB bridge: if the supervisor launch owns it, visual runtime uses `enable_camera_bridge:=false`.
- If supervisor command/status topics change, update gateway parameters to the same names; high-level floor arguments do not expose that remap.
- Task execution currently requires floor `enable_room315_vla_obstacles=false` and gateway `external_obstacles_disabled=true`; the obstacle-enabled path is unsupported and fails closed.
- Visual `plansys2_update_enabled=true` needs separately available Problem Expert predicate services; task launch provides only planner.
- Multiple publishers on `/diagnostics` are intentional and share the same message type.

Appendix D

Repository Source Index

This appendix is a package-oriented index of the project’s source surface. It groups implementation modules, launch files, configuration contracts, Gazebo models, and tests by responsibility so a maintainer can locate the owning file without searching unrelated repository material. Offline and historical tools are identified separately from the active ROS 2 runtime. Runtime-created data, local build output, and external model artifacts are outside this index.

At the documented commit, the repository contains 1791 tracked files, including 112 control-package scripts, 98 control tests, 7 description tests, 47 Gazebo model directories, 13 launch/support modules, 11 messages, and 1 service. Use the package- and category-oriented tables below to locate the corresponding files; use the earlier chapters to interpret their runtime relationships.

D.1 Repository overview

Table D.1 classifies the repository’s top-level areas. The four ROS 2 packages are the maintained runtime surface; the other areas remain important as design context, evidence, reproduction support, or training material.

Table D.1: Repository top-level areas and maintenance classification

Top-level area	Files	Maintenance classification
mfja_rail_interfaces	14	Typed rail messages and the AddShuttle lifecycle service.
mfja_3rd_floor_description	242	Worlds, Gazebo models, simulation plugins, and structural tests.
mfja_robot_control_config	278	Runtime control, rail, planning, perception, dataset, configuration, and tests.
mfja_3rd_floor_bridge	6	Stable user-facing launch entry points and shared composition.
docs	29	Maintained engineering notes, operational guidance, design records, and handover material.
report	1199	Academic report, immutable evidence, and offline reproduction utilities.
tutorial_videos	16	Training material and recorded operator walkthroughs.
(repository root)	7	Repository metadata, overview material, auxiliary Nix shell, and standalone guides.

D.2 Python executables and support modules

These tables cover every tracked Python file directly under `mfja_robot_control_config/scripts`. “CMake-listed” means that the file appears in the package’s explicit `install(PROGRAMS)` list. “Executable bit” records the tracked source mode; it matters for symlink installs and `ros2pkgexecutables`. A support module may be CMake-listed without being a public CLI.

D.2.1 Robot and demonstration control

Table D.2: Python modules: Robot and demonstration control

File	CMake-listed	Executable bit	Primary responsibility
mfja_robot_control_config/scripts/conveyor_loop_mode_controller.py	Yes	Yes	Controls the visual Room 315 switch assemblies and mirrors their accepted loop state in Gazebo.
mfja_robot_control_config/scripts/presentation_left_slot3_kuka_slot2.py	Yes	Yes	Runs the bounded presentation scenario with a left shuttle and KUKA.
mfja_robot_control_config/scripts/robot_gripper_command.py	Yes	Yes	Publish percentage-based actions or bounded positions to an MFJA gripper.
mfja_robot_control_config/scripts/robot_joint_command.py	Yes	Yes	Validates robot selectors and joint targets, then publishes a namespaced JointTrajectory.

D.2.2 Rail runtime and fleet foundations

Table D.3: Python modules: Rail runtime and fleet foundations

File	CMake-listed	Executable bit	Primary responsibility
mfja_robot_control_config/scripts/room_315_device_position_tool.py	Yes	Yes	Converts between Gazebo coordinates and rail segment/arc positions for device calibration.
mfja_robot_control_config/scripts/room_315_kinematic_shuttle.py	Yes	Yes	ROS-independent directed-rail geometry, routing, interpolation, and one-shuttle motion core.
mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py	Yes	Yes	ROS 2/Gazebo adapter for multi-shuttle lifecycle, rail devices, markers, payloads, and pose updates.
mfja_robot_control_config/scripts/room_315_multi_shuttle.py	Yes	No	Room 315 multi-shuttle research utilities.
mfja_robot_control_config/scripts/room_315_rail_defaults.py	Yes	No	Static Room 315 rail defaults shared by the kinematic runtime.
mfja_robot_control_config/scripts/room_315_rail_devices.py	Yes	No	Rail-device schema and YAML loader for the Room 315 kinematic runtime.
mfja_robot_control_config/scripts/room_315_shuttle_geometry.py	Yes	No	Shared world-space geometry checks for Room 315 rail shuttles.

D.2.3 Visual datasets, models, training, and evaluation

Table D.4: Python modules: Visual datasets, models, training, and evaluation

File	CMake-listed	Executable bit	Primary responsibility
mfja_robot_control_config/scripts/room_315_arbitrary_subset_capture_v2.py	Yes	No	Prepare and operate the guarded Room 315 arbitrary-subset capture package.
mfja_robot_control_config/scripts/room_315_arbitrary_subset_visual.py	Yes	No	Plan and audit true arbitrary-identity-subset Room 315 visual datasets.
mfja_robot_control_config/scripts/room_315_arbitrary_subset_visual_v2.py	Yes	Yes	Build and audit the physically feasible Room 315 arbitrary-subset v2 design.
mfja_robot_control_config/scripts/room_315_dataset_partition_overlap_post_hoc.py	Yes	No	Build a post-hoc cross-protocol overlap audit for Room 315 datasets.
mfja_robot_control_config/scripts/room_315_experiment_a_smoke_v2_package.py	Source only	No	Build the isolated, balanced local Experiment-A Smoke V2 package.
mfja_robot_control_config/scripts/room_315_experiment_a_v3r1_corrected_package.py	Yes	No	Build the immutable corrected Room 315 Experiment-A V3R1 package.
mfja_robot_control_config/scripts/room_315_experiment_a_v3r1_package.py	Yes	No	Build the data-light, guarded Room 315 Experiment-A Kairos package.
mfja_robot_control_config/scripts/room_315_grouped_visual_splits.py	Yes	No	Create and verify leakage-safe Room 315 fixed-eight visual splits.
mfja_robot_control_config/scripts/room_315_json_io.py	Yes	No	Shared JSONL readers for Room 315 offline tools.
mfja_robot_control_config/scripts/room_315_kairos_package_checks.py	Yes	No	Fail-closed fixed-eight dataset checks for future Kairos packages.
mfja_robot_control_config/scripts/room_315_kairos_visual_training_package.py	Yes	No	Build and verify the frozen Room 315 Kairos visual-state training package.
mfja_robot_control_config/scripts/room_315_visual_acceptance_v4.py	Yes	No	Pure, fail-closed acceptance gates for Room 315 visual V4.
mfja_robot_control_config/scripts/room_315_visual_calibration_v4.py	Yes	No	Validation-only segment calibration for the Room 315 V4 visual model.
mfja_robot_control_config/scripts/room_315_visual_contract_v4.py	Yes	No	Side-isolated Room 315 visual contract with a legacy V3 adapter.
mfja_robot_control_config/scripts/room_315_visual_dataset_audit.py	Yes	Yes	Audit Room 315 visual-state scenarios, oracle labels, and dataset splits.
mfja_robot_control_config/scripts/room_315_visual_error_analysis.py	Yes	No	Read-only post-pilot error analysis for the Room 315 visual-state model.

File	CMake-listed	Executable bit	Primary responsibility
mfja_robot_control_config/scripts/room_315_visual_final_test_v4.py	Source only	No	One-shot, fail-closed evaluator for the sealed Room 315 visual V4 Test.
mfja_robot_control_config/scripts/room_315_visual_fleet.py	Yes	No	Fail-closed authoritative fleet and block vocabulary for Room 315 vision.
mfja_robot_control_config/scripts/room_315_visual_label_exporter.py	Yes	Yes	Build a curated Room 315 visual-state dataset from approved Gazebo episodes.
mfja_robot_control_config/scripts/room_315_visual_manual_review.py	Yes	Yes	Build and validate the human review gate for Room 315 visual captures.
mfja_robot_control_config/scripts/room_315_visual_model.py	Yes	Yes	Shared Room 315 paired-camera ResNet-18 model definition.
mfja_robot_control_config/scripts/room_315_visual_model_v4.py	Yes	No	Experimental Room 315 V4 paired-camera visual-state model.
mfja_robot_control_config/scripts/room_315_visual_runtime.py	Yes	Yes	Pure runtime for the approved Room 315 paired-camera visual-state model.
mfja_robot_control_config/scripts/room_315_visual_runtime_cpu_smoke.py	Yes	Yes	One-pair CPU inference smoke using an explicitly supplied validation split.
mfja_robot_control_config/scripts/room_315_visual_runtime_fusion.py	Yes	Yes	Deterministic Room 315 visual-fact fusion and PlanSys2 boundary.
mfja_robot_control_config/scripts/room_315_visual_runtime_v4.py	Yes	No	Fail-closed runtime core for the manually reviewed Room 315 V4 model.
mfja_robot_control_config/scripts/room_315_visual_runtime_v4_fault_injection.py	Yes	Yes	Deterministic V4 fault-injection verification.
mfja_robot_control_config/scripts/room_315_visual_runtime_validation.py	Yes	Yes	Pure deterministic validation and stabilization for Room 315 predictions.
mfja_robot_control_config/scripts/room_315_visual_scenario_generator.py	Yes	Yes	Generate deterministic Gazebo scenes for Room 315 visual-state training.
mfja_robot_control_config/scripts/room_315_visual_scenario_runner.py	Yes	Yes	Run Room 315 visual scenarios and capture the new image-only dataset.
mfja_robot_control_config/scripts/room_315_visual_shadow_compare.py	Yes	No	Observation-only paired-frame comparison for V3 active and V4 shadow.
mfja_robot_control_config/scripts/room_315_visual_smoke_report.py	Yes	No	Create a manual-inspection index for a Room 315 visual smoke package.
mfja_robot_control_config/scripts/room_315_visual_state_capture.py	Yes	Yes	Capture one Room 315 scene directly into the visual-state dataset format.

File	CMake-listed	Executable bit	Primary responsibility
mfja_robot_control_config/scripts/room_315_visual_state_dataset.py	Yes	No	Shared helpers for the Room 315 visual-state dataset mode.
mfja_robot_control_config/scripts/room_315_visual_state_inference_node.py	Yes	Yes	ROS 2 runtime node for the approved Room 315 visual-state checkpoint.
mfja_robot_control_config/scripts/room_315_visual_state_smoke.py	Yes	No	Visual-state to PlanSys2 integration smoke helpers.
mfja_robot_control_config/scripts/room_315_visual_training_v4.py	Yes	No	Losses and metrics for the Room 315 fixed-eight visual model V4.
mfja_robot_control_config/scripts/room_315_visual_v3_audit.py	Yes	No	Conditional coverage and integrity audit for Room 315 visual dataset V3.
mfja_robot_control_config/scripts/room_315_visual_v3_capture.py	Yes	No	Guarded capture/resume/finalisation for Room 315 hard-case visual V3.
mfja_robot_control_config/scripts/room_315_visual_v3_common.py	Yes	No	Shared fail-closed utilities for the Room 315 hard-case visual V3 data.
mfja_robot_control_config/scripts/room_315_visual_v3_generator.py	Yes	No	Materialise deterministic, collision-free Room 315 visual V3 manifests.
mfja_robot_control_config/scripts/room_315_visual_v3_pipeline.py	Yes	No	Run the guarded long-duration Room 315 V3 capture pipeline.
mfja_robot_control_config/scripts/room_315_visual_v3_quota_planner.py	Yes	No	Deterministic quota planner for Room 315 hard-case visual dataset V3.
mfja_robot_control_config/scripts/room_315_visual_v3_splitter.py	Yes	No	Create/verify train+validation-only grouped Room 315 visual V3 splits.
mfja_robot_control_config/scripts/room_315_visual_v3r1_audit.py	Yes	No	Static, smoke, and final audits for Room 315 visual V3R1.
mfja_robot_control_config/scripts/room_315_visual_v3r1_capture.py	Yes	No	Guarded capture/resume entry point for Room 315 visual V3R1.
mfja_robot_control_config/scripts/room_315_visual_v3r1_common.py	Yes	No	Shared constants and semantic helpers for Room 315 visual V3R1.
mfja_robot_control_config/scripts/room_315_visual_v3r1_generator.py	Yes	No	Build immutable corrected Room 315 visual V3R1 manifests.
mfja_robot_control_config/scripts/room_315_visual_v3r1_pipeline.py	Yes	No	Run the guarded long-duration Room 315 V3R1 capture pipeline.
mfja_robot_control_config/scripts/room_315_visual_v3r1_quota_planner.py	Yes	No	Deterministic exact-offset quota planner for Room 315 visual V3R1.

File	CMake-listed	Executable bit	Primary responsibility
mfja_robot_control_config/scripts/room_315_visual_v3r1_reuse.py	Yes	No	Validate and immutably import completed V3 episodes into V3R1.
mfja_robot_control_config/scripts/room_315_visual_v3r1_splitter.py	Yes	No	Create the train/validation-only grouped V3R1 split package.
mfja_robot_control_config/scripts/room_315_visual_v4_final_test.py	Yes	No	Preregister, capture, and seal the untouched Room 315 V4 final Test.
mfja_robot_control_config/scripts/room_315_visual_v4_final_test_coverage_compat.py	Source only	No	Project the sealed V2 coverage extension into evaluator-canonical controls.
mfja_robot_control_config/scripts/room_315_visual_v4_final_test_coverage_extension.py	Source only	No	Prepare the pre-inference Room 315 V4 final-Test coverage extension.
mfja_robot_control_config/scripts/room_315_vla_train_local.py	Yes	Yes	Small custom Room 315 visual-state trainer.
mfja_robot_control_config/scripts/room_315_vla_train_v4.py	Yes	No	Fail-closed trainer and evaluator for the isolated Room 315 visual V4.

D.2.4 Language and TaskGoal understanding

Table D.5: Python modules: Language and TaskGoal understanding

File	CMake-listed	Executable bit	Primary responsibility
mfja_robot_control_config/scripts/room_315_task_goal_benchmark.py	Yes	Yes	Evaluate the offline Room 315 English task-goal corpus.
mfja_robot_control_config/scripts/room_315_task_goal_builder.py	Yes	No	Deterministic Room 315 TaskGoal builder and strict JSON parser.
mfja_robot_control_config/scripts/room_315_task_goal_cli.py	Yes	Yes	Interactive Room 315 task-goal command interface.
mfja_robot_control_config/scripts/room_315_task_goal_dialogue.py	Yes	No	Stateful clarification and confirmation for Room 315 TaskGoalDrafts.
mfja_robot_control_config/scripts/room_315_task_goal_fusion.py	Yes	No	Evidence-aware fusion for Room 315 task-goal parsing.
mfja_robot_control_config/scripts/room_315_task_goal_parsers.py	Yes	No	Offline parser interfaces for Room 315 TaskGoal-Draft understanding.
mfja_robot_control_config/scripts/room_315_task_goal_schema.py	Yes	No	Strict offline TaskGoal draft schema for Room 315 goal understanding.
mfja_robot_control_config/scripts/room_315_task_goal_semantic.py	Yes	No	Local semantic-model runtime and strict envelope schema for Room 315 goals.

File	CMake-listed	Executable bit	Primary responsibility
mfja_robot_control_config/scripts/room_315_task_goal_semantic_smoke.py	Yes	Yes	Offline smoke checks for Room 315 task-goal semantic understanding.
mfja_robot_control_config/scripts/room_315_task_goal_validation.py	Yes	No	Deterministic Room 315 domain validation and TaskGoal construction.
mfja_robot_control_config/scripts/setup_room315_intent_model.py	Yes	Yes	Install and verify the local Room 315 intent GGUF checkpoint.

D.2.5 Contracts, planning, and execution

Table D.6: Python modules: Contracts, planning, and execution

File	CMake-listed	Executable bit	Primary responsibility
mfja_robot_control_config/scripts/room_315_closed_loop_executive.py	Yes	No	Closed-loop Room 315 symbolic executive.
mfja_robot_control_config/scripts/room_315_contracts.py	Yes	No	Versioned Room 315 neuro-symbolic closed-loop contracts.
mfja_robot_control_config/scripts/room_315_observed_state_provider.py	Yes	No	ObservedState providers for the Room 315 closed-loop planner boundary.
mfja_robot_control_config/scripts/room_315_pddl_dataset_report.py	Yes	Yes	Coverage report for manual and PDDL-generated Room 315 VLA datasets.
mfja_robot_control_config/scripts/room_315_pddl_language_generator.py	Yes	Yes	Deterministic language generation for Room 315 PDDL scenarios.
mfja_robot_control_config/scripts/room_315_pddl_plan_translator.py	Yes	Yes	Translate Room 315 PDDL plan steps into structured primitive commands.
mfja_robot_control_config/scripts/room_315_pddl_scenario_generator.py	Yes	Yes	Room 315 PlanSys2-backed PDDL scenario generator.
mfja_robot_control_config/scripts/room_315_pddl_validation_gate.py	Yes	No	Strict validation gate for Room 315 PDDL/PlanSys2 VLA scenarios.
mfja_robot_control_config/scripts/room_315_presence_provider.py	Yes	Yes	Deterministic Room 315 shuttle-presence providers.
mfja_robot_control_config/scripts/room_315_runtime_contracts.py	Yes	No	Shared fail-closed contracts for the Room 315 runtime boundary.
mfja_robot_control_config/scripts/room_315_task_execution.py	Yes	Yes	ROS-independent state and transport core for Room 315 task execution.
mfja_robot_control_config/scripts/room_315_task_execution_config.py	Yes	No	Authoritative defaults and validation for Room 315 task execution.

File	CMake-listed	Executable bit	Primary responsibility
mfja_robot_control_config/scripts/room_315_task_execution_node.py	Yes	Yes	ROS 2 gateway from confirmed Room 315 TaskGoal to closed-loop execution.
mfja_robot_control_config/scripts/room_315_visual_observed_state_provider.py	Yes	No	Visual ObservedState provider for Room 315.

D.2.6 VLA supervision, recording, and operations

Table D.7: Python modules: VLA supervision, recording, and operations

File	CMake-listed	Executable bit	Primary responsibility
mfja_robot_control_config/scripts/room_315_payload_case_batch_runner.py	Yes	Yes	Run Room 315 payload training cases end-to-end.
mfja_robot_control_config/scripts/room_315_payload_case_review.py	Yes	Yes	Create a human review pack for Room 315 payload training cases.
mfja_robot_control_config/scripts/room_315_vla_benchmark_suite.py	Yes	Yes	Seeded Room 315 benchmark case and method-comparison utilities.
mfja_robot_control_config/scripts/room_315_vla_dataset_recorder.py	Yes	Yes	Records synchronized camera, task, action, and privileged evaluation data into guarded episodes.
mfja_robot_control_config/scripts/room_315_vla_event_extractor.py	Yes	Yes	Extracts event-level VLA examples from recorded Room 315 episodes.
mfja_robot_control_config/scripts/room_315_vla_obstacle_tool.py	Yes	Yes	Move the Room 315 visual obstacle markers in Gazebo.
mfja_robot_control_config/scripts/room_315_vla_shuttle_identity_tracker.py	Yes	No	Privileged Room 315 shuttle identity tracker.
mfja_robot_control_config/scripts/room_315_vla_split_dataset.py	Yes	Yes	Creates deterministic, leakage-aware train/validation dataset partitions.
mfja_robot_control_config/scripts/room_315_vla_supervisor.py	Yes	Yes	Deterministic safety supervisor that validates primitives and alone publishes typed rail commands.
mfja_robot_control_config/scripts/room_315_vla_to_robformat.py	Yes	Yes	Exports the repository episode representation to the optional LeRobot format.

D.2.7 Qualification and immutable runtime packaging

Table D.8: Python modules: Qualification and immutable runtime packaging

File	CMake-listed	Executable bit	Primary responsibility
mfja_robot_control_config/scripts/room_315_authorize_runtime_v4_closed_loop.py	Yes	Yes	Create immutable V4 Gazebo closed-loop qualification/runtime bundles.
mfja_robot_control_config/scripts/room_315_build_runtime_candidate.py	Yes	No	Build the immutable corrected Experiment-A Full historical V3 archive.
mfja_robot_control_config/scripts/room_315_build_runtime_candidate_v4.py	Yes	No	Build the immutable, shadow-only Room 315 V4 runtime candidate.
mfja_robot_control_config/scripts/room_315_promote_runtime_v4.py	Yes	Yes	Create a manually approved, immutable Room 315 V4 active bundle.
mfja_robot_control_config/scripts/room_315_runtime_acceptance_campaign_v4.py	Yes	Yes	Run the seven V4 Room 315 runtime-acceptance scenarios safely.
mfja_robot_control_config/scripts/room_315_runtime_acceptance_readiness.py	Yes	Yes	Fail-closed readiness gates for Room 315 Gazebo runtime acceptance.
mfja_robot_control_config/scripts/room_315_runtime_acceptance_recorder.py	Yes	Yes	Record one guarded Gazebo runtime-acceptance scenario without controlling it.
mfja_robot_control_config/scripts/room_315_runtime_acceptance_report.py	Yes	Yes	Build a non-approving Room 315 runtime acceptance report.
mfja_robot_control_config/scripts/room_315_v4_closed_loop_fault_campaign.py	Yes	Yes	Qualify fail-closed Room 315 V4 behaviour in Gazebo, never on hardware.

D.3 Internal Python dependency hot spots

The following modules have the largest number of distinct in-repository importers. Treat them as high-impact foundations: a contract change here normally requires a broad regression run.

Table D.9: Internal Python dependency hot spots

Module	Importers	Maintenance significance
mfja_robot_control_config/scripts/room_315_visual_state_dataset.py	24	Shared contract/data/runtime foundation; run consumer tests.
mfja_robot_control_config/scripts/room_315_multi_shuttle.py	18	Shared contract/data/runtime foundation; run consumer tests.
mfja_robot_control_config/scripts/room_315_rail_defaults.py	17	Shared contract/data/runtime foundation; run consumer tests.

Module	Importers	Maintenance significance
mfja_robot_control_config/scripts/room_315_visual_v3_common.py	17	Shared contract/data/runtime foundation; run consumer tests.
mfja_robot_control_config/scripts/room_315_visual_scenario_generator.py	13	Shared contract/data/runtime foundation; run consumer tests.
mfja_robot_control_config/scripts/room_315_visual_fleet.py	12	Shared contract/data/runtime foundation; run consumer tests.
mfja_robot_control_config/scripts/room_315_contracts.py	11	Shared contract/data/runtime foundation; run consumer tests.
mfja_robot_control_config/scripts/room_315_presence_provider.py	9	Shared contract/data/runtime foundation; run consumer tests.
mfja_robot_control_config/scripts/room_315_visual_v3r1_common.py	7	Shared contract/data/runtime foundation; run consumer tests.
mfja_robot_control_config/scripts/room_315_kinematic_shuttle.py	6	Shared contract/data/runtime foundation; run consumer tests.
mfja_robot_control_config/scripts/room_315_json_io.py	6	Shared contract/data/runtime foundation; run consumer tests.
mfja_robot_control_config/scripts/room_315_pddl_scenario_generator.py	6	Shared contract/data/runtime foundation; run consumer tests.
mfja_robot_control_config/scripts/room_315_task_goal_schema.py	6	Shared contract/data/runtime foundation; run consumer tests.
mfja_robot_control_config/scripts/room_315_visual_model_v4.py	5	Shared contract/data/runtime foundation; run consumer tests.
mfja_robot_control_config/scripts/room_315_runtime_contracts.py	5	Shared contract/data/runtime foundation; run consumer tests.
mfja_robot_control_config/scripts/room_315_task_goal_semantic.py	5	Shared contract/data/runtime foundation; run consumer tests.
mfja_robot_control_config/scripts/room_315_observed_state_provider.py	4	Shared contract/data/runtime foundation; run consumer tests.

Module	Importers	Maintenance significance
mfja_robot_control_config/scripts/room_315_visual_runtime_validation.py	4	Shared contract/data/runtime foundation; run consumer tests.

D.4 Additional package source modules

This closes the source-code inventory outside the main control-script, launch, and test references: two compiled Gazebo plugins and eight isolated Experiment-A helpers. The experiment helpers are not installed as ROS executables.

Table D.10: Compiled plugins and isolated Experiment-A source modules

Tracked source	Primary responsibility
mfja_3rd_floor_description/src/SmoothJointTrajectoryController.cc	Gazebo system implementing validated, retimed, deterministic joint-trajectory motion.
mfja_3rd_floor_description/src/SymmetricGripperController.cc	Gazebo system implementing bounded, symmetric, motion-only two-jaw gripper animation.
mfja_robot_control_config/experiment_a_v3r1/experiment_a_core.py	Fail-closed contracts and deterministic sampling for Room 315 Experiment A.
mfja_robot_control_config/experiment_a_v3r1/experiment_a_guard.py	Atomic three-stage authorization guard for Experiment A.
mfja_robot_control_config/experiment_a_v3r1/experiment_a_local.py	Read-only RTX memory preflight for local Experiment-A execution.
mfja_robot_control_config/experiment_a_v3r1/experiment_a_prepare_inputs.py	Stage only authorized Experiment-A inputs for transfer to Kairos.
mfja_robot_control_config/experiment_a_v3r1/experiment_a_smoke_v2.py	Balanced, baseline-first local Smoke V2 for Room 315 Experiment A.
mfja_robot_control_config/experiment_a_v3r1/experiment_a_smoke_v2_guard.py	Single-attempt guard for the isolated local Smoke V2 run.
mfja_robot_control_config/experiment_a_v3r1/experiment_a_train.py	Continuation trainer and post-training Canary evaluator for Experiment A.
mfja_robot_control_config/experiment_a_v3r1/experiment_a_verify.py	Static and runtime fail-closed verification for Experiment A.

D.5 Launch files

Table D.11: Launch files and responsibilities

Launch source	Role
mfja_3rd_floor_bringup/launch/full_floor.launch.py	Stable full-floor wrapper around the shared floor composition
mfja_3rd_floor_bringup/launch/room_315_floor_common.py	Top-level profile defaults, singleton lock, argument forwarding, and delayed includes
mfja_3rd_floor_bringup/launch/room_315_only.launch.py	Stable Room 315-only wrapper around the shared floor composition
mfja_3rd_floor_bringup/launch/single_industrial_robot.launch.py	Stable isolated industrial-robot wrapper

Launch source	Role
mfja_robot_control_config/launch/gripper_range_config.py	Launch-time gripper config validation and temporary SDF/URDF materialization
mfja_robot_control_config/launch/isolated_industrial_robot.launch.py	Lower-level one-industrial-robot Gazebo composition
mfja_robot_control_config/launch/multi_robot_sim.launch.py	Gazebo startup, robot selection/spawn, per-instance bridges, and state publishers
mfja_robot_control_config/launch/room_315_dual_kinematic_shuttles.launch.py	Right/left rail node instantiation and type-safe parameter forwarding
mfja_robot_control_config/launch/room_315_launch_utils.py	Shared world-name parsing helper
mfja_robot_control_config/launch/room_315_runtime_acceptance.launch.py	Guarded runtime-acceptance campaign composition
mfja_robot_control_config/launch/room_315_task_execution.launch.py	PlanSys2 lifecycle and fail-closed task gateway
mfja_robot_control_config/launch/room_315_visual_state_runtime.launch.py	V4 visual inference and optional RGB bridge
mfja_robot_control_config/launch/room_315_vla_supervisor.launch.py	Camera bridge, primitive supervisor, and optional recorder

D.6 Configuration and data contracts

Table D.12: Configuration files and data contracts

Source file	Role
mfja_robot_control_config/config/gazebo_params.yaml	Installed legacy Gazebo parameter file; no current runtime consumer
mfja_robot_control_config/config/gripper_command_defaults.yaml	Per-robot gripper endpoints and percentage defaults
mfja_robot_control_config/config/mfja_default_gui.config	Gazebo GUI layout/profile
mfja_robot_control_config/config/mfja_light_gui.config	Gazebo GUI layout/profile
mfja_robot_control_config/config/robots.yaml	Full-floor robot registry, spawn pose, model, and enable flags
mfja_robot_control_config/config/robots_room_315_only.yaml	Room-only robot registry and defaults
mfja_robot_control_config/config/room315_runtime_safe_gui.config	Gazebo GUI layout/profile
mfja_robot_control_config/config/room_315_kinematics/README.md	Package-local configuration guidance
mfja_robot_control_config/config/room_315_kinematics/SEGMENT_CSV_MIGRATION.md	Package-local configuration guidance

Source file	Role
mfja_robot_control_config/config/room_315_kinematics/rail_devices_left.yaml	Left slots, binary sensors, and stoppers
mfja_robot_control_config/config/room_315_kinematics/rail_devices_right.yaml	Right slots, binary sensors, and stoppers
mfja_robot_control_config/config/room_315_kinematics/rail_network_left.yaml	Left-rail directed topology and public switch mapping
mfja_robot_control_config/config/room_315_kinematics/rail_network_right.yaml	Right-rail directed topology and routing
mfja_robot_control_config/config/room_315_kinematics/raw_segments/A12E.csv	Calibrated directed rail-segment samples (index, x, y, z)
mfja_robot_control_config/config/room_315_kinematics/raw_segments/A12I.csv	Calibrated directed rail-segment samples (index, x, y, z)
mfja_robot_control_config/config/room_315_kinematics/raw_segments/A14.csv	Calibrated directed rail-segment samples (index, x, y, z)
mfja_robot_control_config/config/room_315_kinematics/raw_segments/A1E.csv	Calibrated directed rail-segment samples (index, x, y, z)
mfja_robot_control_config/config/room_315_kinematics/raw_segments/A1I.csv	Calibrated directed rail-segment samples (index, x, y, z)
mfja_robot_control_config/config/room_315_kinematics/raw_segments/A23.csv	Calibrated directed rail-segment samples (index, x, y, z)
mfja_robot_control_config/config/room_315_kinematics/raw_segments/A2E.csv	Calibrated directed rail-segment samples (index, x, y, z)
mfja_robot_control_config/config/room_315_kinematics/raw_segments/A2I.csv	Calibrated directed rail-segment samples (index, x, y, z)
mfja_robot_control_config/config/room_315_kinematics/raw_segments/A34E.csv	Calibrated directed rail-segment samples (index, x, y, z)
mfja_robot_control_config/config/room_315_kinematics/raw_segments/A34I.csv	Calibrated directed rail-segment samples (index, x, y, z)
mfja_robot_control_config/config/room_315_kinematics/raw_segments/A3E.csv	Calibrated directed rail-segment samples (index, x, y, z)
mfja_robot_control_config/config/room_315_kinematics/raw_segments/A3I.csv	Calibrated directed rail-segment samples (index, x, y, z)
mfja_robot_control_config/config/room_315_kinematics/raw_segments/A4E.csv	Calibrated directed rail-segment samples (index, x, y, z)
mfja_robot_control_config/config/room_315_kinematics/raw_segments/A4I.csv	Calibrated directed rail-segment samples (index, x, y, z)
mfja_robot_control_config/config/room_315_visual_state/README.md	Package-local configuration guidance
mfja_robot_control_config/config/room_315_visual_state/blocker_training_scenarios.yaml	Scenario or benchmark manifest
mfja_robot_control_config/config/room_315_visual_state/training_scenarios.yaml	Scenario or benchmark manifest
mfja_robot_control_config/config/room_315_vla/payload_scenarios.yaml	Scenario or benchmark manifest
mfja_robot_control_config/config/room_315_vla/payload_training_cases_expanded_160_speed_sweep.yaml	Scenario or benchmark manifest
mfja_robot_control_config/config/room_315_vla/pddl/domain_room315.pddl	Expert/data-generation symbolic domain
mfja_robot_control_config/config/room_315_vla/pddl/domain_room315_runtime.pddl	Runtime PlanSys2 symbolic domain
mfja_robot_control_config/config/room_315_vla/shuttle_identity.yaml	Canonical identity and visual marker registry
mfja_robot_control_config/config/room_315_vla/task_execution_runtime.yaml	Default fail-closed task-execution parameters
mfja_robot_control_config/config/room_315_vla/task_execution_runtime_v4.yaml	Versioned V4 qualification task-execution snapshot
mfja_robot_control_config/config/room_315_vla/task_goal_english_benchmark.yaml	Offline TaskGoal benchmark corpus

Source file	Role
mfja_robot_control_config/config/room_315_vla/task_goal_understanding.yaml	English TaskGoal parser and local-model policy
mfja_robot_control_config/config/room_315_vla/visual_observed_state_calibration.yaml	Visual observation calibration and validation policy
mfja_robot_control_config/config/room_315_vla/visual_state_experiment_a_dataset_v3.yaml	Versioned experiment/dataset contract
mfja_robot_control_config/config/room_315_vla/visual_state_experiment_a_dataset_v3r1.yaml	Versioned experiment/dataset contract
mfja_robot_control_config/config/room_315_vla/visual_state_final_test_v4.json	Sealed V4 final-test preparation contract
mfja_robot_control_config/config/room_315_vla/visual_state_final_test_v4_coverage_compat_v1.json	Versioned runtime, dataset, or evaluation configuration
mfja_robot_control_config/config/room_315_vla/visual_state_final_test_v4_coverage_extension.json	Versioned runtime, dataset, or evaluation configuration
mfja_robot_control_config/config/room_315_vla/visual_state_runtime.yaml	Active/shadow visual runtime and artifact guard defaults
mfja_robot_control_config/config/room_315_vla/visual_state_training_v4.json	V4 training, split, loss, and evaluation contract
mfja_robot_control_config/config/room_315_vla/visual_state_supervisor.yaml	Primitive supervisor defaults

D.7 Gazebo models

Table D.13: Gazebo model directories

Model directory	Role	Tracked files
mfja_3rd_floor_description/models/carter_droit	Room 315 rail-cell CAD/visual asset	3
mfja_3rd_floor_description/models/carter_gauche	Room 315 rail-cell CAD/visual asset	3
mfja_3rd_floor_description/models/cell_path_left	Room 315 rail-cell CAD/visual asset	3
mfja_3rd_floor_description/models/cell_path_right	Room 315 rail-cell CAD/visual asset	3
mfja_3rd_floor_description/models/cell_static_droit_final	Room 315 rail-cell CAD/visual asset	3
mfja_3rd_floor_description/models/cell_static_gauche_final	Room 315 rail-cell CAD/visual asset	3
mfja_3rd_floor_description/models/ground_plane	Reusable Gazebo environment model	5
mfja_3rd_floor_description/models/kuka_kr6r900sixx	KUKA KR6 R900 Sixx visual robot model	22
mfja_3rd_floor_description/models/kuka_table	Robot/laboratory table asset	3
mfja_3rd_floor_description/models/lab_chair	Reusable Gazebo environment model	3
mfja_3rd_floor_description/models/lab_table	Robot/laboratory table asset	3
mfja_3rd_floor_description/models/lab_table_240cm	Robot/laboratory table asset	3
mfja_3rd_floor_description/models/mfja_3rd_floor	Complete third-floor environment asset	3

Model directory	Role	Tracked files
mfja_3rd_floor_description /models/rail_switch_3pos_d roit	Animated three-position rail-switch model	5
mfja_3rd_floor_description /models/rail_switch_3pos_d roit_static_black	Neutral static rail-switch model	2
mfja_3rd_floor_description /models/room115_closet115	Reusable room furniture or environment asset	3
mfja_3rd_floor_description /models/room115_table2000	Reusable room furniture or environment asset	3
mfja_3rd_floor_description /models/room115_tool_set	Reusable room furniture or environment asset	3
mfja_3rd_floor_description /models/room314_charging_e rgotron	Reusable room furniture or environment asset	3
mfja_3rd_floor_description /models/room314_closet1	Reusable room furniture or environment asset	3
mfja_3rd_floor_description /models/room314_table_180c m	Reusable room furniture or environment asset	3
mfja_3rd_floor_description /models/room314_tiangobox	Reusable room furniture or environment asset	3
mfja_3rd_floor_description /models/room315_shuttle	Generic shuttle template	3
mfja_3rd_floor_description /models/room315_shuttle_L1	Identity-specific preloaded shuttle visual asset	2
mfja_3rd_floor_description /models/room315_shuttle_L2	Identity-specific preloaded shuttle visual asset	2
mfja_3rd_floor_description /models/room315_shuttle_L3	Identity-specific preloaded shuttle visual asset	2
mfja_3rd_floor_description /models/room315_shuttle_L4	Identity-specific preloaded shuttle visual asset	2
mfja_3rd_floor_description /models/room315_shuttle_R1	Identity-specific preloaded shuttle visual asset	2
mfja_3rd_floor_description /models/room315_shuttle_R2	Identity-specific preloaded shuttle visual asset	2
mfja_3rd_floor_description /models/room315_shuttle_R3	Identity-specific preloaded shuttle visual asset	2
mfja_3rd_floor_description /models/room315_shuttle_R4	Identity-specific preloaded shuttle visual asset	2
mfja_3rd_floor_description /models/room315_vla_overhe ad_devices	Paired rail-focused RGB-D camera assembly	2
mfja_3rd_floor_description /models/room315_vla_payloa d_medium_box	VLA payload/occlusion visual asset	2
mfja_3rd_floor_description /models/room315_vla_payloa d_partial_marker_occluder	VLA payload/occlusion visual asset	2
mfja_3rd_floor_description /models/room315_vla_payloa d_small_box	VLA payload/occlusion visual asset	2

Model directory	Role	Tracked files
mfja_3rd_floor_description /models/room315_vla_payload_tall_box	VLA payload/occlusion visual asset	2
mfja_3rd_floor_description /models/room315_vla_payload_wide_box_within_keepout	VLA payload/occlusion visual asset	2
mfja_3rd_floor_description /models/room315_vla_removable_obstacle_marker	Removable VLA obstacle marker	2
mfja_3rd_floor_description /models/room_315	Detailed Room 315 room and rail-cell asset	3
mfja_3rd_floor_description /models/staubli_table	Robot/laboratory table asset	3
mfja_3rd_floor_description /models/staubli_tx2_60l	Stäubli TX2-60L visual robot model	32
mfja_3rd_floor_description /models/sun	Reusable Gazebo environment model	5
mfja_3rd_floor_description /models/table_yaskawa	Robot/laboratory table asset	3
mfja_3rd_floor_description /models/tiago	TIAGo mobile manipulator model	12
mfja_3rd_floor_description /models/tiago_base	TIAGo mobile-base variant	2
mfja_3rd_floor_description /models/yaskawa_hc10	Yaskawa HC10 visual robot model	19
mfja_3rd_floor_description /models/yaskawa_hc10dt	Yaskawa HC10DT visual robot model	21

D.8 Automated tests

This reference lists source tests, not only CMake-registeredament tests. Use direct pytest for the complete source surface and colcon test for the installed package surface.

D.8.1 mfja_3rd_floor_description

Table D.14: Test modules: mfja_3rd_floor_description

Test file	Family	Intent
mfja_3rd_floor_description/test/test_kuka_initial_joint_positions.py	General regression	Regression coverage for kuka initial joint positions.
mfja_3rd_floor_description/test/test_lightweight_world_physics.py	General regression	Regression coverage for lightweight world physics.
mfja_3rd_floor_description/test/test_room315_shuttle_collision_free.py	Rail and fleet safety	Regression coverage for shuttle collision free.
mfja_3rd_floor_description/test/test_industrial_gripper_articulation.py	Robot and model control	Regression coverage for industrial gripper articulation.
mfja_3rd_floor_description/test/test_industrial_robot_collision_free.py	Robot and model control	Regression coverage for industrial robot collision free.
mfja_3rd_floor_description/test/test_tiago_smooth_joint_control.py	Robot and model control	Regression coverage for tiago smooth joint control.
mfja_3rd_floor_description/test/test_room315_vla_camera_model.py	Visual/VLA model and runtime	Regression coverage for vla camera model.

D.8.2 mfja_robot_control_config

Table D.15: Test modules: mfja_robot_control_config

Test file	Family	Intent
mfja_robot_control_config/test/test_room315_contracts.py	Contracts and deployment gates	Regression coverage for contracts.
mfja_robot_control_config/test/test_room315_promote_runtime_v4.py	Contracts and deployment gates	Regression coverage for promote runtime v4.
mfja_robot_control_config/test/test_room315_runtime_acceptance.py	Contracts and deployment gates	Regression coverage for runtime acceptance.
mfja_robot_control_config/test/test_room315_runtime_acceptance_campaign_v4.py	Contracts and deployment gates	Regression coverage for runtime acceptance campaign v4.
mfja_robot_control_config/test/test_room315_runtime_candidate.py	Contracts and deployment gates	Regression coverage for runtime candidate.
mfja_robot_control_config/test/test_room315_runtime_candidate_v4.py	Contracts and deployment gates	Regression coverage for runtime candidate v4.
mfja_robot_control_config/test/test_room315_runtime_contracts.py	Contracts and deployment gates	Regression coverage for runtime contracts.
mfja_robot_control_config/test/test_room315_arbitrary_subset_capture_v2.py	Dataset generation and integrity	Regression coverage for arbitrary subset capture v2.
mfja_robot_control_config/test/test_room315_dataset_partition_overlap_post_hoc.py	Dataset generation and integrity	Regression coverage for dataset partition overlap post hoc.
mfja_robot_control_config/test/test_room315_grouped_visual_splits.py	Dataset generation and integrity	Regression coverage for grouped visual splits.
mfja_robot_control_config/test/test_room315_visual_dataset_audit.py	Dataset generation and integrity	Regression coverage for visual dataset audit.
mfja_robot_control_config/test/test_room315_visual_label_exporter.py	Dataset generation and integrity	Regression coverage for visual label exporter.
mfja_robot_control_config/test/test_room315_visual_scenario_generator.py	Dataset generation and integrity	Regression coverage for visual scenario generator.
mfja_robot_control_config/test/test_room315_visual_scenario_runner.py	Dataset generation and integrity	Regression coverage for visual scenario runner.
mfja_robot_control_config/test/test_room315_visual_state_capture.py	Dataset generation and integrity	Regression coverage for visual state capture.
mfja_robot_control_config/test/test_room315_visual_v3_dataset.py	Dataset generation and integrity	Regression coverage for visual v3 dataset.
mfja_robot_control_config/test/test_room315_visual_v3r1_dataset.py	Dataset generation and integrity	Regression coverage for visual v3r1 dataset.
mfja_robot_control_config/test/test_room315_vla_dataset_recorder.py	Dataset generation and integrity	Regression coverage for vla dataset recorder.
mfja_robot_control_config/test/test_presentation_left_slot3_kuka_slot2.py	General regression	Regression coverage for presentation left slot3 kuka slot2.
mfja_robot_control_config/test/test_room315_camera_bbox_semantics.py	General regression	Regression coverage for camera bbox semantics.
mfja_robot_control_config/test/test_room315_device_marker_lifecycle.py	General regression	Regression coverage for device marker lifecycle.
mfja_robot_control_config/test/test_room315_experiment_a_v3r1.py	General regression	Regression coverage for experiment a v3r1.
mfja_robot_control_config/test/test_room315_fleet_supervisor_safety.py	General regression	Regression coverage for fleet supervisor safety.
mfja_robot_control_config/test/test_room315_identity_keepout_zones.py	General regression	Regression coverage for identity keepout zones.
mfja_robot_control_config/test/test_room315_observed_state_provider.py	General regression	Regression coverage for observed state provider.
mfja_robot_control_config/test/test_room315_payload_models.py	General regression	Regression coverage for payload models.
mfja_robot_control_config/test/test_room315_payload_occlusion_metadata.py	General regression	Regression coverage for payload occlusion metadata.
mfja_robot_control_config/test/test_room315_segment_csv_normalization.py	General regression	Regression coverage for segment csv normalization.

Test file	Family	Intent
mfja_robot_control_config/test/test_room31_5_floor_runtime_singleton.py	Launch and package integration	Regression coverage for floor runtime singleton.
mfja_robot_control_config/test/test_room31_5_full_package_static.py	Launch and package integration	Regression coverage for full package static.
mfja_robot_control_config/test/test_room31_5_pddl_dataset_report.py	PDDL and topology planning	Regression coverage for pddl dataset report.
mfja_robot_control_config/test/test_room31_5_pddl_language_generator.py	PDDL and topology planning	Regression coverage for pddl language generator.
mfja_robot_control_config/test/test_room31_5_pddl_plan_translator.py	PDDL and topology planning	Regression coverage for pddl plan translator.
mfja_robot_control_config/test/test_room31_5_pddl_planner_backend.py	PDDL and topology planning	Regression coverage for pddl planner backend.
mfja_robot_control_config/test/test_room31_5_pddl_planning.py	PDDL and topology planning	Regression coverage for pddl planning.
mfja_robot_control_config/test/test_room31_5_pddl_runtime_input_hardening.py	PDDL and topology planning	Regression coverage for pddl runtime input hardening.
mfja_robot_control_config/test/test_room31_5_pddl_scenario_generator.py	PDDL and topology planning	Regression coverage for pddl scenario generator.
mfja_robot_control_config/test/test_room31_5_pddl_scenario_validation_gate.py	PDDL and topology planning	Regression coverage for pddl scenario validation gate.
mfja_robot_control_config/test/test_room31_5_route_topology.py	PDDL and topology planning	Regression coverage for route topology.
mfja_robot_control_config/test/test_room31_5_block_reservation_runtime.py	Rail and fleet safety	Regression coverage for block reservation runtime.
mfja_robot_control_config/test/test_room31_5_eight_shuttle_visual_pipeline.py	Rail and fleet safety	Regression coverage for eight shuttle visual pipeline.
mfja_robot_control_config/test/test_room31_5_headway_safety.py	Rail and fleet safety	Regression coverage for headway safety.
mfja_robot_control_config/test/test_room31_5_kinematic_shuttle_core.py	Rail and fleet safety	Regression coverage for kinematic shuttle core.
mfja_robot_control_config/test/test_room31_5_multi_shuttle_dataset_recorder.py	Rail and fleet safety	Regression coverage for multi shuttle dataset recorder.
mfja_robot_control_config/test/test_room31_5_multi_shuttle_dataset_report.py	Rail and fleet safety	Regression coverage for multi shuttle dataset report.
mfja_robot_control_config/test/test_room31_5_multi_shuttle_launch_args.py	Rail and fleet safety	Regression coverage for multi shuttle launch args.
mfja_robot_control_config/test/test_room31_5_multi_shuttle_pddl_planning.py	Rail and fleet safety	Regression coverage for multi shuttle pddl planning.
mfja_robot_control_config/test/test_room31_5_multi_shuttle_registry.py	Rail and fleet safety	Regression coverage for multi shuttle registry.
mfja_robot_control_config/test/test_room31_5_rail_devices.py	Rail and fleet safety	Regression coverage for rail devices.
mfja_robot_control_config/test/test_room31_5_rail_segment_names.py	Rail and fleet safety	Regression coverage for rail segment names.
mfja_robot_control_config/test/test_room31_5_shuttle_identity_config.py	Rail and fleet safety	Regression coverage for shuttle identity config.
mfja_robot_control_config/test/test_room31_5_shuttle_identity_tracker.py	Rail and fleet safety	Regression coverage for shuttle identity tracker.
mfja_robot_control_config/test/test_room31_5_shuttle_identity_visual_assets.py	Rail and fleet safety	Regression coverage for shuttle identity visual assets.
mfja_robot_control_config/test/test_room31_5_shuttle_payload_visuals.py	Rail and fleet safety	Regression coverage for shuttle payload visuals.
mfja_robot_control_config/test/test_room31_5_switch_feedback_authority.py	Rail and fleet safety	Regression coverage for switch feedback authority.
mfja_robot_control_config/test/test_gripper_range_launch.py	Robot and model control	Regression coverage for gripper range launch.
mfja_robot_control_config/test/test_mobile_robot_frame_isolation.py	Robot and model control	Regression coverage for mobile robot frame isolation.
mfja_robot_control_config/test/test_robot_gripper_bridge.py	Robot and model control	Regression coverage for robot gripper bridge.

Test file	Family	Intent
mfja_robot_control_config/test/test_robot_gripper_command.py	Robot and model control	Regression coverage for robot gripper command.
mfja_robot_control_config/test/test_robot_joint_command.py	Robot and model control	Regression coverage for robot joint command.
mfja_robot_control_config/test/test_room315_authorize_runtime_v4_closed_loop.py	Task understanding and execution	Regression coverage for authorize runtime v4 closed loop.
mfja_robot_control_config/test/test_room315_closed_loop_executive.py	Task understanding and execution	Regression coverage for closed loop executive.
mfja_robot_control_config/test/test_room315_closed_loop_executive_ros_integration.py	Task understanding and execution	Regression coverage for closed loop executive ros integration.
mfja_robot_control_config/test/test_room315_task_execution.py	Task understanding and execution	Regression coverage for task execution.
mfja_robot_control_config/test/test_room315_task_execution_authorization.py	Task understanding and execution	Regression coverage for task execution authorization.
mfja_robot_control_config/test/test_room315_task_execution_config.py	Task understanding and execution	Regression coverage for task execution config.
mfja_robot_control_config/test/test_room315_task_execution_singleton.py	Task understanding and execution	Regression coverage for task execution singleton.
mfja_robot_control_config/test/test_room315_task_goal_builder.py	Task understanding and execution	Regression coverage for task goal builder.
mfja_robot_control_config/test/test_room315_task_goal_semantic_gateway.py	Task understanding and execution	Regression coverage for task goal semantic gateway.
mfja_robot_control_config/test/test_room315_task_goal_validation.py	Task understanding and execution	Regression coverage for task goal validation.
mfja_robot_control_config/test/test_room315_v4_closed_loop_fault_campaign.py	Task understanding and execution	Regression coverage for v4 closed loop fault campaign.
mfja_robot_control_config/test/test_room315_arbitrary_subset_visual.py	Visual/VLA model and runtime	Regression coverage for arbitrary subset visual.
mfja_robot_control_config/test/test_room315_arbitrary_subset_visual_v2.py	Visual/VLA model and runtime	Regression coverage for arbitrary subset visual v2.
mfja_robot_control_config/test/test_room315_kairos_visual_training_package.py	Visual/VLA model and runtime	Regression coverage for kairos visual training package.
mfja_robot_control_config/test/test_room315_visual_acceptance_v4.py	Visual/VLA model and runtime	Regression coverage for visual acceptance v4.
mfja_robot_control_config/test/test_room315_visual_calibration_v4.py	Visual/VLA model and runtime	Regression coverage for visual calibration v4.
mfja_robot_control_config/test/test_room315_visual_contract_v4.py	Visual/VLA model and runtime	Regression coverage for visual contract v4.
mfja_robot_control_config/test/test_room315_visual_debug_colors.py	Visual/VLA model and runtime	Regression coverage for visual debug colors.
mfja_robot_control_config/test/test_room315_visual_error_analysis.py	Visual/VLA model and runtime	Regression coverage for visual error analysis.
mfja_robot_control_config/test/test_room315_visual_final_test_v4.py	Visual/VLA model and runtime	Regression coverage for visual final test v4.
mfja_robot_control_config/test/test_room315_visual_manual_review.py	Visual/VLA model and runtime	Regression coverage for visual manual review.
mfja_robot_control_config/test/test_room315_visual_model_v4.py	Visual/VLA model and runtime	Regression coverage for visual model v4.
mfja_robot_control_config/test/test_room315_visual_observed_state_provider.py	Visual/VLA model and runtime	Regression coverage for visual observed state provider.
mfja_robot_control_config/test/test_room315_visual_runtime.py	Visual/VLA model and runtime	Regression coverage for visual runtime.
mfja_robot_control_config/test/test_room315_visual_runtime_ros_integration.py	Visual/VLA model and runtime	Regression coverage for visual runtime ros integration.
mfja_robot_control_config/test/test_room315_visual_runtime_v4.py	Visual/VLA model and runtime	Regression coverage for visual runtime v4.
mfja_robot_control_config/test/test_room315_visual_runtime_v4_fault_injection.py	Visual/VLA model and runtime	Regression coverage for visual runtime v4 fault injection.
mfja_robot_control_config/test/test_room315_visual_shadow_compare.py	Visual/VLA model and runtime	Regression coverage for visual shadow compare.

Test file	Family	Intent
mfja_robot_control_config/test/test_room315_visual_training_v4.py	Visual/VLA model and runtime	Regression coverage for visual training v4.
mfja_robot_control_config/test/test_room315_visual_v4_final_test.py	Visual/VLA model and runtime	Regression coverage for visual v4 final test.
mfja_robot_control_config/test/test_room315_visual_v4_final_test_coverage_compat.py	Visual/VLA model and runtime	Regression coverage for visual v4 final test coverage compat.
mfja_robot_control_config/test/test_room315_visual_v4_final_test_coverage_extension.py	Visual/VLA model and runtime	Regression coverage for visual v4 final test coverage extension.
mfja_robot_control_config/test/test_room315_vla_benchmark_suite.py	Visual/VLA model and runtime	Regression coverage for vla benchmark suite.
mfja_robot_control_config/test/test_room315_vla_event_extractor.py	Visual/VLA model and runtime	Regression coverage for vla event extractor.
mfja_robot_control_config/test/test_room315_vla_model_input_boundary.py	Visual/VLA model and runtime	Regression coverage for vla model input boundary.
mfja_robot_control_config/test/test_room315_vla_supervisor.py	Visual/VLA model and runtime	Regression coverage for vla supervisor.
mfja_robot_control_config/test/test_room315_vla_train_v4.py	Visual/VLA model and runtime	Regression coverage for vla train v4.
mfja_robot_control_config/test/test_room315_vla_training_tools.py	Visual/VLA model and runtime	Regression coverage for vla training tools.

D.9 Non-package automation and frozen code

The following source-like files are tracked outside the ROS packages. They are not installed as ROS executables. Treat evidence snapshots as immutable and keep offline report tooling separated from the live control path.

Table D.16: Source-like automation outside the ROS package runtime

Tracked path	Classification
flake.nix	Auxiliary Nix development shell; it is not the complete ROS environment.
report/evidence/current_source_supplement_2026-08-07/mfja_robot_control_config/test/test_room315_floor_runtime_singleton.py	Frozen evidence reproduction, verification, evaluation, or figure helper.
report/evidence/current_source_supplement_2026-08-07/mfja_robot_control_config/test/test_room315_rail_segment_names.py	Frozen evidence reproduction, verification, evaluation, or figure helper.
report/evidence/room315_campaign_figures_2026-08-07/extract_b03_initial_final.py	Frozen evidence reproduction, verification, evaluation, or figure helper.
report/evidence/room315_integrated_campaign_v2/campaign_runner.py	Frozen evidence reproduction, verification, evaluation, or figure helper.
report/evidence/room315_language_semantic_evaluation_v3/runner.py	Frozen evidence reproduction, verification, evaluation, or figure helper.
report/evidence/room315_language_semantic_evaluation_v3/source_snapshot/room_315_task_goal_dialogue.py	Frozen evidence source snapshot; never edit or import it as live runtime code.
report/evidence/room315_language_semantic_evaluation_v3/source_snapshot/room_315_task_goal_fusion.py	Frozen evidence source snapshot; never edit or import it as live runtime code.
report/evidence/room315_language_semantic_evaluation_v3/source_snapshot/room_315_task_goal_parsers.py	Frozen evidence source snapshot; never edit or import it as live runtime code.
report/evidence/room315_language_semantic_evaluation_v3/source_snapshot/room_315_task_goal_schema.py	Frozen evidence source snapshot; never edit or import it as live runtime code.
report/evidence/room315_language_semantic_evaluation_v3/source_snapshot/room_315_task_goal_semantic.py	Frozen evidence source snapshot; never edit or import it as live runtime code.
report/evidence/room315_language_semantic_evaluation_v3/source_snapshot/room_315_task_goal_validation.py	Frozen evidence source snapshot; never edit or import it as live runtime code.
report/evidence/room315_v4_closed_loop_campaign_figures_2026-08-12/extract_b03_v4_initial_final.py	Frozen evidence reproduction, verification, evaluation, or figure helper.

Tracked path	Classification
report/evidence/room315_visual_v4_dataset_release_v1/scripts/download_release.sh	Frozen evidence reproduction, verification, evaluation, or figure helper.
report/evidence/room315_visual_v4_dataset_release_v1/scripts/reproduce_results.py	Frozen evidence reproduction, verification, evaluation, or figure helper.
report/evidence/room315_visual_v4_dataset_release_v1/scripts/verify_extracted_dataset.py	Frozen evidence reproduction, verification, evaluation, or figure helper.
report/evidence/room315_visual_v4_dataset_release_v1/scripts/verify_release.py	Frozen evidence reproduction, verification, evaluation, or figure helper.
report/evidence/room315_visual_v4_submission_2026-08-11/final_test/protocol/evaluator.py	Frozen evidence reproduction, verification, evaluation, or figure helper.
report/evidence/room315_visual_v4_submission_2026-08-11/final_test/protocol/test_evaluator.py	Frozen evidence reproduction, verification, evaluation, or figure helper.
report/scripts/generate_ch7_geometry_figures.py	Report figure-generation utility; outside the ROS runtime.
report/tools/run_room315_integrated_campaign_v4.py	Offline, hash-bound campaign or evaluation runner used by the report.
report/tools/run_room315_language_semantic_evaluation.py	Offline, hash-bound campaign or evaluation runner used by the report.
report/tools/test_run_room315_integrated_campaign.py	Offline regression test for a report campaign runner.